

```

"#22c55e", // Green
"#06b6d4", // Cyan
"#3b82f6", // Blue
"#8b5cf6", // Purple
"#ec4899", // Pink
"#ffffff", // White
];

export const STROKE_WIDTHS = [2, 4, 8, 14, 24];

export const DEFAULT_STYLE: StyleState = {
  toolInk: buildToolInk(),
  penColor: "#1e293b",
  penSize: 3,
  highlighterColor: "#fde047",
  highlighterSize: 24,
  markerColor: "#3b82f6",
  markerSize: 8,
  brushColor: "#1e293b",
  brushSize: 12,
  calligraphyColor: "#1e293b",
  calligraphySize: 6,
  pencilColor: "#64748b",
  pencilSize: 2,
  eraserSize: 24,
  eraserMode: "pixel",

  strokeColor: "#1e293b",
  fillColor: "transparent",
  fillStyle: "transparent",
  dashStyle: "solid",
  strokeWidth: 2,
  shapeStrokeColor: "#1e293b",
  shapeFillColor: "#3b82f6",

  lineColor: "#1e293b",
  lineWidth: 2,
  startArrowhead: "none",
  endArrowhead: "arrow",

  textColor: "#1e293b",
  fontSize: 18,
  fontFamily: FONT_FAMILIES[0].value,
  fontWeight: 400,
  textAlign: "left",
  stickyColor: "yellow",

  connectionLinkType: "relates",
  connectionRouting: "curved",
  connectionArrowStart: "none",
  connectionArrowEnd: "arrow",

  laserColor: "#ef4444",
  laserSize: 10,
  laserMode: "trail",
  laserFadeMs: 1500,
};

export function getToolColor(style: StyleState, tool: ToolType): string {
  if (isStrokeTool(tool)) {
    return style.toolInk?.[tool]?.color ?? DEFAULT_TOOL_INK[tool]?.color ?? "#1e293b";
  }
  switch (tool) {
    case "rectangle":
    case "ellipse":
    case "diamond":
    case "triangle":
    case "star":
    case "cloud": return style.shapeStrokeColor;
    case "line":
    case "arrow": return style.lineColor;
    case "text": return style.textColor;
    case "sticky": return STICKY_COLORS[style.stickyColor]?.bg ?? "#fef9c3";
    case "connect": return getLinkTypeColor(style.connectionLinkType);
    case "laser": return style.laserColor ?? DEFAULT_STYLE.laserColor;
    default: return style.strokeColor;
  }
}

```

```

export function getToolSize(style: StyleState, tool: ToolType): number {
  if (isStrokeTool(tool)) {
    return style.toolInk?.[tool]?.size ?? DEFAULT_TOOL_INK[tool]?.size ?? 4;
  }
  switch (tool) {
    case "eraser": return style.eraserSize;
    case "laser": return style.laserSize ?? DEFAULT_STYLE.laserSize;
    case "rectangle":
    case "ellipse":
    case "diamond":
    case "triangle":
    case "star":
    case "cloud":
    case "line":
    case "arrow": return style.strokeWidth;
    case "text":
    case "sticky": return style.fontSize;
    default: return style.strokeWidth;
  }
}

/**
 * Write colour and/or size for one tool in a SINGLE patch.
 *
 * Colour and size share the `toolInk` map, so two separate patches spread onto
 * one object would silently drop the first – the second `toolInk` wins. Anything
 * setting both at once (applying a pinned pen) must go through here.
 */
/**
 * The sticky colour a value names: a key ("pink") or the colour code
 * getToolColor returns for it ("fce7f3"). Reading a sticky's colour and
 * writing it straight back used to reset it to yellow, because the getter
 * returns codes and the setter only understood names.
 */
export function stickyColorKey(c: string): StickyColor | null {
  if (c in STICKY_COLORS) return c as StickyColor;
  const lc = c.toLowerCase();
  const hit = (Object.keys(STICKY_COLORS) as StickyColor[]).find((k) => STICKY_COLORS[k].bg.toLowerCase() === lc);
  return hit ?? null;
}

export function setToolInk(
  style: StyleState,
  tool: ToolType,
  patch: { color?: string; size?: number }
): Partial<StyleState> {
  if (isStrokeTool(tool)) {
    const current =
      style.toolInk?.[tool] ?? DEFAULT_TOOL_INK[tool] ?? { color: "#1e293b", size: 4 };
    return {
      toolInk: {
        ...style.toolInk,
        [tool]: {
          color: patch.color ?? current.color,
          size: patch.size ?? current.size,
        },
      },
    };
  }
}

const out: Partial<StyleState> = {};
if (patch.color !== undefined) {
  const c = patch.color;
  switch (tool) {
    case "rectangle":
    case "ellipse":
    case "diamond":
    case "triangle":
    case "star":
    case "cloud": Object.assign(out, { shapeStrokeColor: c, shapeFillColor: c, strokeColor: c }); break;
    case "line":
    case "arrow": Object.assign(out, { lineColor: c }); break;
    case "text": Object.assign(out, { textColor: c }); break;
    case "sticky": Object.assign(out, { stickyColor: stickyColorKey(c) ?? style.stickyColor }); break;
    case "laser": Object.assign(out, { laserColor: c }); break;
    default: Object.assign(out, { strokeColor: c });
  }
}

```

```

    }
  }
  if (patch.size !== undefined) {
    const n = patch.size;
    switch (tool) {
      case "eraser": Object.assign(out, { eraserSize: n }); break;
      case "laser": Object.assign(out, { laserSize: n }); break;
      case "text":
      case "sticky": Object.assign(out, { fontSize: n }); break;
      default: Object.assign(out, { strokeWidth: n });
    }
  }
  return out;
}

export function setToolColor(
  style: StyleState,
  tool: ToolType,
  color: string
): Partial<StyleState> {
  return setToolInk(style, tool, { color });
}

export function setToolSize(
  style: StyleState,
  tool: ToolType,
  size: number
): Partial<StyleState> {
  return setToolInk(style, tool, { size });
}

// =====
// Camera & Bounding Box & Document Tree
// =====

export interface Camera {
  x: number;
  y: number;
  zoom: number;
}

export const MIN_ZOOM = 0.05;
export const MAX_ZOOM = 8.0;

export interface BoundingBox {
  x: number;
  y: number;
  width: number;
  height: number;
}

export type ResizeHandle = "nw" | "n" | "ne" | "e" | "se" | "s" | "sw" | "w" | "rotate";

/** Paper style drawn behind a board's elements. */
export type BoardPattern = "dots" | "grid" | "lines" | "plain";

export interface BoardBackground {
  /** CSS colour, or undefined to follow the light/dark theme. */
  color?: string;
  pattern: BoardPattern;
  /** Spacing of the pattern in px. Undefined means the default (24). */
  size?: number;
}

/** Paper spacing limits. Below ~6px a pattern turns into visual noise; above
 * ~120px it stops reading as ruling at all. */
export const BOARD_PATTERN_MIN_SIZE = 6;
export const BOARD_PATTERN_MAX_SIZE = 120;
export const BOARD_PATTERN_DEFAULT_SIZE = 24;

/** Handy jumps, so the common paper sizes are one tap rather than a drag. */
export const BOARD_PATTERN_SIZE_PRESETS = [12, 24, 40, 64];

export const DEFAULT_BOARD_BACKGROUND: BoardBackground = {
  pattern: "dots",
  size: BOARD_PATTERN_DEFAULT_SIZE,
};

```

```

/**
 * Ready-made board colours. Deliberately low-saturation: the background sits
 * behind everything, and a vivid one makes ink and sticky notes unreadable.
 * Each has a dark-mode partner so a board stays legible in either theme.
 */
export const BOARD_BACKGROUNDS: Array<{ id: string; label: string; light: string; dark: string }> = [
  { id: "default", label: "Theme", light: "", dark: "" },
  { id: "paper", label: "Paper", light: "#fdfaf3", dark: "#1c1a17" },
  { id: "warm", label: "Warm", light: "#fbf3e8", dark: "#221c14" },
  { id: "mint", label: "Mint", light: "#eef8f2", dark: "#12201a" },
  { id: "sky", label: "Sky", light: "#eef4fd", dark: "#121a26" },
  { id: "lavender", label: "Lavender", light: "#f4f0fd", dark: "#1a1626" },
  { id: "rose", label: "Rose", light: "#fdf0f3", dark: "#241419" },
  { id: "slate", label: "Slate", light: "#f1f3f5", dark: "#16181b" },
];

export const BOARD_PATTERNS: Array<{ id: BoardPattern; label: string }> = [
  { id: "dots", label: "Dots" },
  { id: "grid", label: "Grid" },
  { id: "lines", label: "Lines" },
  { id: "plain", label: "Plain" },
];

// =====
// Notebooks (paged documents)
// =====

/**
 * A document is either the infinite canvas ProNotes started as, or a paged
 * notebook like GoodNotes / Notability.
 *
 * Keeping both is the point. GoodNotes users' loudest complaint is that its
 * fixed pages cost them the unlimited canvas; the canvas apps have no pages at
 * all. Here they are two kinds of the same document, sharing one tree, one
 * search, one graph, one backup.
 */
export type BoardKind = "canvas" | "notebook";

export type PageSizeId = "a4" | "a4-landscape" | "letter" | "letter-landscape" | "square";

/** Page dimensions in CSS pixels at 96 dpi. */
export const PAGE_SIZES: Record<PageSizeId, { label: string; width: number; height: number }> = {
  a4: { label: "A4", width: 794, height: 1123 },
  "a4-landscape": { label: "A4 landscape", width: 1123, height: 794 },
  letter: { label: "Letter", width: 816, height: 1056 },
  "letter-landscape": { label: "Letter landscape", width: 1056, height: 816 },
  square: { label: "Square", width: 900, height: 900 },
};

export const DEFAULT_PAGE_SIZE: PageSizeId = "a4";

/**
 * A page backed by a page of an imported PDF.
 *
 * Only the reference is stored. The bytes live in IndexedDB (see
 * lib/whiteboard/pdfstore.ts) and the picture is rasterised on demand –
 * rendering 90 lecture pages into the workspace would be tens of megabytes
 * against a ~5 MB localStorage budget, and the failure would arrive on save,
 * after an hour of annotating.
 */
export interface PdfPageRef {
  docId: string;
  /** 1-based, matching how PDFs and people number pages. */
  pageNumber: number;
}

export interface NotebookPage {
  id: string;
  elements: WhiteboardElement[];
  /** Set when this page shows a PDF page underneath your ink. */
  pdf?: PdfPageRef;
  /**
   * Starred pages surface in the Favourites tab of the page rail. In a
   * 90-page lecture notebook, "the page with the exam question on it" is
   * otherwise only findable by scrolling.
   */
  bookmarked?: boolean;
}

```

```

/**
 * Per-page paper. Notability is criticised for not letting one document mix
 * templates; here each page carries its own, falling back to the notebook's.
 */
background?: BoardBackground;
createdAt: number;
updatedAt: number;
}

export interface Board {
  id: string;
  name: string;
  folderId: string | null;
  elements: WhiteboardElement[];
  camera: Camera;
  createdAt: number;
  updatedAt: number;
  tags?: string[];
  starred?: boolean;
  accessedAt?: number;
  color?: string;
  /** Per-board paper: colour + pattern. Undefined means the theme default. */
  background?: BoardBackground;

  /**
   * Undefined means "canvas" — every board saved before notebooks existed is
   * therefore still a valid canvas with no migration needed.
   */
  kind?: BoardKind;
  /** Notebooks only. The canvas keeps using `elements` above. */
  pages?: NotebookPage[];
  activePageId?: string;
  pageSize?: PageSizeId;
  /**
   * Exact sheet size for a notebook made from a PDF, since a PDF page is
   * rarely exactly A4 and letterboxing a lecture slide wastes the screen.
   * Overrides `pageSize` when present.
   */
  customPageSize?: { width: number; height: number };
  /** PDFs this notebook draws from, so unused ones can be pruned. */
  pdfDocIds?: string[];
}

/**
 * A reusable page: the elements and paper of a page you saved, or an imported
 * image laid onto a blank sheet. GoodNotes calls this the template library and
 * lets any PDF page or image become one.
 */
export interface PageTemplate {
  id: string;
  name: string;
  createdAt: number;
  /** Page size the template was designed at, so it can be scaled to fit. */
  width: number;
  height: number;
  background?: BoardBackground;
  elements: WhiteboardElement[];
}

export interface Folder {
  id: string;
  name: string;
  parentId: string | null;
  expanded: boolean;
  createdAt: number;
  color?: string;
}

export const FOLDER_COLORS = [
  "#3b82f6", // blue
  "#ef4444", // red
  "#f59e0b", // amber
  "#10b981", // green
  "#8b5cf6", // purple
  "#ec4899", // pink
  "#6b7280", // gray
];

```

```
// =====
// Touchscreen & Stylus Mode Types
// =====

import type { WheelMode } from "./input";

export type TouchInputMode = "touch-draw" | "stylus-only" | "touch-pan";

export type PressureCurve = "off" | "soft" | "standard" | "firm";

export interface TouchSettings {
  /**
   * Hold the stylus barrel button (or invert to the eraser end) to erase, then
   * release to get your pen back. Off switches the shortcut off entirely for
   * anyone whose pen reports the button by accident.
   */
  stylusButtonErases?: boolean;
  /**
   * Show the raw pointer-event readout. Off by default: it exists so a real
   * device can tell us what its stylus actually sends, because the spec-correct
   * reading of the barrel button does not hold on every Android WebView.
   */
  penDiagnostics?: boolean;
  /**
   * What the Select tool picks up (both the rectangle and the freeform lasso,
   * and Select All). Missing or empty means everything. It lives here because
   * this is the persisted input-preferences object, merged with defaults on
   * load, so adding it needed no migration.
   */
  selectFilter?: SelectCategory[];
  /** How a wheel event is interpreted – see lib/whiteboard/input.ts. */
  wheelMode: WheelMode;
  mode: TouchInputMode;
  palmRejection: boolean;
  twoFingerUndo: boolean;
  touchFeedback: boolean;
  largeHandles: boolean;

  // ■■ S Pen & Inking Engine Enhancements ■■
  /** Stroke smoothing & stabilization (0 to 100, default 50). */
  penSmoothing: number;
  /** Stylus pressure dynamic sensitivity (0 to 100, default 50 = the curve exactly). 0 = uniform line. */
  penPressureSensitivity: number;
  /** Stylus pressure response curve: "off" | "soft" | "standard" | "firm". */
  penPressureCurve: PressureCurve;
  /** Prune trailing capacitive lift-off hooks when lifting pen nib. */
  penAntiHook: boolean;
  /** Draw-and-hold at the end of a stroke to auto-straighten lines / shapes. */
  penAutoStraighten: boolean;
  /** Show live S Pen hover dot previewing thickness and ink color. */
  penHoverIndicator: boolean;
  /**
   * Each pen's own writing feel. A pen without an entry uses the shared
   * values above, so nothing changes until a pen is tuned on its own.
   */
  penFeel?: Partial<Record<StrokeStyle, PenFeel>>;
}

export interface PenFeel {
  curve?: PressureCurve;
  smoothing?: number;
  sensitivity?: number;
}

/** The writing feel a pen actually uses: its own, else the shared one. */
export function penFeelFor(
  ts: Partial<TouchSettings>,
  style: string
): { curve: PressureCurve; smoothing: number; sensitivity: number } {
  const own = ts.penFeel?.[style as StrokeStyle] ?? {};
  return {
    curve: own.curve ?? ts.penPressureCurve ?? "standard",
    smoothing: own.smoothing ?? ts.penSmoothing ?? 50,
    sensitivity: own.sensitivity ?? ts.penPressureSensitivity ?? 50,
  };
}
```

```
// =====
// Selection filter
// =====
// GoodNotes and Notability let the lasso pick up only some kinds of thing –
// the usual reason being to grab your handwriting off a page without also
// dragging the typed text or the image beside it.

export type SelectCategory = "ink" | "text" | "shapes" | "images";

export const SELECT_CATEGORIES: SelectCategory[] = ["ink", "text", "shapes", "images"];

export function selectCategoryOf(el: { type: string }): SelectCategory | null {
  switch (el.type) {
    case "stroke":
      return "ink";
    case "text":
    case "sticky":
      return "text";
    case "image":
      return "images";
    case "rectangle":
    case "ellipse":
    case "diamond":
    case "triangle":
    case "star":
    case "cloud":
    case "line":
    case "arrow":
      return "shapes";
    default:
      return null; // connections and anything unknown are never swept up
  }
}

/**
 * The effective filter. Anything malformed – a missing field, an empty list,
 * a value from a newer build this one does not know – falls back to
 * "everything", because a filter that silently selects nothing looks exactly
 * like the Select tool being broken.
 */
export function effectiveSelectFilter(saved: unknown): Set<SelectCategory> {
  if (!Array.isArray(saved)) return new Set(SELECT_CATEGORIES);
  const known = saved.filter((c): c is SelectCategory =>
    (SELECT_CATEGORIES as string[]).includes(c as string)
  );
  return new Set(known.length ? known : SELECT_CATEGORIES);
}

/**
 * Whether a selection gesture may pick this element right now. Normally
 * locked things are out of reach; in locked-items mode ONLY they are, so
 * they can be found and unlocked without touching anything else.
 */
export function pickableNow(el: { locked?: boolean }, lockedMode: boolean): boolean {
  return lockedMode ? !!el.locked : !el.locked;
}

export function passesSelectFilter(el: { type: string }, filter: Set<SelectCategory>): boolean {
  const cat = selectCategoryOf(el);
  return cat !== null && filter.has(cat);
}

export const DEFAULT_TOUCH_SETTINGS: TouchSettings = {
  stylusButtonErases: true,
  penDiagnostics: false,
  wheelMode: "auto",
  mode: "touch-draw",
  palmRejection: false,
  twoFingerUndo: true,
  touchFeedback: true,
  largeHandles: true,
  penSmoothing: 50,
  penPressureSensitivity: 50,
  penPressureCurve: "standard",
  penAntiHook: true,
  penAutoStraighten: true,
  penHoverIndicator: true,
};
```

```
=====
// === END FILE: src/lib/whiteboard/types.ts ===
=====

=====
// === START FILE: src/lib/whiteboard/workspace.ts ===
// File 130 of 194 | Lines: 237 | Size: 8023 bytes
=====

/**
 * Workspace Backup / Restore / Merge
 * =====
 * One portable JSON document holding the WHOLE workspace (folders, boards,
 * elements, custom link types, pinned pens).
 *
 * This is the migration path off per-browser localStorage – and the same
 * document that gets pushed to and pulled from OneDrive, so the merge rules
 * here are what make two devices converge instead of overwriting each other.
 */

import { migrateState } from "../persistence";
import { allElements, getPages } from "../notebook";
import type { Board, Folder, PinnedBarPosition, PinnedPen, WhiteboardElement } from "../types";

/** Bumped only if the FILE format changes, independent of the store schema. */
export const WORKSPACE_FORMAT = 1;

export interface WorkspaceDoc {
  format: number;
  app: "pronotes";
  exportedAt: number;
  /** Which device wrote this – shown when reporting a sync conflict. */
  deviceLabel?: string;
  folders: Folder[];
  boards: Board[];
  activeBoardId: string | null;
  activeFolderId: string | null;
  customLinkTypes?: Array<{ id: string; name: string; color: string }>;
  pinnedPens?: PinnedPen[];
  pinnedBarPos?: PinnedBarPosition | null;
}

export interface WorkspaceSnapshot {
  folders: Folder[];
  boards: Board[];
  activeBoardId: string | null;
  activeFolderId: string | null;
  customLinkTypes?: Array<{ id: string; name: string; color: string }>;
  pinnedPens?: PinnedPen[];
  pinnedBarPos?: PinnedBarPosition | null;
}

export function buildWorkspaceDoc(
  snapshot: WorkspaceSnapshot,
  deviceLabel?: string
): WorkspaceDoc {
  return {
    format: WORKSPACE_FORMAT,
    app: "pronotes",
    exportedAt: Date.now(),
    deviceLabel,
    folders: snapshot.folders,
    boards: snapshot.boards,
    activeBoardId: snapshot.activeBoardId,
    activeFolderId: snapshot.activeFolderId,
    customLinkTypes: snapshot.customLinkTypes ?? [],
    pinnedPens: snapshot.pinnedPens ?? [],
    pinnedBarPos: snapshot.pinnedBarPos ?? null,
  };
}

export function serializeWorkspace(
  snapshot: WorkspaceSnapshot,
  deviceLabel?: string
```



```

): string {
    return JSON.stringify(buildWorkspaceDoc(snapshot, deviceLabel), null, 2);
}

export class WorkspaceParseError extends Error {}

/**
 * Parse a backup file. Anything structurally wrong is rejected with a message
 * a human can act on, rather than silently importing an empty workspace over
 * good data – restoring the WRONG file is unrecoverable without version control.
 */
export function parseWorkspace(json: string): WorkspaceSnapshot {
    let raw: unknown;
    try {
        raw = JSON.parse(json);
    } catch {
        throw new WorkspaceParseError("That file is not valid JSON.");
    }
    if (!raw || typeof raw !== "object") {
        throw new WorkspaceParseError("That file does not contain a workspace.");
    }
    const doc = raw as Partial<WorkspaceDoc>;
    if (doc.app !== "pronotes") {
        throw new WorkspaceParseError("That file was not exported from ProNotes.");
    }
    if (typeof doc.format !== "number" || doc.format > WORKSPACE_FORMAT) {
        throw new WorkspaceParseError(
            "That backup was made by a NEWER version of ProNotes. Update the app first."
        );
    }
    if (!Array.isArray(doc.boards)) {
        throw new WorkspaceParseError("That backup has no boards in it.");
    }

    // Route through the same hardening as a local load so a hand-edited or
    // truncated backup cannot brick the app.
    const migrated = migrateState({
        version: 1,
        folders: (doc.folders ?? []) as Folder[],
        boards: doc.boards as Board[],
        activeBoardId: doc.activeBoardId ?? null,
        activeFolderId: doc.activeFolderId ?? null,
        customLinkTypes: doc.customLinkTypes,
        pinnedPens: doc.pinnedPens,
        pinnedBarPos: doc.pinnedBarPos,
    });

    return {
        folders: migrated.folders,
        boards: migrated.boards,
        activeBoardId: migrated.activeBoardId,
        activeFolderId: migrated.activeFolderId,
        customLinkTypes: migrated.customLinkTypes,
        pinnedPens: migrated.pinnedPens,
        pinnedBarPos: migrated.pinnedBarPos,
    };
}

/**
 * Every image/PDF id still reachable: live boards, page templates (which
 * outlive the page they were made from), plus any extra boards the caller
 * passes – notably the trash undo buffer, whose pictures and PDFs must NOT
 * be reclaimed while Undo can still bring them back.
 *
 * Pure so the sidebar's delete-undo can be unit-tested without IndexedDB.
 */
export function collectReferencedAssets(
    boards: Board[],
    templates: Array<{ elements: WhiteboardElement[] }> = [],
    extraBoards: Board[] = []
): { pdfIds: Set<string>; imageIds: Set<string> } {
    const pdfIds = new Set<string>();
    const imageIds = new Set<string>();
    const scan = (b: Board) => {
        for (const p of getPages(b)) if (p.pdf?.docId) pdfIds.add(p.pdf.docId);
        for (const el of allElements(b)) {
            const src = (el as { src?: string }).src;
            if (typeof src === "string" && src.startsWith("asset:")) imageIds.add(src.slice(6));
        }
    };
    scan(boards[0]);
    for (const t of templates) {
        for (const e of t.elements) {
            const src = (e as { src?: string }).src;
            if (typeof src === "string" && src.startsWith("asset:")) imageIds.add(src.slice(6));
        }
    }
    for (const b of extraBoards) scan(b);
}

```

```

    }
  };
  for (const b of boards) scan(b);
  for (const b of extraBoards) scan(b);
  for (const t of templates) {
    for (const el of t.elements) {
      const src = (el as { src?: string }).src;
      if (typeof src === "string" && src.startsWith("asset:")) imageIds.add(src.slice(6));
    }
  }
  return { pdfIds, imageIds };
}

export interface MergeReport {
  added: number;
  updatedFromRemote: number;
  keptLocal: number;
  foldersAdded: number;
}

/**
 * Merge a remote workspace into the local one, per board, by `updatedAt`.
 *
 * Whole-document last-write-wins is what silently destroys work: edit board A
 * on the tablet and board B on the PC, and whichever syncs second erases the
 * other's board entirely. Merging per board means both survive; only a board
 * edited on BOTH devices has to pick a winner, and there the newer timestamp
 * wins and the loss is limited to that one board.
 */
export function mergeWorkspaces(
  local: WorkspaceSnapshot,
  remote: WorkspaceSnapshot
): { merged: WorkspaceSnapshot; report: MergeReport } {
  const report: MergeReport = {
    added: 0,
    updatedFromRemote: 0,
    keptLocal: 0,
    foldersAdded: 0,
  };

  const byId = new Map<string, Board>();
  for (const b of local.boards) byId.set(b.id, b);

  for (const remoteBoard of remote.boards) {
    const localBoard = byId.get(remoteBoard.id);
    if (!localBoard) {
      byId.set(remoteBoard.id, remoteBoard);
      report.added++;
      continue;
    }
    const localAt = localBoard.updatedAt ?? 0;
    const remoteAt = remoteBoard.updatedAt ?? 0;
    if (remoteAt > localAt) {
      byId.set(remoteBoard.id, remoteBoard);
      report.updatedFromRemote++;
    } else {
      report.keptLocal++;
    }
  }

  const folderById = new Map<string, Folder>();
  for (const f of local.folders) folderById.set(f.id, f);
  for (const f of remote.folders) {
    if (!folderById.has(f.id)) {
      folderById.set(f.id, f);
      report.foldersAdded++;
    }
  }

  // Union the link types by id – losing a custom type would leave connections
  // rendering as unlabelled grey lines on the device that lost it.
  const linkTypes = new Map<string, { id: string; name: string; color: string }>();
  for (const t of local.customLinkTypes ?? []) linkTypes.set(t.id, t);
  for (const t of remote.customLinkTypes ?? []) if (!linkTypes.has(t.id)) linkTypes.set(t.id, t);

  const boards = [...byId.values()];
  const activeStillExists = boards.some((b) => b.id === local.activeBoardId);

```

```

return {
  merged: {
    folders: [...folderById.values()],
    boards,
    // Keep looking at whatever THIS device was looking at; only fall back if
    // that board no longer exists after the merge.
    activeBoardId: activeStillExists ? local.activeBoardId : boards[0]?.id ?? null,
    activeFolderId: local.activeFolderId,
    customLinkTypes: [...linkTypes.values()],
    // Pens and bar position are per-device preferences, not shared documents.
    pinnedPens: local.pinnedPens,
    pinnedBarPos: local.pinnedBarPos,
  },
  report,
};
}

```

```

=====
// === END FILE: src/lib/whiteboard/workspace.ts ===
=====

```

```

=====
// === START FILE: src/lib/whiteboard/workspaceDb.ts ===
// File 131 of 194 | Lines: 233 | Size: 9269 bytes
=====

```

```

/**
 * Workspace database (IndexedDB)
 * =====
 * The workspace – every board and all of its ink – lives here, NOT in
 * localStorage.
 *
 * The measured problem: a tablet workspace grew past ~4 MB against
 * localStorage's ~5 MB per-origin cap, and EVERY save began failing with
 * QuotaExceededError (see the device logs: dozens of
 * "[whiteboard] Failed to save state" in a row). localStorage cannot be
 * raised – 5 MB is hard-wired – so the only fix is a store without that cap.
 *
 * This follows the same split the industry settled on (tldraw persists
 * documents to IndexedDB; Excalidraw moved scenes off localStorage for the
 * identical 5 MB failure in issue #711): bulk document data goes to
 * IndexedDB, whose quota is a share of free disk (gigabytes, not megabytes),
 * while localStorage keeps only tiny synchronous keys (a save heartbeat and
 * the legacy copy during migration).
 *
 * Design notes (all deliberate):
 * - Same single-snapshot semantics as before: one record, last-write-wins.
 *   Per-board incremental writes would be cheaper per save but introduce
 *   partial-write states (board A saved, board B not); atomic snapshots
 *   cannot half-save.
 * - The record stores BYTES (optionally gzipped), not the object tree, so
 *   IndexedDB's main-thread structured clone is a memcpy rather than a walk
 *   of millions of stroke points (web.dev's IndexedDB best-practice warning).
 * - gzip kicks in above 1 MB of JSON. Ink compresses ~85-90% (repetitive
 *   coordinates), so a 4 MB workspace stores as ~0.5 MB. Below the threshold
 *   the JSON is stored raw: faster, and still human-inspectable when
 *   debugging a device.
 * - The backend is injectable: production uses real IndexedDB, unit tests use
 *   an in-memory Map. No singletons, no ambient state except the save queue.
 * - navigator.storage.persist() is requested at boot so the OS does not treat
 *   these boards as evictable cache data (Capacitor docs call this out: on
 *   Android the persisted API is what protects IndexedDB).
 */

```

```

export const WORKSPACE_DB_NAME = "pronotes-workspace";
export const WORKSPACE_DB_VERSION = 1;
export const WORKSPACE_STORE = "kv";
export const WORKSPACE_RECORD_KEY = "workspace";
/** Payloads at or above this JSON size are gzipped before storing. */
export const GZIP_THRESHOLD_CHARS = 1_000_000;
/** Early-warning line: tell the user BEFORE writes start failing. */
export const STORAGE_PRESSURE_RATIO = 0.85;
/** localStorage heartbeat: ~50 bytes that keep cross-tab save detection alive. */
export const HEARTBEAT_KEY = "z-whiteboard-rev";

```

```

export interface WorkspaceRecord {
  v: 1;
}

```

```

    format: "raw" | "gzip";
    savedAt: number;
    data: string | Uint8Array;
  }

export interface WorkspaceBackend {
  load(): Promise<WorkspaceRecord | null>;
  save(rec: WorkspaceRecord): Promise<void>;
  clear(): Promise<void>;
}

function idbAvailable(): boolean {
  return typeof indexedDB !== "undefined";
}

export function workspaceDbAvailable(): boolean {
  return idbAvailable();
}

function openDb(): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    if (!idbAvailable()) {
      reject(new Error("This browser has no IndexedDB, so the workspace cannot be stored there."));
      return;
    }
    const req = indexedDB.open(WORKSPACE_DB_NAME, WORKSPACE_DB_VERSION);
    req.onupgradeneeded = () => {
      const db = req.result;
      if (!db.objectStoreNames.contains(WORKSPACE_STORE)) db.createObjectStore(WORKSPACE_STORE);
    };
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error ?? new Error("Could not open the workspace store"));
  });
}

function tx<T>(mode: IDBTransactionMode, run: (s: IDBObjectStore) => IDBRequest<T>): Promise<T> {
  return openDb().then(
    (db) =>
      new Promise<T>((resolve, reject) => {
        let settled = false;
        const done = (fn: () => void) => {
          if (!settled) {
            settled = true;
            fn();
          }
        };
        try {
          const t = db.transaction(WORKSPACE_STORE, mode);
          t.oncomplete = () => db.close();
          t.onerror = () => db.close();
          const req = run(t.objectStore(WORKSPACE_STORE));
          req.onsuccess = () => done(() => resolve(req.result));
          req.onerror = () =>
            done(() => {
              try {
                db.close();
              } catch {
                /* closing is best-effort */
              }
              reject(req.error ?? new Error("Workspace store request failed"));
            });
        } catch (err) {
          try {
            db.close();
          } catch {
            /* closing is best-effort */
          }
          reject(err);
        }
      })
  );
}

/** Production backend: real IndexedDB. Thin – all decisions live elsewhere. */
export function createIdbWorkspaceBackend(): WorkspaceBackend {
  return {
    load: () => tx<WorkspaceRecord | undefined>("readonly", (s) => s.get(WORKSPACE_RECORD_KEY)).then((r) => r
      ?? null),
  };
}

```

```

    save: (rec) => tx<void>("readwrite", (s) => s.put(rec, WORKSPACE_RECORD_KEY) as unknown as IDBRequest<void>
      >).then(() => undefined),
    clear: () => tx<void>("readwrite", (s) => s.delete(WORKSPACE_RECORD_KEY) as unknown as IDBRequest<void>).t
      hen(() => undefined),
  };
}

/** Test backend: same interface over a Map. No quota, no async storage. */
export function createMemoryWorkspaceBackend(): WorkspaceBackend & { records: Map<string, WorkspaceRecord> } {
  const records = new Map<string, WorkspaceRecord>();
  return {
    records,
    load: () => Promise.resolve(records.get(WORKSPACE_RECORD_KEY) ?? null),
    save: (rec) => {
      records.set(WORKSPACE_RECORD_KEY, rec);
      return Promise.resolve();
    },
    clear: () => {
      records.delete(WORKSPACE_RECORD_KEY);
      return Promise.resolve();
    },
  };
}

function compressionAvailable(): boolean {
  return typeof CompressionStream !== "undefined" && typeof DecompressionStream !== "undefined";
}

async function gzipString(text: string): Promise<Uint8Array> {
  const stream = new Response(new Blob([text]).stream()).pipeThrough(new CompressionStream("gzip"));
  const buf = await stream.arrayBuffer();
  return new Uint8Array(buf);
}

async function gunzipBytes(bytes: Uint8Array): Promise<string> {
  // Slice the view: response over the whole backing buffer would decode
  // neighbouring bytes if this Uint8Array is itself a view.
  const copy = bytes.slice().buffer as ArrayBuffer;
  const stream = new Response(new Blob([copy]).stream()).pipeThrough(new DecompressionStream("gzip"));
  return stream.text();
}

/**
 * Serialized JSON -> storable record. Large payloads are gzipped; small ones
 * stay raw. Throws only if compression itself fails – callers treat that as
 * "store raw instead", never as lost data.
 */
export async function encodeWorkspace(json: string): Promise<WorkspaceRecord> {
  if (json.length >= GZIP_THRESHOLD_CHARS && compressionAvailable()) {
    const data = await gzipString(json);
    return { v: 1, format: "gzip", savedAt: Date.now(), data };
  }
  return { v: 1, format: "raw", savedAt: Date.now(), data: json };
}

/**
 * Storable record -> serialized JSON. Throws on structural problems or a
 * failed gunzip; callers fall back to the previous copy, never to nothing.
 */
export async function decodeWorkspace(rec: WorkspaceRecord): Promise<string> {
  if (!rec || typeof rec !== "object") throw new Error("Workspace record is not an object");
  if (rec.v !== 1) throw new Error(`Unsupported workspace record version ${rec as { v?: unknown }}.v`);
  if (rec.format === "raw") {
    if (typeof rec.data !== "string") throw new Error("Raw workspace record holds no string");
    return rec.data;
  }
  if (rec.format === "gzip") {
    if (!(rec.data instanceof Uint8Array)) throw new Error("Gzipped workspace record holds no bytes");
    if (!compressionAvailable()) throw new Error("Cannot gunzip without DecompressionStream");
    return gunzipBytes(rec.data);
  }
  throw new Error(`Unknown workspace record format ${rec as { format?: unknown }}.format`);
}

/**
 * Ask the OS to treat this origin's storage as persistent (not evictable
 * cache). Best-effort: resolves false when unsupported or denied. Must never
 * throw into boot.

```

```

*/
export async function requestPersistentStorage(): Promise<boolean> {
  try {
    const storage = (typeof navigator !== "undefined" && navigator.storage) || undefined;
    if (!storage || typeof storage.persist !== "function") return false;
    return await storage.persist();
  } catch {
    return false;
  }
}

export interface StoragePressure {
  usage: number;
  quota: number;
  ratio: number;
}

/** How full the origin's pooled storage is, or null when unmeasurable. */
export async function measureStoragePressure(): Promise<StoragePressure | null> {
  try {
    const storage = (typeof navigator !== "undefined" && navigator.storage) || undefined;
    if (!storage || typeof storage.estimate !== "function") return null;
    const { usage, quota } = await storage.estimate();
    if (typeof usage !== "number" || typeof quota !== "number" || quota <= 0) return null;
    return { usage, quota, ratio: usage / quota };
  } catch {
    return null;
  }
}

```

```

=====
// === END FILE: src/lib/whiteboard/workspaceDb.ts ===
=====

```

```

=====
// === START FILE: src/store/capture.ts ===
// File 132 of 194 | Lines: 55 | Size: 1958 bytes
=====

```

```

/**
 * Screenshot mode and the open capture, kept out of the main store.
 *
 * Screenshot mode is a switch on the SELECT tool: while it is on, the box or
 * loop you draw with Select takes a picture of that area instead of picking
 * up objects. It stays on until you switch it off, so several captures can be
 * taken in a row - and it only acts while Select (box) or Lasso is the tool,
 * so picking up the pen writes as always.
 *
 * Both are TRANSIENT on purpose: nothing here is persisted, so the app never
 * starts up with the select tool mysteriously refusing to select because a
 * mode was left on last week.
 */

```

```

import { create } from "zustand";
import type { CaptureShape, CaptureSource } from "@lib/whiteboard/capture";

```

```

export interface CaptureRequest {
  source: CaptureSource;
  /** Drawn over an area, or taken from the current selection. */
  origin: "area" | "selection";
  shape: CaptureShape;
  /** What was selected, for the panel's "selection only" choice. */
  selectionIds?: string[];
  /** The zoom and pixel ratio it was seen at, for the default sharpness. */
  zoom: number;
  dpr: number;
}

```

```

interface CaptureState {
  screenshotMode: boolean;
  request: CaptureRequest | null;
  setScreenshotMode: (on: boolean) => void;
  open: (request: CaptureRequest) => void;
  close: () => void;
}

```

```

export const useCapture = create<CaptureState>((set) => ({

```

```

    screenshotMode: false,
    request: null,
    setScreenshotMode: (on) => set({ screenshotMode: on }),
    open: (request) => set({ request }),
    close: () => set({ request: null }),
  }));

/**
 * The shape a press will capture with right now, or null when a press does
 * what it normally does. Screenshot mode only borrows the SELECT gestures.
 */
export function screenshotShapeFor(screenshotMode: boolean, tool: string): CaptureShape | null {
  if (!screenshotMode) return null;
  if (tool === "select") return "rectangle";
  if (tool === "lasso") return "lasso";
  return null;
}

=====
// === END FILE: src/store/capture.ts ===
=====

=====
// === START FILE: src/store/whiteboard.ts ===
// File 133 of 194 | Lines: 3538 | Size: 126991 bytes
=====

/**
 * Whiteboard Zustand Store (v3 – Obsidian Canvas & Vector Grade)
 * =====
 * Single source of truth for documents, canvas elements, history, and active editing state.
 */

import { create } from "zustand";
import { nanoid } from "nanoid";
import {
  getActivePageIndex as getActivePageIndexOf,
  addPage as nbAddPage,
  addPageBefore as nbAddPageBefore,
  togglePageBookmark as nbTogglePageBookmark,
  templateFromPage,
  insertTemplatePage,
  getPages,
  pageSizeOf,
  allElements,
  cloneBoardContent,
  deletePage as nbDeletePage,
  duplicatePage as nbDuplicatePage,
  getActivePage,
  getBoardElements,
  isNotebook,
  makeNotebook,
  mapAllBoardElements,
  mapBoardElements,
  movePage as nbMovePage,
  setActivePage,
  setActivePage as nbSetActivePage,
  setPageBackground as nbSetPageBackground,
  stepPage as nbStepPage,
  insertPageAt as nbInsertPageAt,
  copyPagesFrom,
  insertPages as nbInsertPages,
  pageFromPdf,
  resolvePageRange,
  createPage,
} from "../lib/whiteboard/notebook";
import { placeCaptureBeside, placeCaptureOnPage } from "../lib/whiteboard/placement";
import { PAGE_GAP as PAGE_GAP_STORE, minNotebookZoom } from "../lib/whiteboard/notebook";
import { collectReferencedAssets } from "../lib/whiteboard/workspace";
import { isAbandonedTextBox } from "../lib/whiteboard/textbox";
import {
  DEFAULT_STYLE,
  DEFAULT_TOUCH_SETTINGS,
  DEFAULT_PAGE_SIZE,
  DEFAULT_TOOLBAR_ITEMS,
  ALL_TOOLBAR_ITEMS,
  mergeToolbarItems,

```

```

    type ToolbarItemId,
    type Board,
    type BoardBackground,
    type PageSizeId,
    type PageTemplate,
    type Camera,
    type ConnectionElement,
    type ConnectionRouting,
    type Folder,
    type NodeAnchor,
    type DockPosition,
    type SplitOrientation,
    type PinnedBarPosition,
    type PinnedPen,
    type StyleState,
    type ToolType,
    type TouchSettings,
    type NotebookPage,
    type ImageElement,
    type WhiteboardElement,
  } from "../lib/whiteboard/types";
import {
  buildToolInk,
  getToolColor,
  getToolSize,
  isStrokeTool,
  setToolInk,
  STROKE_TOOL_IDS,
} from "../lib/whiteboard/types";
import { MIN_ZOOM, MAX_ZOOM, effectiveSelectFilter, passesSelectFilter } from "../lib/whiteboard/types";
import { resolveDictationLang } from "../lib/whiteboard/speechRecognition";
import { clamp, type Viewport, getElementBounds } from "../lib/whiteboard/geometry";
import { focusBounds } from "../lib/whiteboard/search";
import {
  loadState,
  loadWorkspace,
  scheduleSave,
  flushSave,
  watchExternalSaves,
  loadHistory,
  scheduleHistorySave,
  flushHistorySave,
  restorableHistory,
  HISTORY_PAST_MAX,
  HISTORY_FUTURE_MAX,
} from "../lib/whiteboard/persistence";
import { requestPersistentStorage, workspaceDbAvailable } from "../lib/whiteboard/workspaceDb";
import {
  mergeWorkspaces,
  type MergeReport,
  type WorkspaceSnapshot,
} from "../lib/whiteboard/workspace";

/**
 * Keeps a notebook's camera inside the document.
 *
 * A whiteboard is meant to be unbounded. A notebook is a finite stack of
 * sheets, and panning past the ends put you in an empty grey void with the
 * paper nowhere on screen and no way back except zoom-to-fit – measured on a
 * Galaxy Tab S9 as a full screen of nothing after a few swipes. That is what
 * "it goes to a very far place" was.
 *
 * Geometry: the ACTIVE page always sits at world origin, so page `i` sits at
 * `(i - activeIndex) * stride`. The document therefore runs from
 * `-(activeIndex * stride)` to `(lastIndex - activeIndex) * stride + height`.
 * Both bounds are expressed that way so they keep moving with the anchor and
 * never fight the re-anchoring that turns pages.
 *
 * The slack is deliberately generous – you must be able to pull the first page
 * down to a comfortable writing position – but finite.
 */
function boundCamera<T extends { boards: Board[]; activeBoardId: string | null }>(
  s: T,
  camera: Camera
): Camera {
  const board = s.boards.find((b) => b.id === s.activeBoardId);
  if (!isNotebook(board)) return camera;

```



```

const { width, height } = pageSizeOf(board);
const pages = getPages(board);
const count = Math.max(1, pages.length);
const activeIndex = Math.max(0, getActivePageIndexOf(board));
const stride = height + PAGE_GAP_STORE;

const xSlack = width * 0.5;
const ySlack = height * 0.5;
const top = -(activeIndex * stride) - ySlack;
const bottom = (count - 1 - activeIndex) * stride + height + ySlack;

return {
  ...camera,
  x: clamp(camera.x, -xSlack, width + xSlack),
  y: clamp(camera.y, top, bottom),
};
}

export interface PersistedState {
  version: number;
  folders: Folder[];
  boards: Board[];
  activeBoardId: string | null;
  activeFolderId: string | null;
  customLinkTypes?: Array<{ id: string; name: string; color: string }>;
  pinnedPens?: PinnedPen[];
  pinnedBarPos?: PinnedBarPosition | null;
  toolbarPosition?: DockPosition;
  pageTemplates?: PageTemplate[];
  splitBoardId?: string | null;
  splitRatio?: number;
  splitSide?: "left" | "right";
  splitOrientation?: SplitOrientation;
  recentColors?: string[];
  // Tool setup and preferences. Without these, every pen colour, size, font
  // size and the stylus mode reset to defaults on every launch.
  style?: Partial<StyleState>;
  tool?: ToolType;
  lastStrokeTool?: ToolType;
  touchSettings?: Partial<TouchSettings>;
  snapToGrid?: boolean;
  minimapOpen?: boolean;
  legendOpen?: boolean;
  zoomDockOpen?: boolean;
  shapeRecognition?: boolean;
  defaultPaper?: BoardBackground;
  pinnedBarOpen?: boolean;
  pinnedBarVertical?: boolean;
  propsPanelCollapsed?: boolean;
  /** Spoken language for voice dictation (BCP-47, e.g. "he-IL"). */
  dictationLang?: string;
  /** Samsung-style strict encircle: loops must close, neighbours only grazed stay out. */
  lassoStrict?: boolean;
  /** Tool properties row folded away behind the dock's chevron. */
  toolPropsBarCollapsed?: boolean;
  /** Saved as `toolbarTools`: lists saved under the first name still held the hand. */
  toolbarTools?: ToolbarItemId[];
  toolbarHidden?: ToolbarItemId[];
}

interface HistoryEntry {
  /**
   * Notebooks only. Undo must put the strokes back on the page they came
   * from — without this, undoing after turning the page would restore page 3's
   * content on top of page 7.
   */
  pageId?: string;
  elements: WhiteboardElement[];
  /** Full pages snapshot of a notebook for page structure changes (e.g. merge/insert). */
  pages?: NotebookPage[];
  boardId: string;
  /**
   * The document that was LIVE when this was recorded. Usually the same as
   * `boardId`; different when an action changed another notebook from here -
   * merging pages into a closed notebook from the Duplicate dialog. Undo
   * reverses what you did from where you are (see lastIndexForContext).
   * Missing on entries saved by older builds, which means `boardId`.
   */
}

```

```

    activeAt?: string;
    /** Epoch ms. Orders entries back into one list when restored after a restart. */
    at?: number;
}

interface HistoryState {
    past: HistoryEntry[];
    future: HistoryEntry[];
}

/**
 * One sidebar deletion, restorable. Boards keep their original indices so
 * undo puts the tree back exactly; folders keep whole objects; boards a
 * trashed folder filed to root remember where they came from. Asset bytes
 * stay in IndexedDB while this lives (see collectReferencedAssets).
 */
export interface TrashBuffer {
    removedBoards: Array<{ board: Board; index: number }>;
    removedFolders: Folder[];
    movedToRoot: Array<{ id: string; folderId: string | null }>;
    activeBoardId: string | null;
    activeFolderId: string | null;
    at: number;
}

interface WhiteboardState {
    // Document tree
    folders: Folder[];
    boards: Board[];
    activeBoardId: string | null;
    activeFolderId: string | null;

    // Active board canvas state
    camera: Camera;
    tool: ToolType;
    style: StyleState;
    selectedIds: string[];
    editingId: string | null; // ID of text/sticky element currently being edited inline
    customLinkTypes: Array<{ id: string; name: string; color: string }>;

    // History per board
    history: HistoryState;

    // UI & Touch state
    sidebarOpen: boolean;
    sidebarSortBy: "updated" | "created" | "name";
    sidebarSortOrder: "asc" | "desc";
    pagePanelOpen: boolean;
    isExporting: boolean;
    isReady: boolean;
    minimapOpen: boolean;
    legendOpen: boolean;
    zoomDockOpen: boolean;
    /** Pen strokes that are clearly a shape become that exact shape. */
    shapeRecognition: boolean;
    toggleShapeRecognition: () => void;
    /** Selection inspector folded to its header bar (the compact panel option). */
    propsPanelCollapsed: boolean;
    /**
     * The ACTIVE TOOL's properties row (colour, thickness) folded away. Distinct
     * from propsPanelCollapsed, which is the selection inspector.
     */
    toolPropsBarCollapsed: boolean;
    toggleToolPropsBar: () => void;
    /** The tool bar's buttons, in order (its "Customize tool bar" button). */
    toolbarItems: ToolbarItemId[];
    setToolbarItems: (items: ToolbarItemId[]) => void;
    /**
     * "Locked items" mode of the Select tool: while on, ONLY locked things can
     * be picked, so they can be unlocked (see LockedModeBar). Never saved.
     */
    lockedMode: boolean;
    /** The picture open in the image editor, if any. Never saved. */
    imageStudioId: string | null;
    snapToGrid: boolean;
    touchSettings: TouchSettings;
    /** App default paper: new boards and new pages start with it. Past notes keep theirs. */
    defaultPaper: BoardBackground;

```

```

/** Colours the user actually mixed, most recent first (max 8). */
recentColors: string[];
addRecentColor: (color: string) => void;

// Favorite pens bar
pinnedPens: PinnedPen[];
pinnedBarOpen: boolean;
pinnedBarPos: PinnedBarPosition | null;
toolbarPosition: DockPosition;
addPinnedPen: (pen: Omit<PinnedPen, "id">) => string;
pinCurrentPen: () => string | null;
updatePinnedPen: (id: string, patch: Partial<Omit<PinnedPen, "id">>) => void;
removePinnedPen: (id: string) => void;
reorderPinnedPens: (fromIndex: number, toIndex: number) => void;
applyPinnedPen: (id: string) => void;
togglePinnedBar: () => void;
setPinnedBarPos: (pos: PinnedBarPosition | null) => void;
setToolbarPosition: (pos: DockPosition) => void;

// ----- Split view -----
/** The board shown in the reference pane, or null when the split is closed. */
splitBoardId: string | null;
/** Fraction of the width given to the editable pane, 0.2-0.8. */
splitRatio: number;
setSplitBoard: (boardId: string | null) => void;
setSplitRatio: (ratio: number) => void;
/**
 * Stores a camera against a board that is NOT the active one – the split
 * view's other pane. Both panes then read the same camera per document, so
 * swapping which side is editable does not move anything.
 */
setBoardCamera: (boardId: string, camera: Camera) => void;
swapSplit: () => void;
/**
 * Which slot holds the editable document. "left" is the FIRST slot: the
 * left pane side by side, the TOP pane when stacked.
 */
splitSide: "left" | "right";
/** Side by side (left | right) or stacked (top / bottom). */
splitOrientation: SplitOrientation;
setSplitOrientation: (o: SplitOrientation) => void;
/** Makes the document on `side` the editable one, leaving both in place. */
focusSplitSide: (side: "left" | "right") => void;
/** Opens a split with an explicit document in each slot (first = left/top). */
setSplitPair: (leftBoardId: string, rightBoardId: string, orientation?: SplitOrientation) => void;
/** Vertical rail or horizontal strip – a strip is what you want docked at the top. */
pinnedBarVertical: boolean;
togglePinnedBarOrientation: () => void;

setTouchSettings: (settings: Partial<TouchSettings>) => void;
/** Spoken language for voice dictation (BCP-47, e.g. "he-IL"). */
dictationLang: string;
setDictationLang: (lang: string) => void;
/** Samsung-style strict encircle: loops must close, neighbours only grazed stay out. */
lassoStrict: boolean;
setLassoStrict: (strict: boolean) => void;
toggleMinimap: () => void;
setMinimapOpen: (open: boolean) => void;
toggleLegend: () => void;
setLegendOpen: (open: boolean) => void;
toggleZoomDock: () => void;
setZoomDockOpen: (open: boolean) => void;
setPropsPanelCollapsed: (collapsed: boolean) => void;
toggleSnapToGrid: () => void;
setSnapToGrid: (snap: boolean) => void;
togglePagePanel: () => void;
setPagePanelOpen: (open: boolean) => void;

// Initialization
init: () => void;

// Custom link types
addCustomLinkType: (name: string, color: string) => string;
removeCustomLinkType: (id: string) => void;

// Folders

```

```

createFolder: (name: string, parentId: string | null, color?: string) => string;
renameFolder: (folderId: string, name: string) => void;
deleteFolder: (folderId: string) => void;
toggleFolderExpanded: (folderId: string) => void;
revealFolder: (folderId: string) => void;
/**
 * Open a board AND centre the viewport on one element, selecting it (unless
 * `select: false` - search only shows where a match is). In a notebook the
 * element may be on any page; that page is turned to.
 */
focusElement: (boardId: string, elementId: string, opts?: { select?: boolean }) => void;
moveFolder: (folderId: string, newParentId: string | null) => void;
setFolderColor: (folderId: string, color: string) => void;

// Boards
createBoard: (name: string, folderId: string | null, color?: string) => string;
createNotebook: (name: string, folderId: string | null, pageSize?: PageSizeId, color?: string) => string;
convertToNotebook: (boardId: string, pageSize?: PageSizeId) => void;
/**
 * Deletes PDFs and images in IndexedDB that no board references any more.
 * Without it, deleting a PDF notebook strands tens of megabytes that nothing
 * can ever open again.
 */
pruneOrphanedAssets: () => Promise<void>;
importPdfAsNotebook: (file: File, folderId?: string | null) => Promise<string>;
/**
 * PDF, Word (.docx) or PowerPoint (.pptx) -> a notebook. Office files are
 * converted to a PDF on the device first (lib/whiteboard/officeImport.ts),
 * then imported exactly like a PDF, so everything that works on PDF
 * notebooks works on them. Every import path calls this one.
 */
importDocumentAsNotebook: (
  file: File,
  folderId?: string | null,
  onProgress?: (p: { label: string; done: number; total: number }) => void
) => Promise<string>;
/**
 * Inserts a run of pages from ANOTHER board into a notebook (targetBoardId or active).
 * `range` can be { from, to }, { indices }, or number[] (0-based indices).
 * `atIndex` is 0-based and clamped.
 * Returns how many pages were inserted.
 */
insertPagesFromBoard: (
  sourceBoardId: string,
  range: { from?: number; to?: number; indices?: number[] } | number[],
  atIndex: number,
  targetBoardId?: string
) => number;
/**
 * Inserts a run of pages from a PDF file into the active notebook.
 * `range` is 1-based and inclusive over the PDF's own page numbering.
 */
insertPdfPages: (
  file: File,
  range: { from: number; to: number },
  atIndex: number
) => Promise<number>;
addImagePages: (images: Array<{ src: string; width: number; height: number }>) => void;
/**
 * Adds a captured picture to ANY document, as one undoable step taken from
 * the document you are in. A notebook gets it on a fresh page (after
 * `afterPageId`, or at the end); a whiteboard gets it beside its content.
 * Returns where it went, or null when the document does not exist.
 */
addCaptureToBoard: (
  boardId: string,
  image: {
    src: string;
    naturalWidth: number;
    naturalHeight: number;
    /** Its size on the page it came from, in world units. */
    size: { width: number; height: number };
    afterPageId?: string;
  }
) => { elementId: string; pageId?: string; pageNumber?: number } | null;

// ----- Notebook pages -----
addPage: (afterPageId?: string) => void;

```

```

addPageBefore: (pageId: string) => void;
togglePageBookmark: (pageId: string) => void;

// ----- Page templates -----
pageTemplates: PageTemplate[];
savePageAsTemplate: (pageId: string, name: string) => void;
deletePageTemplate: (templateId: string) => void;
addPageFromTemplate: (templateId: string, afterPageId?: string) => void;
createImageTemplate: (name: string, src: string, width: number, height: number) => void;
deletePage: (pageId: string) => void;
duplicatePage: (pageId: string) => void;
movePage: (pageId: string, toIndex: number) => void;
setActivePage: (pageId: string) => void;
stepPage: (delta: number) => void;
/**
 * Inserts a blank page at `index` (0 = above the first, pages.length = below
 * the last) and keeps the view still, so the new sheet appears exactly where
 * the "+" between two pages was tapped.
 */
insertPageAt: (index: number) => void;
/** Opens a page and shows its TOP, rather than the scroll offset of the last one. */
goToPage: (pageId: string, viewportHeight: number) => void;
reanchorPage: (delta: number, stride: number) => void;
/**
 * The same page hand-over as `reanchorPage`, for a notebook that is NOT the
 * live one - the split view's other pane scrolls continuously too.
 */
reanchorBoardPage: (boardId: string, delta: number, stride: number) => void;
/**
 * Opens a page of a notebook that is not the live one, showing its top -
 * the split pane's page buttons. The notebook's OWN active page changes, so
 * the page you were reading is the page you get when you start writing.
 */
goToBoardPage: (boardId: string, pageId: string, viewportHeight: number) => void;
setPageBackground: (pageId: string, patch: Partial<BoardBackground>) => void;
setPageSize: (pageSize: PageSizeId) => void;
renameBoard: (boardId: string, name: string) => void;
deleteBoard: (boardId: string) => void;
duplicateBoard: (boardId: string) => string | null;
duplicateNotebook: (
  sourceBoardId: string,
  options: { name?: string; folderId?: string | null; pageIndices?: number[] }
) => string | null;
setActiveBoard: (boardId: string) => void;
moveBoard: (boardId: string, newFolderId: string | null) => void;
setBoardColor: (boardId: string, color: string) => void;
setBoardBackground: (
  boardId: string,
  patch: Partial<BoardBackground>,
  opts?: { allPages?: boolean }
) => void;
/** Save paper as the app default for FUTURE boards and pages. */
setDefaultPaper: (patch: Partial<BoardBackground>) => void;
toggleStarred: (boardId: string) => void;
addTag: (boardId: string, tag: string) => void;
removeTag: (boardId: string, tag: string) => void;
findOrCreateBoardByName: (name: string) => string;
searchBoards: (query: string) => Array<{ boardId: string; boardName: string; snippet: string; elementId: string }>;
getRecentBoards: (limit?: number) => Board[];
getStarredBoards: () => Board[];
setSidebarSort: (by: "updated" | "created" | "name", order?: "asc" | "desc") => void;
batchMoveBoards: (boardIds: string[], targetFolderId: string | null) => void;
batchDeleteBoards: (boardIds: string[]) => void;
batchStarBoards: (boardIds: string[], star?: boolean) => void;
/**
 * Full copies via duplicateBoard (notebooks keep every page, images and
 * PDFs stay referenced by id - no byte duplication). Returns the new ids.
 */
batchDuplicateBoards: (boardIds: string[]) => string[];
/**
 * Session-only trash buffer for sidebar deletes. NOT persisted: undo does
 * not survive a restart (the deletion stands), which also bounds how long
 * deleted pictures/PDFs must be protected from the asset pruner.
 */
lastDeleted: TrashBuffer | null;
/**
 * Removes boards + folder subtrees like the old delete paths (folders go,

```

```

    * their boards survive at root), but keeps everything needed to undo.
    * Unknown ids are ignored; a call that removes nothing leaves any
    * existing buffer – and the toast – alone.
    */
trashItems: (boardIds: string[], folderIds: string[]) => void;
/** Restores the trash buffer; a no-op when there is nothing to restore. */
undoTrash: () => void;
/** Drops the buffer (undo expires) and reclaims newly orphaned assets. */
discardTrash: () => void;

// Elements
addElement: (el: WhiteboardElement, commit?: boolean) => void;
addElements: (els: WhiteboardElement[], commit?: boolean) => void;
/** Add an element to a SPECIFIC board without switching the active board
 * (switching boards wipes camera/selection/history – wrong for background writes). */
addElementToBoard: (boardId: string, el: WhiteboardElement, commit?: boolean) => void;
/** Delete elements from a SPECIFIC board without switching the active board. */
deleteElementFromBoard: (boardId: string, ids: string[], commit?: boolean) => void;
updateElement: (id: string, patch: Partial<WhiteboardElement>, commit?: boolean) => void;
updateElements: (ids: string[], patch: Partial<WhiteboardElement>, commit?: boolean) => void;
deleteElements: (ids: string[], commit?: boolean) => void;
deleteSelected: () => void;
duplicateElements: (ids: string[]) => string[];
toggleLock: (ids: string[]) => void;
/** Unlocks these (never locks). Returns how many were locked before. */
unlockElements: (ids: string[]) => number;
setLockedMode: (on: boolean) => void;
setImageStudioId: (id: string | null) => void;
bringToFront: (ids: string[]) => void;
sendToBack: (ids: string[]) => void;
bringForward: (ids: string[]) => void;
sendBackward: (ids: string[]) => void;

// Connections
createConnection: (
  source: { boardId: string; elementId: string; anchor?: NodeAnchor },
  target: { boardId: string; elementId: string; anchor?: NodeAnchor },
  linkType?: string,
  routing?: ConnectionRouting
) => string;

// Selection & Editing
setSelected: (ids: string[]) => void;
addToSelection: (id: string) => void;
removeFromSelection: (id: string) => void;
toggleSelection: (id: string) => void;
clearSelection: () => void;
selectAll: () => void;
selectByType: (type: "stroke" | "text" | "image" | "shape" | "sticky" | "connection") => void;
invertSelection: () => void;
setEditingId: (id: string | null) => void;
/**
 * Removes a text box that was started but never written in, together with
 * the undo entries that only recorded its creation. Runs automatically
 * whenever editing ends (see the subscription at the bottom of this file).
 * Returns true when something was removed.
 */
discardAbandonedText: (id: string, boardId: string) => boolean;

// Camera
setCamera: (camera: Partial<Camera>) => void;
panCamera: (dx: number, dy: number) => void;
zoomTo: (zoom: number, centerX?: number, centerY?: number, viewport?: Viewport) => void;
/**
 * The canvas area's measured size. The store cannot see the DOM, and
 * `window.innerWidth` is the WHOLE window - sidebar, panels and all - so
 * anything derived from it (the notebook zoom floor, a fit) came out wrong
 * by however wide the sidebar was. The canvas reports its real size here.
 */
canvasViewport: Viewport;
setCanvasViewport: (vp: Viewport) => void;
zoomToFit: (viewport?: Viewport) => void;
resetCamera: () => void;

/**
 * The pen you last drew with. Reaching for "the pen" after using the eraser
 * must return the pen you were holding, not a hardcoded default.
 */

```

```

lastStrokeTool: ToolType;

// Tool / Style
setTool: (tool: ToolType) => void;
setStyle: (patch: Partial<StyleState>) => void;

// History
undo: () => void;
redo: () => void;
commitHistory: () => void;
canUndo: () => boolean;
canRedo: () => boolean;

// UI
toggleSidebar: () => void;
setSidebarOpen: (open: boolean) => void;

// Persistence
getPersistedState: () => PersistedState;
saveNow: () => void;

// Workspace backup / sync
getWorkspaceSnapshot: () => WorkspaceSnapshot;
replaceWorkspace: (snapshot: WorkspaceSnapshot) => void;
mergeWorkspace: (remote: WorkspaceSnapshot) => MergeReport;
}

const DEFAULT_CAMERA: Camera = { x: 0, y: 0, zoom: 1 };

/**
 * Deep-clone elements for history snapshots. A shallow `{ ...el }` shares
 * nested structures (stroke.points, connection.source/target) by reference
 * between the snapshot and the live board – any in-place mutation would
 * retroactively corrupt every snapshot.
 */
function clonePagesForHistory(pages: NotebookPage[]): NotebookPage[] {
  if (typeof structuredClone === "function") {
    return pages.map((p) => structuredClone(p));
  }
  return JSON.parse(JSON.stringify(pages)) as NotebookPage[];
}

/**
 * Sanitizes 0-based page indices from callers (modal, tests, future API):
 * drops non-integers and negatives, dedupes while preserving order.
 *
 * Deduping matters: copyPagesFrom keys its id-remap by source element id,
 * so passing the same page twice would give both copies the SAME new ids.
 * Out-of-range values are left for copyPagesFrom to drop via filter(Boolean).
 */
function sanitizePageIndices(indices: number[]): number[] {
  const seen = new Set<number>();
  const out: number[] = [];
  for (const i of indices) {
    if (!Number.isInteger(i) || i < 0 || seen.has(i)) continue;
    seen.add(i);
    out.push(i);
  }
  return out;
}

/**
 * Puts a history snapshot back. For a notebook the snapshot is restored onto
 * the page it was taken from, re-selecting that page, so undo is never applied
 * to whichever page happens to be open now.
 * If the entry records a full pages list (e.g. from page insertion/merge),
 * it restores the notebook's entire page list.
 */
function restoreHistoryElements(board: Board, entry: HistoryEntry): Board {
  if (entry.pages && isNotebook(board)) {
    const restoredPages = clonePagesForHistory(entry.pages);
    const activeId =
      entry.pageId && restoredPages.some((p) => p.id === entry.pageId)
        ? entry.pageId
        : restoredPages[0]?.id ?? board.activePageId;
    const restoredDocIds = [
      ...new Set(
        restoredPages

```

```

        .map((p) => p.pdf?.docId)
        .filter((d): d is string => typeof d === "string")
    ),
];
return {
    ...board,
    pages: restoredPages,
    activePageId: activeId,
    pdfDocIds: restoredDocIds,
    updatedAt: Date.now(),
};
}
if (!isNotebook(board) || !entry.pageId) {
    return mapBoardElements(board, () => entry.elements);
}
return mapBoardElements(setActivePage(board, entry.pageId), () => entry.elements);
}

/**
 * Runs a page operation on the active board, if it is a notebook.
 *
 * Page turns clear the SELECTION – handles would otherwise hang over elements
 * that are no longer on screen – but deliberately keep the undo history. Undo
 * spans the whole notebook the way it does in GoodNotes and Notability, and
 * each history entry records the page it was taken on, so undoing after
 * turning the page edits the page the edit actually happened on and brings
 * that page back into view (see `restoreHistoryElements`). Clearing history
 * here instead would make undo silently do nothing after any page turn.
 */
function applyToActiveNotebook(
    set: (fn: (s: WhiteboardState) => Partial<WhiteboardState>) => void,
    get: () => WhiteboardState,
    fn: (board: Board) => Board,
    opts: { keepSelection?: boolean } = {}
): void {
    const activeId = get().activeBoardId;
    const board = get().boards.find((b) => b.id === activeId);
    if (!board || !isNotebook(board)) return;

    set((s) => ({
        boards: s.boards.map((b) => (b.id === activeId ? fn(b) : b)),
        ...(opts.keepSelection ? {} : { selectedIds: [], editingId: null }),
    }));
    get().saveNow();
}

/**
 * Undo reverses what you did FROM the document you are in.
 *
 * History is one list. Every entry records which document was live when it
 * was made (`activeAt`), and undo/redo take the newest entry made from the
 * live document. That one rule covers every case:
 *
 * - Split view: draw on the right, tap left, draw on the left. Undo takes the
 *   left stroke, and then stops - it never reaches across the divider and
 *   edits the pane you are not in.
 * - Merging pages into a CLOSED notebook from the Duplicate dialog: the merge
 *   was made from the notebook on screen, so one undo from there cancels it,
 *   even though the pages changed are somewhere else.
 * - Switching notebooks from the sidebar: each keeps its own undo, and none
 *   silently edits a document that is not on screen.
 *
 * Before, switching board wiped the whole list. In split view tapping the other
 * pane IS a board switch, so every glance across the divider lost your undo.
 */
function contextOf(e: HistoryEntry): string {
    return e.activeAt ?? e.boardId;
}

export function lastIndexOfContext(list: HistoryEntry[], activeBoardId: string | null): number {
    if (!activeBoardId) return -1;
    for (let i = list.length - 1; i >= 0; i--) if (contextOf(list[i]) === activeBoardId) return i;
    return -1;
}

function cloneElementsForHistory(elements: WhiteboardElement[]): WhiteboardElement[] {
    if (typeof structuredClone === "function") {
        return elements.map((el) => structuredClone(el));
    }

```



```

    }
    return JSON.parse(JSON.stringify(elements)) as WhiteboardElement[];
}

/**
 * History worth persisting: entries for boards that still exist. Pruning here
 * (not only at load) keeps a deleted notebook's steps from accumulating on
 * disk forever.
 */
function persistableHistory(s: WhiteboardState): { past: HistoryEntry[]; future: HistoryEntry[] } {
    const ids = new Set(s.boards.map((b) => b.id));
    const keep = (e: HistoryEntry) =>
        ids.has(e.boardId) && (Array.isArray(e.elements) || Array.isArray(e.pages));
    return { past: s.history.past.filter(keep), future: s.history.future.filter(keep) };
}

function makeUniqueName(name: string, existingNames: string[]): string {
    const trimmed = name.trim() || "Untitled";
    if (!existingNames.includes(trimmed)) return trimmed;
    let i = 2;
    while (existingNames.includes(`${trimmed} (${i})`)) {
        i++;
    }
    return `${trimmed} (${i})`;
}

function getInitialDemoState(): { folders: Folder[]; boards: Board[]; activeBoardId: string } {
    const folderId = nanoid(8);
    const boardId = nanoid(8);
    const welcomeTextId = nanoid(8);
    const sticky1Id = nanoid(8);
    const sticky2Id = nanoid(8);
    const connId = nanoid(8);

    const folders: Folder[] = [
        {
            id: folderId,
            name: "Getting Started",
            parentId: null,
            expanded: true,
            createdAt: Date.now(),
            color: "#3b82f6",
        },
    ];

    const boards: Board[] = [
        {
            id: boardId,
            name: "Welcome to Knowledge Whiteboard",
            folderId: folderId,
            createdAt: Date.now(),
            updatedAt: Date.now(),
            accessedAt: Date.now(),
            starred: true,
            tags: ["welcome", "guide"],
            color: "#3b82f6",
            camera: { x: 0, y: 0, zoom: 1 },
            elements: [
                {
                    id: welcomeTextId,
                    type: "text",
                    x: -260,
                    y: -180,
                    width: 520,
                    height: 90,
                    rotation: 0,
                    opacity: 1,
                    text: "# ■ Knowledge Canvas & Infinite Whiteboard\nDouble-click anywhere to type, or use [[WikiLinks]] to connect your thoughts.",
                    color: "#1e293b",
                    fontSize: 22,
                    fontFamily: "var(--font-inter), Inter, system-ui, sans-serif",
                    fontWeight: 600,
                    align: "center",
                    createdAt: Date.now(),
                    updatedAt: Date.now(),
                },
            ],
        },
    ];
}

```

```

    id: sticky1Id,
    type: "sticky",
    x: -280,
    y: -20,
    width: 220,
    height: 180,
    rotation: -0.03,
    opacity: 1,
    text: "■ **Core Ideas**\n- Obsidian-grade [[WikiLinks]]\n- Bi-directional Backlinks (Ctrl+B)\n- Interactive Knowledge Graph (Ctrl+G)\n#ideas #canvas",
    color: "yellow",
    fontSize: 16,
    align: "left",
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: sticky2Id,
    type: "sticky",
    x: 60,
    y: -20,
    width: 220,
    height: 180,
    rotation: 0.02,
    opacity: 1,
    text: "■ **Smart Connectors**\n- Drag arrows between notes\n- Typed relationships (*Causes*, *Depends*, *Relates*)\n- Auto-docking anchor ports",
    color: "blue",
    fontSize: 16,
    align: "left",
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: connId,
    type: "connection",
    x: 0,
    y: 0,
    rotation: 0,
    opacity: 1,
    source: { boardId, elementId: sticky1Id, anchor: "right" },
    target: { boardId, elementId: sticky2Id, anchor: "left" },
    linkType: "leads_to",
    label: "Powers",
    routing: "curved",
    dashStyle: "solid",
    startArrowhead: "none",
    endArrowhead: "arrow",
    strokeWidth: 2.5,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
],
],
];

return { folders, boards, activeBoardId: boardId };
}

/**
 * Applies a loaded (or absent) workspace to the store. Shared by the
 * synchronous legacy boot and the async IndexedDB boot so both behave
 * identically – the async path only changes WHEN this runs, never WHAT.
 */
function initFromSnapshot(saved: PersistedState | null): void {
  const set = useWhiteboard.setState;
  // Any previously saved state is honored – even one with zero boards
  // (the user may have deliberately deleted everything). The demo board is
  // only seeded when there is no saved state at all, otherwise the user's
  // folders / link types would be wiped and demo content re-injected.
  if (saved && Array.isArray(saved.boards)) {
    const activeBoard = saved.boards.find((b) => b.id === saved.activeBoardId) || saved.boards[0];
    set({
      folders: (saved.folders || []).map((f) => ({
        ...f,
        parentId: f.parentId ?? null,
        expanded: f.expanded !== false,
      })),
    });
  }
}

```

```

boards: saved.boards,
activeBoardId: activeBoard ? activeBoard.id : null,
activeFolderId: saved.activeFolderId || null,
customLinkTypes: saved.customLinkTypes || [],
pinnedPens: saved.pinnedPens || [],
pinnedBarPos: saved.pinnedBarPos ?? null,
toolbarPosition: saved.toolbarPosition ?? "bottom",
splitBoardId: saved.splitBoardId ?? null,
splitRatio: typeof saved.splitRatio === "number" ? saved.splitRatio : 0.5,
splitSide: saved.splitSide === "right" ? "right" : "left",
splitOrientation: saved.splitOrientation === "stacked" ? "stacked" : "side",
pageTemplates: saved.pageTemplates ?? [],
pinnedBarVertical: saved.pinnedBarVertical !== false,
recentColors: saved.recentColors || [],
// Merge over the defaults rather than replacing them, so a preference
// added in a later version still gets its default on an old save.
// buildToolInk fills any pen missing from the saved map – from the
// legacy per-pen fields where they exist, then defaults. Without it a
// save written before the pen rework leaves the four newest pens with
// no ink at all.
lastStrokeTool: (saved.lastStrokeTool && isStrokeTool(saved.lastStrokeTool)
  ? saved.lastStrokeTool
  : "gel") as ToolType,
style: {
  ...DEFAULT_STYLE,
  ...(saved.style ?? {}),
  toolInk: buildToolInk(saved.style?.toolInk, saved.style),
},
touchSettings: { ...DEFAULT_TOUCH_SETTINGS, ...(saved.touchSettings ?? {}) },
// Never reopen straight into the eraser: a stray tap on a restored
// board would silently destroy work before the user notices the tool.
tool: saved.tool && saved.tool !== "eraser" ? saved.tool : "select",
snapToGrid: saved.snapToGrid ?? false,
minimapOpen: saved.minimapOpen ?? false,
legendOpen: saved.legendOpen ?? false,
zoomDockOpen: saved.zoomDockOpen ?? true,
shapeRecognition: saved.shapeRecognition ?? false,
defaultPaper: saved.defaultPaper ?? { pattern: "plain" },
// Undo survives restarts: this workspace's steps come back oldest
// first (last 100). Entries for deleted boards are dropped, and
// corrupt ones never load – a bad entry costs an undo step, not a board.
history: restorableHistory(
  saved.boards.map((b) => b.id),
  loadHistory()
),
pinnedBarOpen: saved.pinnedBarOpen ?? true,
propsPanelCollapsed: saved.propsPanelCollapsed ?? true,
toolPropsBarCollapsed: saved.toolPropsBarCollapsed ?? false,
toolbarItems: mergeToolbarItems(saved.toolbarTools, saved.toolbarHidden),
dictationLang: resolveDictationLang(saved.dictationLang),
lassoStrict: saved.lassoStrict ?? false,
camera: activeBoard?.camera || { ...DEFAULT_CAMERA },
isReady: true,
});
} else {
const initial = getInitialDemoState();
set({
  folders: initial.folders,
  boards: initial.boards,
  activeBoardId: initial.activeBoardId,
  activeFolderId: initial.folders[0]?.id || null,
  isReady: true,
});
useWhiteboard.getState().saveNow();
}
}

export const useWhiteboard = create<WhiteboardState>((set, get) => ({
  folders: [],
  boards: [],
  activeBoardId: null,
  activeFolderId: null,

  camera: { ...DEFAULT_CAMERA },
  canvasViewport: { width: 1200, height: 800 },
  tool: "select",
  style: { ...DEFAULT_STYLE },
  lastStrokeTool: "gel" as ToolType,

```

```

selectedIds: [],
editingId: null,
customLinkTypes: [],

history: { past: [], future: [] },

sidebarOpen: true,
sidebarSortBy: "updated",
sidebarSortOrder: "desc",
pagePanelOpen: false,
isExporting: false,
isReady: false,
minimapOpen: false,
legendOpen: false,
zoomDockOpen: true,
shapeRecognition: false,
snapToGrid: false,
touchSettings: { ...DEFAULT_TOUCH_SETTINGS },
dictationLang: resolveDictationLang(typeof navigator !== "undefined" ? navigator.language : undefined),
lassoStrict: false,
defaultPaper: { pattern: "plain" },

recentColors: [],

addRecentColor: (color) => {
  const c = color.toLowerCase();
  set((s) => ({
    // Most recent first, no duplicates, capped – an unbounded list would
    // grow forever in localStorage and overflow the palette row.
    recentColors: [c, ...s.recentColors.filter((x) => x !== c)].slice(0, 8),
  }));
  get().saveNow();
},

// ---- Favorite pens bar -----
pinnedPens: [],
pinnedBarOpen: true,
// The selection inspector opens folded: on a tablet the full panel buries
// the shape being edited. One tap unfolds it; the choice persists.
propsPanelCollapsed: true,
toolPropsBarCollapsed: false,
toolbarItems: [...DEFAULT_TOOLBAR_ITEMS],
lockedMode: false,
imageStudioId: null,
pinnedBarPos: null,
toolbarPosition: "bottom",

addPinnedPen: (pen) => {
  const id = nanoid(8);
  set((s) => ({ pinnedPens: [...s.pinnedPens, { ...pen, id }] }));
  get().saveNow();
  return id;
},

/**
 * Pin whatever the user is drawing with right now. Only stroke tools can be
 * pinned – pinning "select" or "rectangle" would produce a pen that draws
 * nothing.
 */
pinCurrentPen: () => {
  const s = get();
  const tool = isStrokeTool(s.tool) ? s.tool : s.lastStrokeTool;
  return get().addPinnedPen({
    tool: tool as PinnedPen["tool"],
    color: getToolColor(s.style, tool),
    size: getToolSize(s.style, tool),
  });
},

updatePinnedPen: (id, patch) => {
  set((s) => ({
    pinnedPens: s.pinnedPens.map((p) => (p.id === id ? { ...p, ...patch } : p)),
  }));
  get().saveNow();
},

removePinnedPen: (id) => {
  set((s) => ({ pinnedPens: s.pinnedPens.filter((p) => p.id !== id) }));
}

```

```

    get().saveNow();
  },

  reorderPinnedPens: (fromIndex, toIndex) => {
    set((s) => {
      const next = [...s.pinnedPens];
      if (
        fromIndex < 0 || fromIndex >= next.length ||
        toIndex < 0 || toIndex >= next.length ||
        fromIndex === toIndex
      ) {
        return {} as Partial<WhiteboardState>;
      }
      const [moved] = next.splice(fromIndex, 1);
      next.splice(toIndex, 0, moved);
      return { pinnedPens: next };
    });
    get().saveNow();
  },

  /** Select a pinned pen: its tool becomes active with its own color & size. */
  applyPinnedPen: (id) => {
    const pin = get().pinnedPens.find((p) => p.id === id);
    if (!pin) return;
    set((s) => ({
      tool: pin.tool as ToolType,
      style: {
        ...s.style,
        // One combined patch: colour and size share the toolInk map, so two
        // spreads would drop the colour.
        ...setToolInk(s.style, pin.tool as ToolType, { color: pin.color, size: pin.size }),
      },
      // Switching to a drawing tool should drop any lingering selection, the
      // same way setTool does.
      selectedIds: [],
    })));
  },

  pinnedBarVertical: true,

  /**
   * Rotate the bar between a vertical rail and a horizontal strip. The position
   * is kept and simply re-clamped on the next render, so rotating can never
   * throw the bar off screen.
   */
  togglePinnedBarOrientation: () => {
    set((s) => ({ pinnedBarVertical: !s.pinnedBarVertical }));
    get().saveNow();
  },

  togglePinnedBar: () => {
    set((s) => ({ pinnedBarOpen: !s.pinnedBarOpen }));
    get().saveNow();
  },

  setPinnedBarPos: (pos) => {
    set({ pinnedBarPos: pos });
    get().saveNow();
  },

  setToolbarPosition: (pos) => {
    set({ toolbarPosition: pos });
    get().saveNow();
  },

  // ----- Split view -----
  splitBoardId: null,
  splitRatio: 0.5,
  splitSide: "left",
  splitOrientation: "side",

  setSplitOrientation: (o) => {
    if (o === get().splitOrientation) return;
    // The documents keep their slots (left becomes top), so nothing jumps to
    // the other end of the screen; each keeps the share of space it had.
    set({ splitOrientation: o });
    get().saveNow();
  },

```

```

/**
 * Editing on either side, without duplicating the store.
 *
 * There is one `activeBoardId`, camera, selection and history, so only one
 * document can be edited at a time. Rather than duplicating all of that,
 * focus simply follows you: tapping the other pane makes ITS document the
 * active one and hands the reference role to the document you left.
 *
 * Neither pane moves – the panes keep their positions and only their roles
 * swap – and because `setActiveBoard` already stores the outgoing board's
 * camera and restores the incoming one, each side keeps its own view.
 */
focusSplitSide: (side) => {
  const { splitSide, splitBoardId, activeBoardId, editingId } = get();
  if (side === splitSide || !splitBoardId || !activeBoardId) return;
  // Never switch out from under an open text editor: the edit would be
  // discarded and the keystrokes would land nowhere.
  if (editingId) return;
  const incoming = splitBoardId;
  set({ splitBoardId: activeBoardId, splitSide: side });
  get().setActiveBoard(incoming);
},

setSplitPair: (leftBoardId, rightBoardId, orientation) => {
  if (leftBoardId === rightBoardId) return;
  // The first (left / top) document becomes the editable one, which is what
  // someone choosing it first expects.
  set({
    splitBoardId: rightBoardId,
    splitSide: "left",
    ...(orientation ? { splitOrientation: orientation } : {}),
  });
  get().setActiveBoard(leftBoardId);
  get().saveNow();
},

setSplitBoard: (boardId) => {
  // Showing the document you are editing beside itself is confusing rather
  // than useful: the reference pane would not follow your edits, so the two
  // halves would silently disagree.
  if (boardId && boardId === get().activeBoardId) return;
  set({ splitBoardId: boardId });
  get().saveNow();
},

setBoardCamera: (boardId, camera) => {
  // Only for a board being shown elsewhere. The active board's camera is the
  // store's own `camera`, and writing both would fight it every frame.
  if (boardId === get().activeBoardId) return;
  set((s) => ({
    boards: s.boards.map((b) => (b.id === boardId ? { ...b, camera } : b)),
  }));
},

setSplitRatio: (ratio) => {
  // Neither pane may be dragged away to nothing, which would look like the
  // split had broken with no way to recover it.
  set({ splitRatio: Math.max(0.2, Math.min(0.8, ratio)) });
  get().saveNow();
},

/** Puts the reference document in the editable pane and vice versa. */
/** Exchanges which document is on which SIDE, keeping widths sensible. */
swapSplit: () => {
  const { splitBoardId, activeBoardId, splitSide, splitRatio } = get();
  if (!splitBoardId || !activeBoardId) return;

  // Swapping SIDES only moves which slot each document occupies. It must
  // NOT change which one is editable – doing that made this button a
  // duplicate of simply tapping the other pane, and left the documents
  // exactly where they were.
  //
  // The left slot is always `splitRatio` wide, so flipping the ratio too is
  // what keeps each document the width it already had as it changes sides.
  set({
    splitSide: splitSide === "left" ? "right" : "left",
    splitRatio: 1 - splitRatio,
  });
}

```

```

    });
    get().saveNow();
  },

  setTouchSettings: (patch) => {
    set((s) => ({ touchSettings: { ...s.touchSettings, ...patch } }));
    get().saveNow();
  },
  setDictationLang: (lang) => {
    set({ dictationLang: resolveDictationLang(lang) });
    get().saveNow();
  },
  setLassoStrict: (strict) => {
    set({ lassoStrict: strict });
    get().saveNow();
  },
  toggleMinimap: () => {
    set((s) => ({ minimapOpen: !s.minimapOpen }));
    get().saveNow();
  },
  setMinimapOpen: (open) => set({ minimapOpen: open }),
  toggleLegend: () => {
    set((s) => ({ legendOpen: !s.legendOpen }));
    get().saveNow();
  },
  setLegendOpen: (open) => set({ legendOpen: open }),
  toggleZoomDock: () => {
    set((s) => ({ zoomDockOpen: !s.zoomDockOpen }));
    get().saveNow();
  },
  setZoomDockOpen: (open) => set({ zoomDockOpen: open }),
  toggleShapeRecognition: () => {
    set((s) => ({ shapeRecognition: !s.shapeRecognition }));
    get().saveNow();
  },
  setPropsPanelCollapsed: (collapsed) => {
    set({ propsPanelCollapsed: collapsed });
    get().saveNow();
  },
  toggleToolPropsBar: () => {
    set((s) => ({ toolPropsBarCollapsed: !s.toolPropsBarCollapsed }));
    get().saveNow();
  },
  setToolbarItems: (items) => {
    // Never empty: a tool bar with no buttons could not be fixed from itself.
    if (items.length === 0) return;
    set({ toolbarItems: items });
    get().saveNow();
  },
  toggleSnapToGrid: () => {
    set((s) => ({ snapToGrid: !s.snapToGrid }));
    get().saveNow();
  },
  setSnapToGrid: (snap) => set({ snapToGrid: snap }),
  togglePagePanel: () => set((s) => ({ pagePanelOpen: !s.pagePanelOpen })),
  setPagePanelOpen: (open) => set({ pagePanelOpen: open }),

  init: () => {
    // No IndexedDB (unit tests, SSR, exotic WebViews): synchronous legacy
    // load – the old behavior, byte for byte.
    if (!workspaceDbAvailable()) {
      initFromSnapshot(loadState());
      return;
    }
    // Real browsers: the workspace lives in IndexedDB now, so boot waits on
    // it. The loading screen stays up until isReady flips, and this can never
    // reject – every failure path falls back to localStorage, then to demo.
    void (async () => {
      try {
        void requestPersistentStorage();
        initFromSnapshot((await loadWorkspace()).state);
      } catch {
        try {
          initFromSnapshot(loadState());
        } catch {
          initFromSnapshot(null);
        }
      }
    })
  }
}

```

```

    }
  })();
},

addCustomLinkType: (name, color) => {
  const id = `custom:${nanoid(6)}`;
  set((s) => ({
    customLinkTypes: [...s.customLinkTypes, { id, name, color }],
  }));
  get().saveNow();
  return id;
},

removeCustomLinkType: (id) => {
  set((s) => ({
    customLinkTypes: s.customLinkTypes.filter((t) => t.id !== id),
  }));
  get().saveNow();
},

// ----- Folders -----
createFolder: (name, parentId, color = "#f59e0b") => {
  const id = nanoid(8);
  const existing = get().folders.map((f) => f.name);
  const uniqueName = makeUniqueName(name, existing);
  const newFolder: Folder = {
    id,
    name: uniqueName,
    parentId: parentId ?? null,
    expanded: true,
    createdAt: Date.now(),
    color,
  };
  set((s) => ({
    folders: [
      ...s.folders.map((f) => (f.id === parentId ? { ...f, expanded: true } : f)),
      newFolder,
    ],
    activeFolderId: id,
  }));
  get().saveNow();
  return id;
},

renameFolder: (folderId, name) => {
  set((s) => ({
    folders: s.folders.map((f) => (f.id === folderId ? { ...f, name: name.trim() || f.name } : f)),
  }));
  get().saveNow();
},

deleteFolder: (folderId) => {
  set((s) => {
    // Collect the FULL descendant set so grandchildren are not orphaned
    // (an orphaned folder is invisible in the tree but persisted forever).
    const doomed = new Set<string>([folderId]);
    let changed = true;
    while (changed) {
      changed = false;
      for (const f of s.folders) {
        if (f.parentId && doomed.has(f.parentId) && !doomed.has(f.id)) {
          doomed.add(f.id);
          changed = true;
        }
      }
    }
  })
  return {
    folders: s.folders.filter((f) => !doomed.has(f.id)),
    boards: s.boards.map((b) =>
      b.folderId && doomed.has(b.folderId) ? { ...b, folderId: null } : b
    ),
    activeFolderId: s.activeFolderId && doomed.has(s.activeFolderId) ? null : s.activeFolderId,
  };
};
get().saveNow();
},

toggleFolderExpanded: (folderId) => {

```



```

    set((s) => ({
      folders: s.folders.map((f) =>
        f.id === folderId ? { ...f, expanded: !f.expanded } : f
      ),
    }));
    get().saveNow();
  },

  /**
   * Jump to a specific element: switch board if needed, centre the camera on
   * it, and select it.
   *
   * The camera is the WORLD point shown at the viewport centre
   * (screen = (world - camera) * zoom + viewport/2), so centring is simply
   * setting the camera to the element's own centre – no viewport size needed,
   * which matters because the store cannot see the viewport.
   */
  focusElement: (boardId, elementId, opts) => {
    const board = get().boards.find((b) => b.id === boardId);
    if (!board) return;
    let siblings = getBoardElements(board);
    let el = siblings.find((e) => e.id === elementId);
    // A notebook shows one page's elements at a time, so an element on
    // another page was "not found" and the jump silently did nothing.
    let pageId: string | null = null;
    if (!el && isNotebook(board)) {
      for (const page of getPages(board)) {
        const found = (page.elements ?? []).find((e) => e.id === elementId);
        if (found) {
          el = found;
          siblings = page.elements;
          pageId = page.id;
          break;
        }
      }
    }
    if (!el) return;

    if (get().activeBoardId !== boardId) get().setActiveBoard(boardId);

    // Page-local coordinates ARE world coordinates once that page is the
    // active one (it is anchored at world y = 0), so the camera maths is the
    // same as on a canvas.
    const bounds = focusBounds(el, siblings);
    const zoom = get().camera.zoom;
    const vp = get().canvasViewport;
    // Never arrive zoomed so far out that the element is a speck, but do not
    // zoom IN on someone who deliberately zoomed out to survey - and never so
    // far in that the element runs off the screen.
    const fit = Math.min(
      (vp.width * 0.85) / Math.max(bounds.width, 1),
      (vp.height * 0.7) / Math.max(bounds.height, 1)
    );
    const floor = isNotebook(board)
      ? Math.min(Math.max(MIN_ZOOM, minNotebookZoom(vp, pageSizeOf(board))), MAX_ZOOM)
      : MIN_ZOOM;
    set((s) => ({
      boards: pageId
        ? s.boards.map((b) => (b.id === boardId ? nbSetActivePage(b, pageId as string) : b))
        : s.boards,
      camera: {
        x: bounds.x + bounds.width / 2,
        y: bounds.y + bounds.height / 2,
        zoom: clamp(Math.min(Math.max(zoom, 0.6), fit), floor, MAX_ZOOM),
      },
      selectedIds: opts?.select === false ? [] : [elementId],
      editingId: null,
    }));
    get().saveNow();
  },

  /**
   * Make a folder visible and selected in the sidebar: expand it AND every
   * ancestor above it, then mark it active. Used when "entering" a folder from
   * somewhere outside the tree (e.g. clicking a folder node in the Graph).
   */
  revealFolder: (folderId) => {
    const folders = get().folders;

```

```

const byId = new Map(folders.map((f) => [f.id, f]));
if (!byId.has(folderId)) return;

// Walk up to the root, collecting the chain to expand. The visited guard
// keeps a pre-existing parentId cycle from spinning forever.
const toExpand = new Set<string>([folderId]);
const visited = new Set<string>([folderId]);
let cursor = byId.get(folderId)?.parentId ?? null;
while (cursor && !visited.has(cursor)) {
  visited.add(cursor);
  toExpand.add(cursor);
  cursor = byId.get(cursor)?.parentId ?? null;
}

set((s) => ({
  folders: s.folders.map((f) =>
    toExpand.has(f.id) ? { ...f, expanded: true } : f
  ),
  activeFolderId: folderId,
  sidebarOpen: true,
}));
get().saveNow();
},

moveFolder: (folderId, newParentId) => {
  if (folderId === newParentId) return;
  if (newParentId !== null) {
    const folders = get().folders;
    const byId = new Map(folders.map((f) => [f.id, f]));
    if (!byId.has(newParentId)) return; // parent must exist
    // Reject moves that would create a cycle (folder into its own
    // descendant) – a parentId cycle hangs every recursive tree walk.
    let cursor: string | null = newParentId;
    const visited = new Set<string>();
    while (cursor) {
      if (cursor === folderId) return;
      if (visited.has(cursor)) break; // guard against pre-existing corruption
      visited.add(cursor);
      cursor = byId.get(cursor)?.parentId ?? null;
    }
  }
  set((s) => ({
    folders: s.folders.map((f) => (f.id === folderId ? { ...f, parentId: newParentId } : f)),
  }));
  get().saveNow();
},

setFolderColor: (folderId, color) => {
  set((s) => ({
    folders: s.folders.map((f) => (f.id === folderId ? { ...f, color } : f)),
  }));
  get().saveNow();
},

// ----- Notebook pages -----
//
// Every page action rebuilds the active board through the pure helpers in
// notebook.ts, then clears selection and history. Selection survives a page
// turn otherwise, leaving handles floating over elements that are no longer
// on screen; history is per-page and would otherwise undo across the turn.
addPage: (afterPageId) => applyToActiveNotebook(set, get, (b) => nbAddPage(b, afterPageId)),
addPageBefore: (pageId) => applyToActiveNotebook(set, get, (b) => nbAddPageBefore(b, pageId)),
togglePageBookmark: (pageId) =>
  applyToActiveNotebook(set, get, (b) => nbTogglePageBookmark(b, pageId), {
    keepSelection: true,
  }),

// ----- Page templates -----
pageTemplates: [],

savePageAsTemplate: (pageId, name) => {
  const board = get().boards.find((b) => b.id === get().activeBoardId);
  if (!board || !isNotebook(board)) return;
  const page = getPages(board).find((p) => p.id === pageId);
  if (!page) return;
  const dim = pageSizeOf(board);
  set((s) => ({
    pageTemplates: [...s.pageTemplates, templateFromPage(page, name, dim.width, dim.height)],
  }));
}

```

```

    }));
    get().saveNow();
  },

  deletePageTemplate: (templateId) => {
    set((s) => ({ pageTemplates: s.pageTemplates.filter((t) => t.id !== templateId) }));
    get().saveNow();
  },

  addPageFromTemplate: (templateId, afterPageId) => {
    const template = get().pageTemplates.find((t) => t.id === templateId);
    if (!template) return;
    applyToActiveNotebook(set, get, (b) => insertTemplatePage(b, template, afterPageId));
  },

  /**
   * Turns an image into a template: one image element covering the sheet.
   * The image is scaled to FIT rather than fill, so a template made from a
   * differently proportioned scan is never cropped.
   */
  createImageTemplate: (name, src, width, height) => {
    const board = get().boards.find((b) => b.id === get().activeBoardId);
    const dim = pageSizeOf(board);
    const scale = Math.min(dim.width / width, dim.height / height);
    const w = width * scale;
    const h = height * scale;

    const template: PageTemplate = {
      id: `tpl_${Date.now().toString(36)}_${Math.random().toString(36).slice(2, 7)}`,
      name: name.trim() || "Image template",
      createdAt: Date.now(),
      width: dim.width,
      height: dim.height,
      elements: [
        {
          id: nanoid(8),
          type: "image",
          x: (dim.width - w) / 2,
          y: (dim.height - h) / 2,
          width: w,
          height: h,
          angle: 0,
          opacity: 1,
          src,
          // A template is a backdrop you write on top of, so it starts locked.
          locked: true,
          createdAt: Date.now(),
          updatedAt: Date.now(),
        } as unknown as WhiteboardElement,
      ],
    };
    set((s) => ({ pageTemplates: [...s.pageTemplates, template] }));
    get().saveNow();
  },

  deletePage: (pageId) => applyToActiveNotebook(set, get, (b) => nbDeletePage(b, pageId)),
  duplicatePage: (pageId) => applyToActiveNotebook(set, get, (b) => nbDuplicatePage(b, pageId)),
  movePage: (pageId, toIndex) =>
    applyToActiveNotebook(set, get, (b) => nbMovePage(b, pageId, toIndex), { keepSelection: true }),
  setActivePage: (pageId) => applyToActiveNotebook(set, get, (b) => nbSetActivePage(b, pageId)),
  stepPage: (delta) => applyToActiveNotebook(set, get, (b) => nbStepPage(b, delta)),

  // The view is anchored to the ACTIVE page (it always sits at world y = 0), so
  // any page operation that changes which page is active also moves what you
  // see unless the camera is compensated. Before this, adding a page made the
  // view jump. Inserting at `index` makes the new page the anchor; sliding the
  // camera by (index - oldActive) strides leaves every page above the new one
  // exactly where it was on screen, and the new sheet appears in the gap.
  insertPageAt: (index) => {
    const activeId = get().activeBoardId;
    const board = get().boards.find((b) => b.id === activeId);
    if (!board || !isNotebook(board)) return;

    const before = getActivePageIndexOf(board);
    const next = nbInsertPageAt(board, index);
    const at = getActivePageIndexOf(next);
    const stride = pageSizeOf(board).height + PAGE_GAP_STORE;
    const shift = (at - before) * stride;

```

```

    set((s) => ({
      boards: s.boards.map((b) => (b.id === activeId ? next : b)),
      camera: { ...s.camera, y: s.camera.y - shift },
      selectedIds: [],
      editingId: null,
    }));
    get().saveNow();
  },

  goToPage: (pageId, viewportHeight) => {
    const activeId = get().activeBoardId;
    const board = get().boards.find((b) => b.id === activeId);
    if (!board || !isNotebook(board)) return;
    const next = nbSetActivePage(board, pageId);
    const z = get().camera.zoom || 1;
    // Page top 24 px below the top of the screen.
    const y = viewportHeight / 2 / z - 24 / z;
    set((s) => ({
      boards: s.boards.map((b) => (b.id === activeId ? next : b)),
      camera: { ...s.camera, y },
      selectedIds: [],
      editingId: null,
    }));
    get().saveNow();
  },

  /**
   * Scrolling across a page boundary.
   *
   * The active page is always anchored at world Y = 0 – that is what lets
   * hit-testing, the selection overlay, the inline editor and export keep
   * working in plain world coordinates instead of every one of them having to
   * know which page it is on. So crossing into the next page swaps which page
   * is active AND slides the camera by one page stride in the opposite
   * direction. The two cancel out, so nothing moves on screen: the scroll
   * looks continuous while the coordinate system stays still.
   */
  reanchorPage: (delta, stride) => {
    const activeId = get().activeBoardId;
    const board = get().boards.find((b) => b.id === activeId);
    if (!board || !isNotebook(board)) return;

    const before = getActivePageIndexOf(board);
    const moved = nbStepPage(board, delta);
    const after = getActivePageIndexOf(moved);
    if (after === before) return; // already at the first or last page

    const shift = (after - before) * stride;
    set((s) => ({
      boards: s.boards.map((b) => (b.id === activeId ? moved : b)),
      camera: { ...s.camera, y: s.camera.y - shift },
      selectedIds: [],
      editingId: null,
    }));
    get().saveNow();
  },

  reanchorBoardPage: (boardId, delta, stride) => {
    if (boardId === get().activeBoardId) {
      get().reanchorPage(delta, stride);
      return;
    }
    const board = get().boards.find((b) => b.id === boardId);
    if (!board || !isNotebook(board)) return;
    const before = getActivePageIndexOf(board);
    const moved = nbStepPage(board, delta);
    const after = getActivePageIndexOf(moved);
    if (after === before) return;
    // Page AND camera in one update. Two updates would draw one frame with the
    // new anchor and the old camera - a visible jump of a whole page.
    const cam = board.camera ?? { ...DEFAULT_CAMERA };
    const shift = (after - before) * stride;
    set((s) => ({
      boards: s.boards.map((b) =>
        b.id === boardId ? { ...moved, camera: { ...cam, y: cam.y - shift } } : b
      ),
    }));
  },

```

```

goToBoardPage: (boardId, pageId, viewportHeight) => {
  if (boardId === get().activeBoardId) {
    get().goToPage(pageId, viewportHeight);
    return;
  }
  const board = get().boards.find((b) => b.id === boardId);
  if (!board || !isNotebook(board)) return;
  const next = nbSetActivePage(board, pageId);
  const cam = board.camera ?? { ...DEFAULT_CAMERA };
  const z = cam.zoom || 1;
  // Same framing as goToPage: the page top 24 px below the top of the pane.
  const y = viewportHeight / 2 / z - 24 / z;
  set((s) => ({
    boards: s.boards.map((b) => (b.id === boardId ? { ...next, camera: { ...cam, y } } : b)),
  }));
  get().saveNow();
},

setPageBackground: (pageId, patch) =>
  applyToActiveNotebook(set, get, (b) => nbSetPageBackground(b, pageId, patch), {
    keepSelection: true,
  }),
setPageSize: (pageSize) =>
  applyToActiveNotebook(set, get, (b) => ({ ...b, pageSize, updatedAt: Date.now() }), {
    keepSelection: true,
  }),

// ----- Boards -----
createNotebook: (name, folderId, pageSize = DEFAULT_PAGE_SIZE, color = "#3b82f6") => {
  const id = get().createBoard(name, folderId, color);
  set((s) => ({
    boards: s.boards.map((b) => (b.id === id ? makeNotebook(b, pageSize) : b)),
  }));
  get().saveNow();
  return id;
},

/**
 * Turns an existing canvas into a notebook. Whatever was already drawn
 * becomes page 1 rather than being stranded on a field the notebook view
 * never reads – see `makeNotebook`. There is no reverse action: flattening
 * a multi-page notebook back onto one canvas would have to either overlap
 * every page at the same coordinates or throw pages away.
 */
convertToNotebook: (boardId, pageSize = DEFAULT_PAGE_SIZE) => {
  const board = get().boards.find((b) => b.id === boardId);
  if (!board || !isNotebook(board)) return;
  set((s) => ({
    boards: s.boards.map((b) => (b.id === boardId ? makeNotebook(b, pageSize) : b)),
    selectedIds: [],
    editingId: null,
    // Old history entries carry no pageId and would restore onto whichever
    // page is open, so they cannot survive the conversion.
    history: { past: [], future: [] },
  }));
  get().saveNow();
},

/**
 * Opens a PDF as a notebook: one page per PDF page, each showing the
 * document underneath and accepting ink on top.
 *
 * The bytes go to IndexedDB once and each page keeps only a reference. The
 * alternative – rasterising every page into the workspace – is tens of
 * megabytes against a ~5 MB localStorage budget, and it fails on SAVE, so
 * you would lose an hour of annotation rather than be told up front.
 */
importDocumentAsNotebook: async (file, folderId = null, onProgress) => {
  const { officeKindOf, convertOfficeToPdf, LEGACY_OFFICE_MESSAGE } = await import(
    "../lib/whiteboard/officeImport"
  );
  const kind = officeKindOf(file);
  if (kind === "legacy") throw new Error(LEGACY_OFFICE_MESSAGE);
  const pdf = kind ? await convertOfficeToPdf(file, onProgress) : file;
  return get().importPdfAsNotebook(pdf, folderId);
},

importPdfAsNotebook: async (file, folderId = null) => {

```

```

const { readPdfFile, openPdf, pdfPageSize, notebookSizeForPdf } = await import(
  "../lib/whiteboard/pdf"
);
const { putPdf } = await import("../lib/whiteboard/pdfstore");

const bytes = await readPdfFile(file);
const doc = await openPdf(bytes);
const pageCount = doc.numPages;
if (pageCount < 1) throw new Error(`${file.name} has no pages.`);

// Page 1 sets the sheet size for the notebook. Mixed-size PDFs exist but
// are rare; the alternative is a per-page sheet size, which would make
// continuous scrolling jump about.
const sheet = notebookSizeForPdf(await pdfPageSize(doc, 1));

const docId = `pdf_${Date.now().toString(36)}_${nanoid(6)}`;
await putPdf({ id: docId, name: file.name, bytes, pageCount, addedAt: Date.now() });

// A converted Word / PowerPoint file is pictures only; the text captured
// while converting it is stored beside it under the same id. Failing to
// keep it costs text selection and search, never the import.
const { takeOfficeText } = await import("../lib/whiteboard/pdfTextStore");
const officeText = takeOfficeText(file);
if (officeText) {
  try {
    const { saveOfficeText } = await import("../lib/whiteboard/pdfTextIndex");
    await saveOfficeText(docId, officeText);
  } catch {
    /* the notebook is still whole */
  }
}

const now = Date.now();
const pages = Array.from({ length: pageCount }, (_, i) => ({
  id: `pg_${docId}_${i + 1}`,
  elements: [],
  pdf: { docId, pageNumber: i + 1 },
  createdAt: now,
  updatedAt: now,
})));

const id = nanoid(8);
const name = makeUniqueName(
  file.name.replace(/\.pdf$/i, ""),
  get().boards.map(b => b.name)
);
const board: Board = {
  id,
  name,
  folderId,
  elements: [],
  camera: { ...DEFAULT_CAMERA },
  createdAt: now,
  updatedAt: now,
  accessedAt: now,
  color: "#ef4444",
  kind: "notebook",
  pages,
  activePageId: pages[0].id,
  customPageSize: sheet,
  pdfDocIds: [docId],
  // A PDF page IS the paper; drawing dots over a lecture slide is noise.
  background: { pattern: "plain" },
};

set((s) => ({
  boards: [...s.boards, board],
  activeBoardId: id,
  camera: { ...DEFAULT_CAMERA },
  selectedIds: [],
  editingId: null,
  history: { past: [], future: [] },
})));
get().saveNow();
return id;
},
/**

```

```

* Inserts pages from another board into the active notebook.
*
* A whiteboard counts as a single page, so "put this sketch in front of my
* lecture notes" works without converting it first.
*
* Nothing is moved: the source keeps its pages. Copying rather than moving
* is the safe default - an accidental insert costs a delete, an accidental
* move costs work you cannot get back.
*/
insertPagesFromBoard: (sourceBoardId, range, atIndex, targetBoardId) => {
  const targetId = targetBoardId ?? get().activeBoardId;
  const target = get().boards.find((b) => b.id === targetId);
  const source = get().boards.find((b) => b.id === sourceBoardId);
  if (!target || !isNotebook(target) || !source) return 0;

  let indices: number[];
  if (Array.isArray(range)) {
    indices = sanitizePageIndices(range);
  } else if (range && Array.isArray((range as { indices?: number[] }).indices)) {
    indices = sanitizePageIndices((range as { indices: number[] }).indices);
  } else {
    const sourcePageCount = isNotebook(source) ? getPages(source).length : 1;
    const r = resolvePageRange(
      sourcePageCount,
      (range as { from?: number }).from ?? 1,
      (range as { to?: number }).to ?? sourcePageCount
    );
    indices = r.indices;
  }
  if (indices.length === 0) return 0;

  let incoming: NotebookPage[];
  if (isNotebook(source)) {
    incoming = copyPagesFrom(source, indices, target.id, () => nanoid(8));
  } else {
    const asPage = {
      id: "tmp",
      elements: source.elements ?? [],
      background: source.background,
      createdAt: Date.now(),
      updatedAt: Date.now(),
    };
    incoming = copyPagesFrom(
      { ...source, kind: "notebook", pages: [asPage] } as Board,
      [0],
      target.id,
      () => nanoid(8)
    );
  }
  if (incoming.length === 0) return 0;

  // Snapshot target's current pages for single-step undo!
  const historySnapshot: HistoryEntry = {
    activeAt: get().activeBoardId ?? target.id,
    boardId: target.id,
    pageId: target.activePageId,
    pages: clonePagesForHistory(target.pages ?? []),
    elements: [],
    at: Date.now(),
  };

  // Every PDF the inserted pages still point at has to be listed on the new
  // board too, or the notebook's own record of its documents goes stale.
  const addedDocIds = incoming
    .map((p) => p.pdf?.docId)
    .filter((d): d is string => typeof d === "string");

  set((s) => ({
    boards: s.boards.map((b) =>
      b.id === targetId
        ? {
            ...nbInsertPages(b, incoming, atIndex),
            pdfDocIds: [...new Set([...(b.pdfDocIds ?? []), ...addedDocIds])],
          }
        : b
    ),
    history: {
      past: [...s.history.past, historySnapshot].slice(-HISTORY_PAST_MAX),
    }
  }));

```

```

    future: [],
  },
  selectedIds: targetId === s.activeBoardId ? [] : s.selectedIds,
  editingId: targetId === s.activeBoardId ? null : s.editingId,
}));
get().saveNow();
scheduleHistorySave(() => persistableHistory(get()));
return incoming.length;
},

/**
 * Inserts a run of a PDF's pages into the active notebook.
 *
 * The bytes go to IndexedDB once and each page keeps only a reference, for
 * the reasons in `importPdfAsNotebook`. The WHOLE document is stored even
 * when only three pages are inserted: pages are referenced by number into
 * the original file, and re-cutting the PDF to match the range would cost
 * more than the space it saves and lose the ability to insert more later.
 */
insertPdfPages: async (file, range, atIndex) => {
  const targetId = get().activeBoardId;
  const target = get().boards.find((b) => b.id === targetId);
  if (!target || !isNotebook(target)) {
    throw new Error("Pages can only be inserted into a notebook.");
  }

  const { readPdfFile, openPdf } = await import("../lib/whiteboard/pdf");
  const { putPdf } = await import("../lib/whiteboard/pdfstore");

  const bytes = await readPdfFile(file);
  const doc = await openPdf(bytes);
  const pageCount = doc.numPages;
  if (pageCount < 1) throw new Error(`${file.name} has no pages.`);

  const { indices } = resolvePageRange(pageCount, range.from, range.to);
  if (indices.length === 0) return 0;

  const docId = `pdf_${Date.now().toString(36)}_${nanoid(6)}`;
  await putPdf({ id: docId, name: file.name, bytes, pageCount, addedAt: Date.now() });

  const incoming = indices.map((i) => pageFromPdf(docId, i + 1));

  set((s) => ({
    boards: s.boards.map((b) =>
      b.id === targetId
        ? {
            ...nbInsertPages(b, incoming, atIndex),
            pdfDocIds: [...new Set([...(b.pdfDocIds ?? []), docId])],
          }
        : b
    ),
    selectedIds: [],
    editingId: null,
  }));
  get().saveNow();
  return incoming.length;
},

/**
 * Appends one page per image to the active notebook.
 *
 * Each image is scaled to FIT its sheet and centred – never cropped, because
 * silently cutting the edge off someone's scanned worksheet is destructive
 * and invisible. They are locked, like a PDF page: a backdrop to write on,
 * not an object to drag out of place by accident.
 */
addImagePages: (images) => {
  const activeId = get().activeBoardId;
  const board = get().boards.find((b) => b.id === activeId);
  if (!board || !isNotebook(board) || images.length === 0) return;

  const dim = pageSizeOf(board);
  const now = Date.now();
  const pages = images.map((img, i) => {
    const scale = Math.min(dim.width / img.width, dim.height / img.height);
    const w = img.width * scale;
    const h = img.height * scale;
    return {

```



```

id: `pg_img_${now.toString(36)}_${i}_${nanoid(4)}`,
elements: [
  {
    id: nanoid(8),
    type: "image",
    x: (dim.width - w) / 2,
    y: (dim.height - h) / 2,
    width: w,
    height: h,
    angle: 0,
    opacity: 1,
    src: img.src,
    locked: true,
    createdAt: now,
    updatedAt: now,
  } as unknown as WhiteboardElement,
],
background: { pattern: "plain" as const },
createdAt: now,
updatedAt: now,
};
});

set((s) => ({
  boards: s.boards.map((b) =>
    b.id === activeId
      ? { ...b, pages: [...getPages(b), ...pages], activePageId: pages[0].id, updatedAt: now }
      : b
  ),
  selectedIds: [],
  editingId: null,
}));
get().saveNow();
},

addCaptureToBoard: (boardId, image) => {
  const target = get().boards.find((b) => b.id === boardId);
  if (!target) return null;
  const now = Date.now();
  const elementId = nanoid(8);
  const imageAt = (rect: { x: number; y: number; width: number; height: number }): ImageElement => ({
    id: elementId,
    type: "image",
    ...rect,
    rotation: 0,
    opacity: 1,
    src: image.src,
    naturalWidth: image.naturalWidth,
    naturalHeight: image.naturalHeight,
    createdAt: now,
    updatedAt: now,
  });
  // The step belongs to the document that RECEIVED the picture. Filed under
  // the one you captured in, its whole-document snapshot of the other
  // notebook was restored by an Undo there - wiping everything written in
  // that notebook since. Kept with the target, Undo in the target removes
  // it in its proper turn, after any later edits.
  const activeAt = boardId;
  const pushHistory = (entry: HistoryEntry) => (s: WhiteboardState) => ({
    past: [...s.history.past, entry].slice(-HISTORY_PAST_MAX),
    future: s.history.future.filter((e) => contextOf(e) !== activeAt),
  });

  if (isNotebook(target)) {
    const pages = getPages(target);
    const after = image.afterPageId ? pages.findIndex((p) => p.id === image.afterPageId) : -1;
    const at = after >= 0 ? after + 1 : pages.length;
    // A plain new sheet with the NOTEBOOK's paper, like "add page"; not
    // locked like an image page, because a clipping is something you move.
    const page: NotebookPage = {
      ...createPage(target.background),
      elements: [imageAt(placeCaptureOnPage(pageSizeOf(target), image.size))],
    };
    const entry: HistoryEntry = {
      activeAt,
      boardId,
      pageId: target.activePageId,
      pages: clonePagesForHistory(pages),
    };
  }
}

```

```

    elements: [],
    at: now,
  };
  set((s) => ({
    boards: s.boards.map((b) => (b.id === boardId ? nbInsertPages(b, [page], at) : b)),
    history: pushHistory(entry)(s),
    selectedIds: boardId === s.activeBoardId ? [] : s.selectedIds,
    editingId: boardId === s.activeBoardId ? null : s.editingId,
  }));
  get().saveNow();
  scheduleHistorySave(() => persistableHistory(get()));
  return { elementId, pageId: page.id, pageNumber: at + 1 };
}

const existing = getBoardElements(target);
const entry: HistoryEntry = {
  activeAt,
  boardId,
  elements: cloneElementsForHistory(existing),
  at: now,
};
const el = imageAt(placeCaptureBeside(existing, image.size));
set((s) => ({
  boards: s.boards.map((b) => (b.id === boardId ? mapBoardElements(b, (els) => [...els, el]) : b)),
  history: pushHistory(entry)(s),
}));
get().saveNow();
scheduleHistorySave(() => persistableHistory(get()));
return { elementId };
},

pruneOrphanedAssets: async () => {
  // One shared collector (unit-tested in workspace.ts): live boards,
  // templates, AND the trash undo buffer. A picture or PDF on a deleted
  // board must survive until Undo expires – pruning it early would make
  // "Undo" restore a board with holes in it.
  const { pdfIds, imageIds } = collectReferencedAssets(
    get().boards,
    get().pageTemplates,
    (get().lastDeleted?.removedBoards ?? []).map((r) => r.board)
  );

  try {
    const [{ pruneUnusedPdfs }, { pruneUnusedImages }] = await Promise.all([
      import("../lib/whiteboard/pdfstore"),
      import("../lib/whiteboard/imagestore"),
    ]);
    await Promise.all([pruneUnusedPdfs(pdfIds), pruneUnusedImages(imageIds)]);
  } catch {
    // Reclaiming space is housekeeping: failing to do it must never break
    // the delete the user actually asked for.
  }
},

createBoard: (name, folderId, color = "#3b82f6") => {
  const id = nanoid(8);
  const existing = get().boards.map((b) => b.name);
  const uniqueName = makeUniqueName(name, existing);
  const newBoard: Board = {
    id,
    name: uniqueName,
    folderId: folderId ?? null,
    elements: [],
    background: { ...get().defaultPaper },
    camera: { ...DEFAULT_CAMERA },
    createdAt: Date.now(),
    updatedAt: Date.now(),
    accessedAt: Date.now(),
    starred: false,
    tags: [],
    color,
  };
  set((s) => ({
    folders: folderId
      ? s.folders.map((f) => (f.id === folderId ? { ...f, expanded: true } : f))
      : s.folders,
    boards: [
      // Capture the outgoing board's viewport before switching away

```

```

    ...s.boards.map((b) => (b.id === s.activeBoardId ? { ...b, camera: { ...s.camera } } : b)),
    newBoard,
  ],
  activeBoardId: id,
  camera: { ...DEFAULT_CAMERA },
  selectedIds: [],
  editingId: null,
  history: { past: [], future: [] },
});
get().saveNow();
return id;
},

renameBoard: (boardId, name) => {
  set((s) => ({
    boards: s.boards.map((b) =>
      b.id === boardId ? { ...b, name: name.trim() || b.name, updatedAt: Date.now() } : b
    ),
  }));
  get().saveNow();
},

deleteBoard: (boardId) => {
  set((s) => {
    // Strip cross-board connections in OTHER boards that pointed at the
    // deleted board (they would silently dangle forever).
    const remainingBoards = s.boards
      .filter((b) => b.id !== boardId)
      .map((b) =>
        mapAllBoardElements(b, (els) =>
          els.filter((el) => {
            if (el.type !== "connection") return true;
            const conn = el as ConnectionElement;
            return conn.source.boardId !== boardId && conn.target.boardId !== boardId;
          })
        )
      );
  });

  // Deleting a NON-active board must not hijack the user's current canvas,
  // camera, selection, or undo history.
  const wasActive = s.activeBoardId === boardId;
  if (!wasActive) {
    return { boards: remainingBoards };
  }
  const nextActive = remainingBoards[0]?.id || null;
  const nextBoard = remainingBoards.find((b) => b.id === nextActive);
  return {
    boards: remainingBoards,
    activeBoardId: nextActive,
    camera: nextBoard?.camera || { ...DEFAULT_CAMERA },
    selectedIds: [],
    editingId: null,
    history: { past: [], future: [] },
  };
};
get().saveNow();
// Deleting a board is the one moment orphans appear. Fire and forget: the
// delete has already happened and must not wait on housekeeping.
void get().pruneOrphanedAssets();
},

duplicateBoard: (boardId) => {
  const board = get().boards.find((b) => b.id === boardId);
  if (!board) return null;
  const id = nanoid(8);
  const uniqueName = makeUniqueName(`${board.name} (Copy)`, get().boards.map((b) => b.name));

  // Create a mapping from old element IDs to new element IDs
  const idMap = new Map<string, string>();
  for (const el of allElements(board)) {
    idMap.set(el.id, nanoid(8));
  }

  const remapElement = (el: WhiteboardElement): WhiteboardElement => {
    const newId = idMap.get(el.id) || nanoid(8);
    if (el.type === "connection") {
      const conn = el as ConnectionElement;
      return {

```

```

    ...conn,
    id: newId,
    source: {
      ...conn.source,
      boardId: conn.source.boardId === boardId ? id : conn.source.boardId,
      elementId: idMap.get(conn.source.elementId) || conn.source.elementId,
    },
    target: {
      ...conn.target,
      boardId: conn.target.boardId === boardId ? id : conn.target.boardId,
      elementId: idMap.get(conn.target.elementId) || conn.target.elementId,
    },
    updatedAt: Date.now(),
  };
}
return {
  ...el,
  id: newId,
  updatedAt: Date.now(),
};
};
};

const newBoard: Board = {
  ...board,
  id,
  name: uniqueName,
  // Copies every page of a notebook, giving each element a fresh id.
  ...cloneBoardContent(board, remapElement),
  createdAt: Date.now(),
  updatedAt: Date.now(),
  accessedAt: Date.now(),
};
set((s) => ({
  boards: [
    // Capture the outgoing board's viewport before switching away
    ...s.boards.map((b) => (b.id === s.activeBoardId ? { ...b, camera: { ...s.camera } } : b)),
    newBoard,
  ],
  activeBoardId: id,
  camera: { ...board.camera },
  selectedIds: [],
  editingId: null,
  history: { past: [], future: [] },
}));
get().saveNow();
return id;
},

duplicateNotebook: (sourceBoardId, options) => {
  const source = get().boards.find((b) => b.id === sourceBoardId);
  if (!source) return null;
  const targetBoardId = nanoid(8);
  const uniqueName = makeUniqueName(
    options.name?.trim() || `${source.name} (Copy)`,
    get().boards.map((b) => b.name)
  );
};

let copiedPages: NotebookPage[];
if (isNotebook(source)) {
  const sourcePages = getPages(source);
  const indices = options.pageIndices
    ? sanitizePageIndices(options.pageIndices)
    : sourcePages.map((_, i) => i);
  copiedPages = copyPagesFrom(source, indices, targetBoardId, () => nanoid(8));
} else {
  // A canvas has no pages, but it does have contents. Copy it through
  // copyPagesFrom like every other path, so connections are remapped to
  // the new board with fresh element ids instead of dangling at the old
  // board/elements.
  const asPage = {
    id: "tmp",
    elements: source.elements ?? [],
    background: source.background,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };
  copiedPages = copyPagesFrom(
    { ...source, kind: "notebook", pages: [asPage] } as Board,

```

```

    [0],
    targetBoardId,
    () => nanoid(8)
  );
}

if (copiedPages.length === 0) return null;

const referencedDocIds = [
  ...new Set(
    copiedPages
      .map((p) => p.pdf?.docId)
      .filter((d): d is string => typeof d === "string")
  ),
];

const newBoard: Board = {
  ...source,
  id: targetBoardId,
  name: uniqueName,
  folderId: options.folderId !== undefined ? options.folderId : (source.folderId ?? null),
  kind: "notebook",
  elements: [], // notebooks must never populate board.elements directly
  pages: copiedPages,
  activePageId: copiedPages[0].id,
  pdfDocIds: referencedDocIds,
  createdAt: Date.now(),
  updatedAt: Date.now(),
  accessedAt: Date.now(),
};

set((s) => ({
  boards: [...s.boards, newBoard],
}));
get().saveNow();
return targetBoardId;
},

setActiveBoard: (boardId) => {
  const board = get().boards.find((b) => b.id === boardId);
  if (!board) return;
  set((s) => ({
    activeBoardId: boardId,
    camera: board.camera || { ...DEFAULT_CAMERA },
    selectedIds: [],
    editingId: null,
    // History is deliberately KEPT: entries carry their board, and undo only
    // takes the live board's (see lastIndexForBoard). Wiping it here meant
    // that in split view every tap across the divider lost your undo.
    boards: s.boards.map((b) => {
      if (b.id === boardId) return { ...b, accessedAt: Date.now() };
      // Persist the outgoing board's viewport in memory immediately –
      // the debounced save would otherwise never capture it.
      if (b.id === s.activeBoardId) return { ...b, camera: { ...s.camera } };
      return b;
    }),
  }));
  get().saveNow();
},

moveBoard: (boardId, newFolderId) => {
  const s = get();
  const board = s.boards.find((b) => b.id === boardId);
  if (!board) return;
  if (newFolderId !== null && !s.folders.some((f) => f.id === newFolderId)) return;
  if (board.folderId === newFolderId) return;
  set((prev) => ({
    boards: prev.boards.map((b) => (b.id === boardId ? { ...b, folderId: newFolderId } : b)),
  }));
  get().saveNow();
},

setBoardColor: (boardId, color) => {
  set((s) => ({
    boards: s.boards.map((b) => (b.id === boardId ? { ...b, color } : b)),
  }));
  get().saveNow();
},

```

```

/** Per-board paper: colour and/or pattern. Undefined colour = follow theme.
 *
 * By default only the board default changes: pages with their own paper keep
 * it, and new pages start with the default. With `allPages`, every existing
 * page in a notebook is rewritten too.
 */
setBoardBackground: (boardId, patch, opts) => {
  set((s) => ({
    boards: s.boards.map((b) => {
      if (b.id !== boardId) return b;
      const background = { pattern: "dots" as const, ...(b.background ?? {}), ...patch };
      if (!opts?.allPages || !isNotebook(b) || !Array.isArray(b.pages)) {
        return { ...b, background, updatedAt: Date.now() };
      }
    }
    return {
      ...b,
      background,
      pages: b.pages.map((p) => ({
        ...p,
        background: { pattern: "dots" as const, ...(p.background ?? {}), ...patch },
        updatedAt: Date.now(),
      })),
      updatedAt: Date.now(),
    };
  })),
  get().saveNow();
},

setDefaultPaper: (patch) => {
  set((s) => ({ defaultPaper: { ...s.defaultPaper, ...patch } }));
  get().saveNow();
},

toggleStarred: (boardId) => {
  set((s) => ({
    boards: s.boards.map((b) => (b.id === boardId ? { ...b, starred: !b.starred } : b)),
  }));
  get().saveNow();
},

addTag: (boardId, tag) => {
  const clean = tag.trim().replace(/^#/, "");
  if (!clean) return;
  set((s) => ({
    boards: s.boards.map((b) =>
      b.id === boardId && !b.tags?.includes(clean)
        ? { ...b, tags: [...(b.tags || []), clean] }
        : b
    ),
  }));
  get().saveNow();
},

removeTag: (boardId, tag) => {
  set((s) => ({
    boards: s.boards.map((b) =>
      b.id === boardId ? { ...b, tags: (b.tags || []).filter((t) => t !== tag) } : b
    ),
  }));
  get().saveNow();
},

findOrCreateBoardByName: (name) => {
  const existing = get().boards.find((b) => b.name.toLowerCase() === name.toLowerCase());
  if (existing) return existing.id;
  return get().createBoard(name, get().activeFolderId);
},

searchBoards: (query) => {
  const q = query.toLowerCase().trim();
  if (!q) return [];
  const results: Array<{ boardId: string; boardName: string; snippet: string; elementId: string }> = [];
  for (const board of get().boards) {
    if (board.name.toLowerCase().includes(q)) {
      results.push({
        boardId: board.id,
        boardName: board.name,

```

```

        snippet: `Board name matches "${query}"`,
        elementId: "",
    });
}
for (const el of allElements(board)) {
    if (el.type === "text" || el.type === "sticky") {
        const text = (el as { text?: string }).text || "";
        if (text.toLowerCase().includes(q)) {
            results.push({
                boardId: board.id,
                boardName: board.name,
                snippet: text.slice(0, 100),
                elementId: el.id,
            });
        }
    }
}
return results;
},

getRecentBoards: (limit = 5) => {
    return [...get().boards]
        .sort((a, b) => (b.accessedAt || b.createdAt) - (a.accessedAt || a.createdAt))
        .slice(0, limit);
},

getStarredBoards: () => {
    return get().boards.filter((b) => b.starred);
},

// ----- Elements -----
addElement: (el, commit = true) => {
    const activeId = get().activeBoardId;
    if (!activeId) return;
    if (commit) get().commitHistory();
    set((s) => ({
        boards: s.boards.map((b) =>
            b.id === activeId ? mapBoardElements(b, (els) => [...els, el]) : b
        ),
    }));
    get().saveNow();
},

addElementToBoard: (boardId, el, commit = true) => {
    const activeId = get().activeBoardId;
    if (!activeId || el.length === 0) return;
    if (commit) get().commitHistory();
    set((s) => ({
        boards: s.boards.map((b) =>
            b.id === activeId ? mapBoardElements(b, (prev) => [...prev, ...els]) : b
        ),
    }));
    get().saveNow();
},

addElementToBoard: (boardId, el, commit = true) => {
    if (commit) get().commitHistory();
    set((s) => ({
        boards: s.boards.map((b) =>
            b.id === boardId ? mapBoardElements(b, (els) => [...els, el]) : b
        ),
    }));
    get().saveNow();
},

deleteElementFromBoard: (boardId, ids, commit = true) => {
    if (!ids.length) return;
    if (commit) get().commitHistory();
    set((s) => ({
        boards: s.boards.map((b) =>
            b.id === boardId
                ? mapBoardElements(b, (els) => els.filter((el) => !ids.includes(el.id)))
                : b
        ),
    }));
    // If the deleted element was being edited, close the editor
    editingId: s.editingId && ids.includes(s.editingId) ? null : s.editingId,
});

```

```

    get().saveNow();
  },

  updateElement: (id, patch, commit = false) => {
    const activeId = get().activeBoardId;
    if (!activeId) return;
    if (commit) get().commitHistory();
    set((s) => ({
      boards: s.boards.map((b) => {
        if (b.id !== activeId) return b;
        return mapBoardElements(b, (els) =>
          els.map((el) =>
            el.id === id ? ({ ...el, ...patch, updatedAt: Date.now() } as WhiteboardElement) : el
          )
        );
      }
    }));
    get().saveNow();
  },

  updateElements: (ids, patch, commit = false) => {
    const activeId = get().activeBoardId;
    if (!activeId || ids.length === 0) return;
    if (commit) get().commitHistory();
    set((s) => ({
      boards: s.boards.map((b) => {
        if (b.id !== activeId) return b;
        return mapBoardElements(b, (els) =>
          els.map((el) =>
            ids.includes(el.id) ? ({ ...el, ...patch, updatedAt: Date.now() } as WhiteboardElement) : el
          )
        );
      }
    }));
    get().saveNow();
  },

  deleteElements: (ids, commit = true) => {
    const activeId = get().activeBoardId;
    if (!activeId || ids.length === 0) return;
    // Locked elements can be selected but never destroyed from the canvas.
    const board = get().boards.find((b) => b.id === activeId);
    const deletable = ids.filter((id) => {
      const el = board ? getBoardElements(board).find((e) => e.id === id) : undefined;
      return el ? !el.locked : true; // unknown ids pass through (connections cleanup)
    });
    if (deletable.length === 0) return;
    ids = deletable;
    if (commit) get().commitHistory();
    set((s) => ({
      boards: s.boards.map((b) => {
        if (b.id !== activeId) return b;
        // Filter out elements and any connection referencing them
        return mapBoardElements(b, (els) =>
          els.filter((el) => {
            if (ids.includes(el.id)) return false;
            if (el.type === "connection") {
              const conn = el as ConnectionElement;
              if (ids.includes(conn.source.elementId) || ids.includes(conn.target.elementId)) {
                return false;
              }
            }
            return true;
          })
        );
      }
    }));
    selectedIds: s.selectedIds.filter((id) => !ids.includes(id)),
    editingId: s.editingId && ids.includes(s.editingId) ? null : s.editingId,
  }));
    get().saveNow();
  },

  deleteSelected: () => {
    get().deleteElements(get().selectedIds, true);
  },

  duplicateElements: (ids) => {
    const activeId = get().activeBoardId;

```



```

    if (!activeId || ids.length === 0) return [];
    get().commitHistory();
    const board = get().boards.find((b) => b.id === activeId);
    if (!board) return [];

    const cloned: WhiteboardElement[] = [];
    const idMap = new Map<string, string>();

    for (const id of ids) {
      const el = getBoardElements(board).find((e) => e.id === id);
      if (el && el.type !== "connection") {
        const newId = nanoid(8);
        idMap.set(id, newId);
        cloned.push({
          ...el,
          id: newId,
          x: el.x + 30,
          y: el.y + 30,
          createdAt: Date.now(),
          updatedAt: Date.now(),
        });
      }
    }

    set((s) => ({
      boards: s.boards.map((b) =>
        b.id === activeId ? mapBoardElements(b, (els) => [...els, ...cloned]) : b
      ),
      selectedIds: cloned.map((c) => c.id),
    }));
    get().saveNow();
    return cloned.map((c) => c.id);
  },

  unlockElements: (ids) => {
    const activeId = get().activeBoardId;
    const board = get().boards.find((b) => b.id === activeId);
    if (!board || ids.length === 0) return 0;
    const want = new Set(ids);
    const count = getBoardElements(board).filter((el) => want.has(el.id) && el.locked).length;
    if (count === 0) return 0;
    get().commitHistory();
    set((s) => ({
      boards: s.boards.map((b) =>
        b.id !== activeId
          ? b
          : mapBoardElements(b, (els) =>
              els.map((el) => (want.has(el.id) && el.locked ? { ...el, locked: false, updatedAt: Date.now() } : el))
            )
        ),
    }));
    get().saveNow();
    return count;
  },

  setLockedMode: (on) => {
    // Entering or leaving the mode starts from an empty selection: what was
    // picked belongs to the other set (unlocked vs locked).
    set({ lockedMode: on, selectedIds: [] });
    if (on && get().tool !== "select" && get().tool !== "lasso") set({ tool: "select" });
  },

  setImageStudioId: (id) => set({ imageStudioId: id }),

  toggleLock: (ids) => {
    const activeId = get().activeBoardId;
    if (!activeId || ids.length === 0) return;
    get().commitHistory();
    set((s) => ({
      boards: s.boards.map((b) => {
        if (b.id !== activeId) return b;
        return mapBoardElements(b, (els) =>
          els.map((el) =>
            ids.includes(el.id) ? { ...el, locked: !el.locked, updatedAt: Date.now() } : el
          )
        );
      })
    }));
  },

```

```

    }));
    get().saveNow();
  },

  bringToFront: (ids) => {
    const activeId = get().activeBoardId;
    if (!activeId || ids.length === 0) return;
    get().commitHistory();
    set((s) => ({
      boards: s.boards.map((b) => {
        if (b.id !== activeId) return b;
        return mapBoardElements(b, (els) => [
          ...els.filter((e) => !ids.includes(e.id)),
          ...els.filter((e) => ids.includes(e.id)),
        ]);
      }),
    }));
    get().saveNow();
  },

  sendToBack: (ids) => {
    const activeId = get().activeBoardId;
    if (!activeId || ids.length === 0) return;
    get().commitHistory();
    set((s) => ({
      boards: s.boards.map((b) => {
        if (b.id !== activeId) return b;
        return mapBoardElements(b, (els) => [
          ...els.filter((e) => ids.includes(e.id)),
          ...els.filter((e) => !ids.includes(e.id)),
        ]);
      }),
    }));
    get().saveNow();
  },

  bringForward: (ids) => {
    const activeId = get().activeBoardId;
    if (!activeId || ids.length === 0) return;
    get().commitHistory();
    set((s) => ({
      boards: s.boards.map((b) => {
        if (b.id !== activeId) return b;
        // Move each selected element up by ONE position (swap with the
        // element above it), not all the way to the top.
        return mapBoardElements(b, (prev) => {
          const elements = [...prev];
          for (let i = elements.length - 2; i >= 0; i--) {
            if (ids.includes(elements[i].id) && !ids.includes(elements[i + 1].id)) {
              [elements[i], elements[i + 1]] = [elements[i + 1], elements[i]];
            }
          }
          return elements;
        });
      }),
    }));
    get().saveNow();
  },

  sendBackward: (ids) => {
    const activeId = get().activeBoardId;
    if (!activeId || ids.length === 0) return;
    get().commitHistory();
    set((s) => ({
      boards: s.boards.map((b) => {
        if (b.id !== activeId) return b;
        // Move each selected element down by ONE position (swap with the
        // element below it), not all the way to the back.
        return mapBoardElements(b, (prev) => {
          const elements = [...prev];
          for (let i = 1; i < elements.length; i++) {
            if (ids.includes(elements[i].id) && !ids.includes(elements[i - 1].id)) {
              [elements[i], elements[i - 1]] = [elements[i - 1], elements[i]];
            }
          }
          return elements;
        });
      }),
    }));
  },

```

```

    });
    get().saveNow();
  },

  // ----- Connections -----
  createConnection: (source, target, linkType = "relates", routing = "curved") => {
    const activeId = get().activeBoardId;
    if (!activeId) return "";
    get().commitHistory();
    const connId = nanoid(8);
    const newConn: ConnectionElement = {
      id: connId,
      type: "connection",
      x: 0,
      y: 0,
      rotation: 0,
      opacity: 1,
      source: {
        boardId: source.boardId,
        elementId: source.elementId,
        anchor: source.anchor || "auto",
      },
      target: {
        boardId: target.boardId,
        elementId: target.elementId,
        anchor: target.anchor || "auto",
      },
      linkType,
      routing,
      dashStyle: "solid",
      startArrowhead: "none",
      endArrowhead: "arrow",
      strokeWidth: 2.5,
      createdAt: Date.now(),
      updatedAt: Date.now(),
    };
    get().addElement(newConn, false);
    return connId;
  },

  // ----- Selection & Inline Editing -----
  setSelected: (ids) => set({ selectedIds: ids }),
  addToSelection: (id) =>
    set((s) => ({ selectedIds: s.selectedIds.includes(id) ? s.selectedIds : [...s.selectedIds, id] })),
  removeFromSelection: (id) =>
    set((s) => ({ selectedIds: s.selectedIds.filter((sid) => sid !== id) })),
  toggleSelection: (id) =>
    set((s) => ({
      selectedIds: s.selectedIds.includes(id)
        ? s.selectedIds.filter((sid) => sid !== id)
        : [...s.selectedIds, id],
    })),
  clearSelection: () => set({ selectedIds: [] }),
  selectAll: () => {
    const board = get().boards.find((b) => b.id === get().activeBoardId);
    if (!board) return;
    // Obeys the Select tool's filter, so "select all" with only Handwriting
    // ticked grabs every stroke on the page and nothing else.
    const filter = effectiveSelectFilter(get().touchSettings.selectFilter);
    // In locked-items mode "all" means every LOCKED thing, ready to unlock.
    const lockedMode = get().lockedMode;
    set({
      selectedIds: getBoardElements(board)
        .filter((el) => (lockedMode ? !el.locked && el.type !== "connection" : !el.locked && passesSelectFilter(el, filter)))
        .map((el) => el.id),
    });
  },

  selectByType: (type) => {
    const board = get().boards.find((b) => b.id === get().activeBoardId);
    if (!board) return;
    const elements = getBoardElements(board);
    const ids = elements
      .filter((el) => {
        if (el.locked) return false;
        if (type === "stroke") return el.type === "stroke";
        if (type === "text") return el.type === "text";
      })
      .map((el) => el.id);
  },

```

```

    if (type === "sticky") return el.type === "sticky";
    if (type === "image") return el.type === "image";
    if (type === "connection") return el.type === "connection";
    if (type === "shape") {
      return [
        "rectangle",
        "ellipse",
        "diamond",
        "triangle",
        "star",
        "cloud",
        "line",
        "arrow",
      ].includes(el.type);
    }
    return false;
  })
  .map((el) => el.id);
set({ selectedIds: ids });
},

invertSelection: () => {
  const board = get().boards.find((b) => b.id === get().activeBoardId);
  if (!board) return;
  const currentSelected = new Set(get().selectedIds);
  const elements = getBoardElements(board);
  const inverted = elements
    .filter((el) => !el.locked && !currentSelected.has(el.id))
    .map((el) => el.id);
  set({ selectedIds: inverted });
},

setEditingId: (id) => {
  set({ editingId: id });
  if (id) {
    set({ selectedIds: [id] });
  }
},

discardAbandonedText: (id, boardId) => {
  const board = get().boards.find((b) => b.id === boardId);
  if (!board) return false;
  const el = allElements(board).find((e) => e.id === id);
  if (!isAbandonedTextBox(el)) return false;

  const withoutIt = (els: WhiteboardElement[]) =>
    els.filter((e) => {
      if (e.id === id) return false;
      if (e.type === "connection") {
        const c = e as ConnectionElement;
        if (c.source.elementId === id || c.target.elementId === id) return false;
      }
      return true;
    });

  set((s) => ({
    boards: s.boards.map((b) => (b.id === boardId ? mapAllBoardElements(b, withoutIt) : b)),
    selectedIds: s.selectedIds.filter((x) => x !== id),
  }));

  // Undo. Placing the box pushed a snapshot of the page BEFORE it existed
  // (and the editor's first auto-resize may have pushed another that holds
  // the empty box). Left on the stack, the next undo would "undo" nothing
  // visible - or resurrect the empty box. So pop every entry at the top
  // that is identical to the page as it is now, ignoring the discarded box.
  // Comparing CONTENT, not just ids, is what stops this eating a real edit:
  // an earlier move leaves a snapshot with the old position, which differs.
  const after = get().boards.find((b) => b.id === boardId);
  if (after) {
    const pageEls = (pageId?: string) => {
      if (!pageId || !isNotebook(after)) return getBoardElements(after);
      return getPages(after).find((pg) => pg.id === pageId)?.elements ?? [];
    };
    const past = [...get().history.past];
    let popped = 0;
    while (past.length > 0) {
      const top = past[past.length - 1];
      if (top.boardId !== boardId) break;
    }
  }
}

```

```

    const same = JSON.stringify(withoutIt(top.elements)) === JSON.stringify(pageEls(top.pageId));
    if (!same) break;
    past.pop();
    popped++;
  }
  if (popped > 0) set((s) => ({ history: { ...s.history, past } }));
}

get().saveNow();
return true;
},

// ----- Camera -----
setCamera: (camera) =>
  set((s) => ({ camera: boundCamera(s, { ...s.camera, ...camera }) })),

panCamera: (dx, dy) =>
  set((s) => ({
    camera: boundCamera(s, {
      ...s.camera,
      x: s.camera.x - dx / s.camera.zoom,
      y: s.camera.y - dy / s.camera.zoom,
    }),
  })),

setCanvasViewport: (vp) => {
  const cur = get().canvasViewport;
  if (!(vp.width > 0) || !(vp.height > 0)) return;
  if (vp.width === cur.width && vp.height === cur.height) return;
  set({ canvasViewport: vp });
},

zoomTo: (zoom, centerX, centerY, customViewport) => {
  set((s) => {
    const vp = customViewport || get().canvasViewport;
    // A notebook stops zooming out once the sheet fits: past that point the
    // page only shrinks and the grey desk grows, which is the whole
    // complaint. A whiteboard is infinite and keeps the plain floor.
    const nbBoard = s.boards.find((b) => b.id === s.activeBoardId);
    const floor = isNotebook(nbBoard)
      ? Math.min(Math.max(MIN_ZOOM, minNotebookZoom(vp, pageSizeOf(nbBoard))), MAX_ZOOM)
      : MIN_ZOOM;
    const newZoom = clamp(zoom, floor, MAX_ZOOM);
    if (centerX !== undefined && centerY !== undefined) {
      const worldX = (centerX - vp.width / 2) / s.camera.zoom + s.camera.x;
      const worldY = (centerY - vp.height / 2) / s.camera.zoom + s.camera.y;
      const newCameraX = worldX - (centerX - vp.width / 2) / newZoom;
      const newCameraY = worldY - (centerY - vp.height / 2) / newZoom;
      return { camera: { x: newCameraX, y: newCameraY, zoom: newZoom } };
    }
    return { camera: { ...s.camera, zoom: newZoom } };
  });
},

zoomToFit: (customViewport) => {
  const board = get().boards.find((b) => b.id === get().activeBoardId);
  if (!board) return;
  // The stored canvas size, not the window: in split view the window is not
  // the canvas. Defaults to 1200x800 before the first measurement.
  const vp = customViewport || get().canvasViewport;
  if (isNotebook(board)) {
    // Fit the SHEET width, not the ink bounds: on a sparse page the ink box
    // is arbitrary (one dot fits to a dot), while the sheet is the stable
    // frame the user reads. The active page stays anchored at world y = 0,
    // so its centre is always half a sheet down.
    const dim = pageSizeOf(board);
    const MARGIN = 24;
    const fitZoom = clamp((vp.width - MARGIN * 2) / dim.width, MIN_ZOOM, 2.0);
    set({ camera: { x: dim.width / 2, y: dim.height / 2, zoom: fitZoom } });
    return;
  }
  const fitElements = getBoardElements(board);
  if (fitElements.length === 0) {
    get().resetCamera();
    return;
  }
  let minX = Infinity, minY = Infinity, maxX = -Infinity, maxY = -Infinity;
  for (const el of fitElements) {

```

```

    if (el.type === "connection") continue;
    const b = getElementBounds(el);
    if (b.x < minX) minX = b.x;
    if (b.y < minY) minY = b.y;
    if (b.x + b.width > maxX) maxX = b.x + b.width;
    if (b.y + b.height > maxY) maxY = b.y + b.height;
  }
  if (!isFinite(minX)) {
    get().resetCamera();
    return;
  }
  const padding = 120;
  const contentW = maxX - minX + padding * 2;
  const contentH = maxY - minY + padding * 2;
  const cx = (minX + maxX) / 2;
  const cy = (minY + maxY) / 2;
  const zoom = clamp(Math.min(vp.width / contentW, vp.height / contentH), MIN_ZOOM, 2.0);
  set({ camera: { x: cx, y: cy, zoom } });
},

resetCamera: () => set({ camera: { ...DEFAULT_CAMERA } }),

// ----- Tool / Style -----
setTool: (tool) => {
  // Remember which pen family was in hand, so "go back to the pen" restores
  // the pen you were actually using rather than a hardcoded default.
  set(isStrokeTool(tool) ? { tool, lastStrokeTool: tool } : { tool });
  // Locked-items mode belongs to the Select tool: any other tool ends it.
  if (tool !== "select" && tool !== "lasso" && get().lockedMode) set({ lockedMode: false });
  if (tool !== "select" && tool !== "lasso") {
    // Picking up a pen (or any drawing tool) while typing used to leave the
    // text editor open over the canvas, still grabbing the keyboard. It ends
    // the edit now. Select/Lasso deliberately do NOT: placing a text box
    // starts the edit and then switches to Select in the same breath, and
    // ending it there would throw away the box before you typed a letter.
    // The Text tool keeps it too - the next tap elsewhere starts a new box.
    if (tool === "text") set({ selectedIds: [] });
    else set({ selectedIds: [], editingId: null });
  }
},

setStyle: (patch) => {
  set((s) => ({ style: { ...s.style, ...patch } }));
  // saveNow is debounced, so dragging a size slider coalesces into one write.
  get().saveNow();
},

// ----- History -----
commitHistory: () => {
  const board = get().boards.find((b) => b.id === get().activeBoardId);
  if (!board) return;
  set((s) => ({
    history: {
      past: [
        // 100 undo steps, per the feature request (was 50).
        ...s.history.past.slice(-(HISTORY_PAST_MAX - 1)),
        {
          elements: cloneElementsForHistory(getBoardElements(board)),
          boardId: board.id,
          activeAt: board.id,
          pageId: getActivePage(board)?.id,
          at: Date.now(),
        },
      ],
      // A new edit ends the redo of things done from HERE only; the other
      // pane's redo is still valid and must survive.
      future: s.history.future.filter((e) => contextOf(e) !== board.id),
    },
  }));
  scheduleHistorySave(() => persistableHistory(get()));
},

undo: () => {
  const state = get();
  if (state.editingId) return; // never revert an in-progress inline edit
  const at = lastIndexForContext(state.history.past, state.activeBoardId);
  if (at < 0) return;
  const last = state.history.past[at];

```

```

const board = state.boards.find((b) => b.id === last.boardId);
if (!board) return;
const currentSnapshot: HistoryEntry =
  last.pages && isNotebook(board)
    ? {
        pages: clonePagesForHistory(board.pages ?? []),
        elements: [],
        boardId: board.id,
        activeAt: contextOf(last),
        pageId: board.activePageId,
        at: Date.now(),
      }
    : {
        elements: cloneElementsForHistory(getBoardElements(board)),
        boardId: board.id,
        activeAt: contextOf(last),
        pageId: getActivePage(board)?.id,
        at: Date.now(),
      };
set((s) => ({
  history: {
    past: s.history.past.filter((_, i) => i !== at),
    future: [...s.history.future, currentSnapshot].slice(-HISTORY_FUTURE_MAX),
  },
  boards: s.boards.map((b) =>
    b.id === board.id ? restoreHistoryElements(b, last) : b
  ),
  selectedIds: board.id === s.activeBoardId ? [] : s.selectedIds,
  editingId: board.id === s.activeBoardId ? null : s.editingId,
}));
get().saveNow();
scheduleHistorySave(() => persistableHistory(get()));
},

redo: () => {
  const state = get();
  if (state.editingId) return; // never swap content under the open editor
  const at = lastIndexForContext(state.history.future, state.activeBoardId);
  if (at < 0) return;
  const next = state.history.future[at];
  const board = state.boards.find((b) => b.id === next.boardId);
  if (!board) return;
  const currentSnapshot: HistoryEntry =
    next.pages && isNotebook(board)
      ? {
          pages: clonePagesForHistory(board.pages ?? []),
          elements: [],
          boardId: board.id,
          activeAt: contextOf(next),
          pageId: board.activePageId,
          at: Date.now(),
        }
      : {
          elements: cloneElementsForHistory(getBoardElements(board)),
          boardId: board.id,
          activeAt: contextOf(next),
          pageId: getActivePage(board)?.id,
          at: Date.now(),
        };
  set((s) => ({
    history: {
      past: [...s.history.past, currentSnapshot].slice(-HISTORY_PAST_MAX),
      future: s.history.future.filter((_, i) => i !== at),
    },
    boards: s.boards.map((b) =>
      b.id === board.id ? restoreHistoryElements(b, next) : b
    ),
    selectedIds: board.id === s.activeBoardId ? [] : s.selectedIds,
    editingId: board.id === s.activeBoardId ? null : s.editingId,
  }));
  get().saveNow();
  scheduleHistorySave(() => persistableHistory(get()));
},

canUndo: () => lastIndexForContext(get().history.past, get().activeBoardId) >= 0,
canRedo: () => lastIndexForContext(get().history.future, get().activeBoardId) >= 0,

toggleSidebar: () => set((s) => ({ sidebarOpen: !s.sidebarOpen })),

```

```

setSidebarOpen: (open) => set({ sidebarOpen: open }),

setSidebarSort: (by, order) => {
  set((s) => ({
    sidebarSortBy: by,
    sidebarSortOrder:
      order ??
      (s.sidebarSortBy === by
        ? s.sidebarSortOrder === "asc"
          ? "desc"
          : "asc"
        : by === "name"
          ? "asc"
          : "desc"),
  }));
},

batchMoveBoards: (boardIds, targetFolderId) => {
  if (boardIds.length === 0) return;
  const s = get();
  if (targetFolderId !== null && !s.folders.some((f) => f.id === targetFolderId)) return;
  const toMoveSet = new Set(boardIds);
  set((prev) => ({
    boards: prev.boards.map((b) => (toMoveSet.has(b.id) ? { ...b, folderId: targetFolderId } : b)),
  }));
  get().saveNow();
},

batchDeleteBoards: (boardIds) => {
  if (boardIds.length === 0) return;
  const toDeleteSet = new Set(boardIds);
  set((s) => {
    const remainingBoards = s.boards
      .filter((b) => !toDeleteSet.has(b.id))
      .map((b) =>
        mapAllBoardElements(b, (els) =>
          els.filter((el) => {
            if (el.type !== "connection") return true;
            const conn = el as ConnectionElement;
            return !toDeleteSet.has(conn.source.boardId) && !toDeleteSet.has(conn.target.boardId);
          })
        )
      );
    const activeWasDeleted = s.activeBoardId && toDeleteSet.has(s.activeBoardId);
    if (!activeWasDeleted) {
      return { boards: remainingBoards };
    }
    const nextActive = remainingBoards[0]?.id || null;
    const nextBoard = remainingBoards.find((b) => b.id === nextActive);
    return {
      boards: remainingBoards,
      activeBoardId: nextActive,
      camera: nextBoard?.camera || { ...DEFAULT_CAMERA },
      selectedIds: [],
      editingId: null,
      history: { past: [], future: [] },
    };
  });
  get().saveNow();
},

batchStarBoards: (boardIds, star) => {
  if (boardIds.length === 0) return;
  const toStarSet = new Set(boardIds);
  set((prev) => ({
    boards: prev.boards.map((b) => {
      if (!toStarSet.has(b.id)) return b;
      return { ...b, starred: star !== undefined ? star : !b.starred };
    }),
  }));
  get().saveNow();
},

lastDeleted: null,

batchDuplicateBoards: (boardIds) => {
  const ids: string[] = [];

```



```

    for (const id of boardIds) {
      const created = get().duplicateBoard(id);
      if (created) ids.push(created);
    }
    return ids;
  },

  trashItems: (boardIds, folderIds) => {
    const s = get();
    const byId = new Map(s.boards.map((b) => [b.id, b]));
    const byFolder = new Map(s.folders.map((f) => [f.id, f]));
    // Expand folder subtrees (same rule as deleteFolder: grandchildren die
    // with the folder, so they can never be orphaned).
    const doomed = new Set<string>();
    for (const fid of folderIds) {
      if (!byFolder.has(fid) || doomed.has(fid)) continue;
      doomed.add(fid);
      let changed = true;
      while (changed) {
        changed = false;
        for (const f of s.folders) {
          if (f.parentId && doomed.has(f.parentId) && !doomed.has(f.id)) {
            doomed.add(f.id);
            changed = true;
          }
        }
      }
    }
    const doomedFolders = s.folders.filter((f) => doomed.has(f.id));
    const trashBoards = boardIds.map((id) => byId.get(id)).filter((b) => b !== undefined);
    if (trashBoards.length === 0 && doomedFolders.length === 0) return;
    const trashSet = new Set(trashBoards.map((b) => b.id));
    const removedBoards = s.boards
      .map((b, index) => ({ board: b, index }))
      .filter(({ board }) => trashSet.has(board.id));
    // Boards a trashed folder filed to root remember their folder for undo.
    const movedToRoot: Array<{ id: string; folderId: string | null }> = [];
    const remaining = s.boards
      .filter((b) => !trashSet.has(b.id))
      .map((b) => {
        if (b.folderId && doomed.has(b.folderId)) {
          movedToRoot.push({ id: b.id, folderId: b.folderId });
          return { ...b, folderId: null as string | null };
        }
        return b;
      });
    const activeWasDeleted = s.activeBoardId !== null && trashSet.has(s.activeBoardId);
    const nextActive = activeWasDeleted ? remaining[0]?.id || null : s.activeBoardId;
    const nextBoard = remaining.find((b) => b.id === nextActive);
    set({
      boards: remaining,
      folders: s.folders.filter((f) => !doomed.has(f.id)),
      activeBoardId: nextActive,
      activeFolderId: s.activeFolderId && doomed.has(s.activeFolderId) ? null : s.activeFolderId,
      ...(activeWasDeleted
        ? {
            camera: nextBoard?.camera || { ...DEFAULT_CAMERA },
            selectedIds: [],
            editingId: null,
            history: { past: [], future: [] },
          }
        : {}),
      lastDeleted: {
        removedBoards,
        removedFolders: doomedFolders,
        movedToRoot,
        activeBoardId: s.activeBoardId,
        activeFolderId: s.activeFolderId,
        at: Date.now(),
      },
    });
    get().saveNow();
    // No immediate prune: the buffer's pictures/PDFs stay referenced until
    // undo expires (discardTrash) or is used (undoTrash).
  },

  undoTrash: () => {
    const s = get();

```

```

const buf = s.lastDeleted;
if (!buf) return;
// Reinsert removed boards at their original indices (clamped: the user
// may have created boards meanwhile, which must not move).
const returning = [...buf.removedBoards].sort((a, b) => a.index - b.index);
const boards = [...s.boards];
for (const { board, index } of returning) {
  if (boards.some(b => b.id === board.id)) continue;
  boards.splice(Math.min(index, boards.length), 0, board);
}
const folderIds = new Set(s.folders.map(f => f.id));
const folders = [...s.folders, ...buf.removedFolders.filter(f => !folderIds.has(f.id))];
// Restore filed-to-root boards only if they are still at root: a board
// the user re-filed meanwhile keeps its new home.
const movedBack = new Map(buf.movedToRoot.map(m => [m.id, m.folderId]));
const restored = boards.map(b => {
  const was = movedBack.get(b.id);
  if (was !== undefined && (b.folderId === null || b.folderId === undefined)) {
    return { ...b, folderId: was };
  }
  return b;
});
// Never hijack: only take back the active board/folder when what the
// user looks at now is gone.
const activeAlive = restored.some(b => b.id === s.activeBoardId);
const folderAlive = !s.activeFolderId || folders.some(f => f.id === s.activeFolderId);
set({
  boards: restored,
  folders,
  activeBoardId: !activeAlive && buf.activeBoardId ? buf.activeBoardId : s.activeBoardId,
  activeFolderId: !folderAlive ? buf.activeFolderId : s.activeFolderId,
  lastDeleted: null,
});
get().saveNow();
},

discardTrash: () => {
  if (!get().lastDeleted) return;
  set({ lastDeleted: null });
  // The buffer was the only thing keeping those files referenced.
  void get().pruneOrphanedAssets();
},

getPersistedState: () => {
  const s = get();
  return {
    version: 1,
    folders: s.folders,
    // Inject the live camera into the active board on every save so the
    // viewport position survives reloads (previously board.camera was frozen
    // at creation time and never updated).
    boards: s.boards.map(b =>
      b.id === s.activeBoardId ? { ...b, camera: { ...s.camera } } : b
    ),
    activeBoardId: s.activeBoardId,
    activeFolderId: s.activeFolderId,
    customLinkTypes: s.customLinkTypes,
    pinnedPens: s.pinnedPens,
    pinnedBarPos: s.pinnedBarPos,
    toolbarPosition: s.toolbarPosition,
    splitBoardId: s.splitBoardId,
    splitRatio: s.splitRatio,
    splitSide: s.splitSide,
    splitOrientation: s.splitOrientation,
    pageTemplates: s.pageTemplates,
    pinnedBarVertical: s.pinnedBarVertical,
    recentColors: s.recentColors,
    style: s.style,
    tool: s.tool,
    lastStrokeTool: s.lastStrokeTool,
    touchSettings: s.touchSettings,
    dictationLang: s.dictationLang,
    lassoStrict: s.lassoStrict,
    snapToGrid: s.snapToGrid,
    minimapOpen: s.minimapOpen,
    legendOpen: s.legendOpen,
    zoomDockOpen: s.zoomDockOpen,
    shapeRecognition: s.shapeRecognition,
  };
}

```

```

    propsPanelCollapsed: s.propsPanelCollapsed,
    toolPropsBarCollapsed: s.toolPropsBarCollapsed,
    toolbarTools: s.toolbarItems,
    toolbarHidden: ALL_TOOLBAR_ITEMS.filter((id) => !s.toolbarItems.includes(id)),
    defaultPaper: s.defaultPaper,
    pinnedBarOpen: s.pinnedBarOpen,
  };
},

saveNow: () => {
  scheduleSave(() => get().getPersistedState());
},

/** Everything worth backing up or syncing, with the live camera folded in. */
getWorkspaceSnapshot: () => {
  const s = get();
  return {
    folders: s.folders,
    boards: s.boards.map((b) =>
      b.id === s.activeBoardId ? { ...b, camera: { ...s.camera } } : b
    ),
    activeBoardId: s.activeBoardId,
    activeFolderId: s.activeFolderId,
    customLinkTypes: s.customLinkTypes,
    pinnedPens: s.pinnedPens,
    pinnedBarPos: s.pinnedBarPos,
    toolbarPosition: s.toolbarPosition,
    splitBoardId: s.splitBoardId,
    splitRatio: s.splitRatio,
    splitSide: s.splitSide,
    splitOrientation: s.splitOrientation,
    pageTemplates: s.pageTemplates,
    recentColors: s.recentColors,
    style: s.style,
    tool: s.tool,
    touchSettings: s.touchSettings,
    dictationLang: s.dictationLang,
    lassoStrict: s.lassoStrict,
    snapToGrid: s.snapToGrid,
    minimapOpen: s.minimapOpen,
    legendOpen: s.legendOpen,
    zoomDockOpen: s.zoomDockOpen,
    shapeRecognition: s.shapeRecognition,
    propsPanelCollapsed: s.propsPanelCollapsed,
    toolPropsBarCollapsed: s.toolPropsBarCollapsed,
    toolbarTools: s.toolbarItems,
    toolbarHidden: ALL_TOOLBAR_ITEMS.filter((id) => !s.toolbarItems.includes(id)),
    defaultPaper: s.defaultPaper,
    pinnedBarOpen: s.pinnedBarOpen,
  };
},

/**
 * Swap the entire workspace (restore from a backup). History is dropped on
 * purpose: undo stacks describe elements that no longer exist, and replaying
 * them over restored data would corrupt it.
 */
replaceWorkspace: (snapshot) => {
  const activeBoard =
    snapshot.boards.find((b) => b.id === snapshot.activeBoardId) || snapshot.boards[0];
  set({
    folders: snapshot.folders,
    boards: snapshot.boards,
    activeBoardId: activeBoard ? activeBoard.id : null,
    activeFolderId: snapshot.activeFolderId ?? null,
    customLinkTypes: snapshot.customLinkTypes ?? [],
    pinnedPens: snapshot.pinnedPens ?? [],
    pinnedBarPos: snapshot.pinnedBarPos ?? null,
    camera: activeBoard?.camera ? { ...activeBoard.camera } : { ...DEFAULT_CAMERA },
    selectedIds: [],
    editingId: null,
    history: { past: [], future: [] },
  });
  get().saveNow();
},

/** Fold a remote workspace in per board (see mergeWorkspaces for the rules). */
mergeWorkspace: (remote) => {

```

```

    const { merged, report } = mergeWorkspaces(get().getWorkspaceSnapshot(), remote);
    get().replaceWorkspace(merged);
    return report;
  },
});

export function useActiveBoard(): Board | null {
  return useWhiteboard((s) => {
    const board = s.boards.find((b) => b.id === s.activeBoardId);
    return board ?? null;
  });
}

export function useActiveElements(): WhiteboardElement[] {
  return useWhiteboard((s) => {
    const board = s.boards.find((b) => b.id === s.activeBoardId);
    return getBoardElements(board);
  });
}

// Editing ends in many places - Done, tapping away, switching tools, turning
// the page, switching board, deleting - and a dozen of them set `editingId`
// directly. Hooking the TRANSITION catches every one, where patching each
// caller would miss the next one someone adds. The discard's own `set` sees
// the previous editingId already null, so this cannot loop.
useWhiteboard.subscribe((state, prev) => {
  if (prev.editingId && prev.editingId !== state.editingId && prev.activeBoardId) {
    state.discardAbandonedText(prev.editingId, prev.activeBoardId);
  }
});

if (typeof window !== "undefined") {
  // Debug handle for the dev server only - a release build must not hand
  // the whole store to anything running in the page.
  if (process.env.NODE_ENV !== "production") {
    (window as unknown as Record<string, unknown>).__wbStore = useWhiteboard;
  }
  const flushAll = () => {
    // Best-effort by necessity: IndexedDB writes are async and the page may
    // die mid-write. The 500 ms debounced autosave bounds the exposure, and
    // visibilitychange/pagehide below fire while the page can still finish.
    flushSave(() => useWhiteboard.getState().getPersistedState());
    flushHistorySave(() => persistableHistory(useWhiteboard.getState()));
  };
  window.addEventListener("beforeunload", flushAll);
  if (typeof document !== "undefined" && typeof document.addEventListener === "function") {
    // Mobile apps are backgrounded far more often than unloaded - and
    // beforeunload is unreliable there. Flushing on hide catches the
    // app-switch-to-WhatsApp case while writes can still complete.
    document.addEventListener("visibilitychange", () => {
      if (document.visibilityState === "hidden") flushAll();
    });
  }
  window.addEventListener("pagehide", flushAll);

  // Multi-tab: the workspace save stamps a tiny localStorage heartbeat, so a
  // write from another tab still surfaces here even though the boards
  // themselves live in IndexedDB (which fires no cross-tab events).
  watchExternalSaves(() => {
    window.dispatchEvent(new CustomEvent("wb:external-change"));
  });
}

=====
// === END FILE: src/store/whiteboard.ts ===
=====

=====
// === START FILE: tailwind.config.ts ===
// File 134 of 194 | Lines: 64 | Size: 1696 bytes
=====

import type { Config } from "tailwindcss";
import tailwindcssAnimate from "tailwindcss-animate";

const config: Config = {
  darkMode: "class",

```

```
    content: [
      './pages/**/*.{js,ts,jsx,tsx,mdx}',
      './components/**/*.{js,ts,jsx,tsx,mdx}',
      './app/**/*.{js,ts,jsx,tsx,mdx}',
    ],
    theme: {
      extend: {
        colors: {
          background: 'hsl(var(--background))',
          foreground: 'hsl(var(--foreground))',
          card: {
            DEFAULT: 'hsl(var(--card))',
            foreground: 'hsl(var(--card-foreground))'
          },
          popover: {
            DEFAULT: 'hsl(var(--popover))',
            foreground: 'hsl(var(--popover-foreground))'
          },
          primary: {
            DEFAULT: 'hsl(var(--primary))',
            foreground: 'hsl(var(--primary-foreground))'
          },
          secondary: {
            DEFAULT: 'hsl(var(--secondary))',
            foreground: 'hsl(var(--secondary-foreground))'
          },
          muted: {
            DEFAULT: 'hsl(var(--muted))',
            foreground: 'hsl(var(--muted-foreground))'
          },
          accent: {
            DEFAULT: 'hsl(var(--accent))',
            foreground: 'hsl(var(--accent-foreground))'
          },
          destructive: {
            DEFAULT: 'hsl(var(--destructive))',
            foreground: 'hsl(var(--destructive-foreground))'
          },
          border: 'hsl(var(--border))',
          input: 'hsl(var(--input))',
          ring: 'hsl(var(--ring))',
          chart: {
            '1': 'hsl(var(--chart-1))',
            '2': 'hsl(var(--chart-2))',
            '3': 'hsl(var(--chart-3))',
            '4': 'hsl(var(--chart-4))',
            '5': 'hsl(var(--chart-5))'
          }
        },
        borderRadius: {
          lg: 'var(--radius)',
          md: 'calc(var(--radius) - 2px)',
          sm: 'calc(var(--radius) - 4px)'
        }
      }
    },
    plugins: [tailwindcssAnimate],
  };
export default config;
```

```
=====
// === END FILE: tailwind.config.ts ===
=====
```

```
=====
// === START FILE: tests/adversarial_challenger1.test.ts ===
// File 135 of 194 | Lines: 835 | Size: 27885 bytes
=====
```

```
import assert from "node:assert/strict";
import {
  isPointInPolygon,
  isElementInPolygon,
  getElementBounds,
  getElementCenter,
  getUnionBounds,
  getNodeAnchorPorts,
```

```

    getResolvedAnchorPoint,
    getOptimalConnectionAnchors,
    getConnectionPath,
    getArrowheadPath,
    worldToScreen,
    screenToWorld,
    rotatePoint,
    clamp,
    lerp,
    alignElements,
    distributeElements,
    isVisibleInViewport,
    getResizeHandles,
    hitTestElement,
    HIT_THRESHOLD,
  } from "../src/lib/whiteboard/geometry";
import { GRID_SIZE, snapToGrid, snapPointToGrid } from "../src/lib/whiteboard/grid";
import { exportBoardToSvg } from "../src/lib/whiteboard/export";
import type {
  Board,
  WhiteboardElement,
  StrokeElement,
  RectangleElement,
  EllipseElement,
  DiamondElement,
  TriangleElement,
  StarElement,
  CloudElement,
  LineElement,
  ArrowElement,
  ConnectionElement,
  TextElement,
  StickyElement,
  ImageElement,
} from "../src/lib/whiteboard/types";

console.log("=====");
console.log("=== EMPIRICAL CHALLENGER 1: COMPREHENSIVE ADVERSARIAL STRESS SUITE ===");
console.log("=====");

let totalAssertions = 0;
function pass(msg: string) {
  totalAssertions++;
  console.log(` ✓ ${msg}`);
}

// =====
// AREA 1: MATHEMATICAL PRECISION OF isPointInPolygon AND isElementInPolygon
// =====
console.log("\n--- [Area 1] Precision of isPointInPolygon & isElementInPolygon ---");

// 1.1 Concave Geometries (U-shape / Horseshoe with inner bay)
const uShapePolygon = [
  { x: 0, y: 0 },
  { x: 100, y: 0 },
  { x: 100, y: 100 },
  { x: 70, y: 100 },
  { x: 70, y: 30 },
  { x: 30, y: 30 },
  { x: 30, y: 100 },
  { x: 0, y: 100 },
];
// Points inside the solid arms
assert.equal(isPointInPolygon(15, 50, uShapePolygon), true, "Left vertical arm of U");
assert.equal(isPointInPolygon(85, 50, uShapePolygon), true, "Right vertical arm of U");
assert.equal(isPointInPolygon(50, 15, uShapePolygon), true, "Top horizontal bridge of U");

// Points in the hollow bay / cavity (within bounding box [0,0,100,100] but OUTSIDE polygon)
assert.equal(isPointInPolygon(50, 60, uShapePolygon), false, "Cavity inside U must be outside polygon");
assert.equal(isPointInPolygon(50, 90, uShapePolygon), false, "Deep cavity opening of U must be outside");
assert.equal(isPointInPolygon(-10, 50, uShapePolygon), false, "Far exterior point");
pass("Concave U-shape / cavity point containment verified");

// 1.2 Star Non-Convex Polygon (10 vertices)
const star5Polygon = [
  { x: 50, y: 0 },
  { x: 62, y: 35 },
  { x: 100, y: 35 },

```

```

    { x: 69, y: 57 },
    { x: 81, y: 95 },
    { x: 50, y: 72 },
    { x: 19, y: 95 },
    { x: 31, y: 57 },
    { x: 0, y: 35 },
    { x: 38, y: 35 },
  ];
  // Star center is inside
  assert.equal(isPointInPolygon(50, 50, star5Polygon), true, "Star center is inside");
  // Star tips are inside
  assert.equal(isPointInPolygon(50, 5, star5Polygon), true, "Near top tip is inside");
  // Outer notches between tips (inside bounding box [0,0,100,95] but outside star body)
  assert.equal(isPointInPolygon(10, 10, star5Polygon), false, "Top-left notch outside star");
  assert.equal(isPointInPolygon(90, 10, star5Polygon), false, "Top-right notch outside star");
  assert.equal(isPointInPolygon(50, 90, star5Polygon), false, "Bottom notch outside star");
  pass("10-vertex non-convex Star polygon point containment verified");

  // 1.3 Self-Intersecting / Bow-tie / Figure-8 Polygon (Even-Odd Rule)
  const bowTiePolygon = [
    { x: 0, y: 0 },
    { x: 100, y: 100 },
    { x: 100, y: 0 },
    { x: 0, y: 100 },
  ];
  // Left triangle (0,0)-(50,50)-(0,100): interior point (20, 50) -> IN
  assert.equal(isPointInPolygon(20, 50, bowTiePolygon), true, "Left lobe of bow-tie polygon is inside");
  // Right triangle (100,0)-(50,50)-(100,100): interior point (80, 50) -> IN
  assert.equal(isPointInPolygon(80, 50, bowTiePolygon), true, "Right lobe of bow-tie polygon is inside");
  // Top and bottom triangular cutouts (50, 10) and (50, 90) -> OUT (ray intersects 2 edges, even -> outside)
  assert.equal(isPointInPolygon(50, 10, bowTiePolygon), false, "Top waist of bow-tie is outside (even-odd rule)");
  assert.equal(isPointInPolygon(50, 90, bowTiePolygon), false, "Bottom waist of bow-tie is outside (even-odd rule)");
  pass("Bow-tie / self-intersecting figure-8 polygon verified under Even-Odd rule");

  // 1.4 Colinear Vertices and Horizontal/Vertical Segments
  const colinearPolygon = [
    { x: 0, y: 0 },
    { x: 50, y: 0 }, // colinear with (0,0) and (100,0)
    { x: 100, y: 0 },
    { x: 100, y: 50 }, // colinear with (100,0) and (100,100)
    { x: 100, y: 100 },
    { x: 0, y: 100 },
  ];
  assert.equal(isPointInPolygon(50, 50, colinearPolygon), true, "Center of colinear polygon is inside");
  assert.equal(isPointInPolygon(50, -5, colinearPolygon), false, "Point just above colinear top edge is outside");
  assert.equal(isPointInPolygon(105, 50, colinearPolygon), false, "Point just right of colinear right edge is outside");
  pass("Colinear segment polygon verified");

  // 1.5 Degenerate Polygon Inputs
  assert.equal(isPointInPolygon(50, 50, []), false, "Empty polygon returns false");
  assert.equal(isPointInPolygon(50, 50, [{ x: 10, y: 10 }]), false, "1-point polygon returns false");
  assert.equal(isPointInPolygon(50, 50, [{ x: 10, y: 10 }, { x: 20, y: 20 }]), false, "2-point polygon returns false");
  pass("Degenerate polygon edge cases return false safely without exception");

  // 1.6 Massive 1,000-vertex polygon performance and precision test
  const circlePointsCount = 1000;
  const circlePoly: Array<{ x: number; y: number }> = [];
  for (let i = 0; i < circlePointsCount; i++) {
    const theta = (i / circlePointsCount) * 2 * Math.PI;
    circlePoly.push({ x: 500 + 300 * Math.cos(theta), y: 500 + 300 * Math.sin(theta) });
  }
  const startTimer = performance.now();
  for (let iter = 0; iter < 100; iter++) {
    assert.equal(isPointInPolygon(500, 500, circlePoly), true, "Center inside 1000-gon");
    assert.equal(isPointInPolygon(500, 790, circlePoly), true, "Near perimeter inside 1000-gon");
    assert.equal(isPointInPolygon(500, 810, circlePoly), false, "Outside 1000-gon perimeter");
    assert.equal(isPointInPolygon(0, 0, circlePoly), false, "Far outside 1000-gon");
  }
  const elapsedMs = performance.now() - startTimer;
  assert.ok(elapsedMs < 100, `1000-gon 100 iterations took ${elapsedMs.toFixed(2)}ms (should be < 100ms)`);
  pass(`1,000-vertex high-density polygon precision and performance verified (${elapsedMs.toFixed(2)}ms for 400 tests)`);

```

```
// 1.7 Element Containment across all element types with isElementInPolygon
const lassoRectPoly = [
  { x: 100, y: 100 },
  { x: 300, y: 100 },
  { x: 300, y: 300 },
  { x: 100, y: 300 },
];

const testElements: WhiteboardElement[] = [
  {
    id: "el-stroke",
    type: "stroke",
    x: 120,
    y: 120,
    rotation: 0,
    opacity: 1,
    points: [[0, 0, 0.5], [10, 10, 0.5]],
    color: "#000",
    strokeWidth: 2,
    strokeStyle: "pen",
    createdAt: 1,
    updatedAt: 1,
  },
  {
    id: "el-rect",
    type: "rectangle",
    x: 150,
    y: 150,
    width: 40,
    height: 40,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    fillColor: "#fff",
    fillStyle: "solid",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  },
  {
    id: "el-ellipse",
    type: "ellipse",
    x: 200,
    y: 150,
    width: 40,
    height: 40,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    fillColor: "transparent",
    fillStyle: "transparent",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  },
  {
    id: "el-diamond",
    type: "diamond",
    x: 150,
    y: 200,
    width: 40,
    height: 40,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    fillColor: "transparent",
    fillStyle: "transparent",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  },
  {
    id: "el-line",
    type: "line",
    x: 120,
```



```
y: 220,  
x2: 60,  
y2: 40,  
rotation: 0,  
opacity: 1,  
strokeColor: "#000",  
dashStyle: "solid",  
strokeWidth: 2,  
createdAt: 1,  
updatedAt: 1,  
},  
{  
  id: "el-arrow",  
  type: "arrow",  
  x: 180,  
  y: 220,  
  x2: 50,  
  y2: -30,  
  startArrowhead: "none",  
  endArrowhead: "arrow",  
  arrowheadSize: 12,  
  rotation: 0,  
  opacity: 1,  
  strokeColor: "#000",  
  dashStyle: "solid",  
  strokeWidth: 2,  
  createdAt: 1,  
  updatedAt: 1,  
},  
{  
  id: "el-text",  
  type: "text",  
  x: 150,  
  y: 260,  
  width: 100,  
  height: 30,  
  rotation: 0,  
  opacity: 1,  
  text: "Hello",  
  color: "#000",  
  fontSize: 16,  
  fontFamily: "Inter",  
  fontWeight: 400,  
  align: "left",  
  createdAt: 1,  
  updatedAt: 1,  
},  
{  
  id: "el-sticky",  
  type: "sticky",  
  x: 140,  
  y: 130,  
  width: 60,  
  height: 60,  
  rotation: 0,  
  opacity: 1,  
  text: "Note",  
  color: "yellow",  
  fontSize: 14,  
  align: "center",  
  createdAt: 1,  
  updatedAt: 1,  
},  
{  
  id: "el-image",  
  type: "image",  
  x: 160,  
  y: 160,  
  width: 50,  
  height: 50,  
  naturalWidth: 100,  
  naturalHeight: 100,  
  rotation: 0,  
  opacity: 1,  
  src: "",  
  createdAt: 1,  
  updatedAt: 1,  
},
```

```

];

for (const el of testElements) {
  assert.equal(isElementInPolygon(el, lassoRectPoly), true, `${el.type} inside polygon must be detected`);
  const shiftedOutside: WhiteboardElement = { ...el, x: el.x + 500, y: el.y + 500 };
  assert.equal(isElementInPolygon(shiftedOutside, lassoRectPoly), false, `${el.type} shifted outside polygon must return false`);
}
pass("All element types correctly detected by isElementInPolygon (inside vs outside)");

// 1.8 Element in the cavity of a concave polygon
// A small rectangle located at (45, 55) is inside the bounding box of uShapePolygon, but in the hollow space!
const rectInCavity: RectangleElement = {
  id: "rect-cavity",
  type: "rectangle",
  x: 45,
  y: 55,
  width: 10,
  height: 10,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "#fff",
  fillStyle: "solid",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: 1,
  updatedAt: 1,
};
assert.equal(isElementInPolygon(rectInCavity, uShapePolygon), false, "Element strictly inside concave cavity must NOT be selected");
pass("Concave cavity containment precision: elements in open bay are NOT erroneously selected");

// =====
// AREA 2: LINE & ARROW RESIZING VECTOR MATH ACROSS EXTREME ASPECT RATIOS & NEGATIVE COORDINATES
// =====
console.log("\n--- [Area 2] Line & Arrow Resizing Vector Math ---");

// Helper function to simulate handle resizing on a Line/Arrow according to canvas resize logic
function simulateLineResize(
  el: LineElement | ArrowElement,
  handle: string,
  dx: number,
  dy: number
): { x: number; y: number; x2: number; y2: number } {
  const b = getElementBounds(el);
  const p1 = { x: el.x, y: el.y };
  const p2 = { x: el.x + el.x2, y: el.y + el.y2 };
  const oldW = Math.max(b.width, 1);
  const oldH = Math.max(b.height, 1);

  let newX = b.x;
  let newY = b.y;
  let newW = b.width;
  let newH = b.height;

  if (handle.includes("e")) newW = Math.max(b.width + dx, 10);
  if (handle.includes("s")) newH = Math.max(b.height + dy, 10);
  if (handle.includes("w")) {
    const potentialW = b.width - dx;
    if (potentialW >= 10) {
      newX = b.x + dx;
      newW = potentialW;
    }
  }
  if (handle.includes("n")) {
    const potentialH = b.height - dy;
    if (potentialH >= 10) {
      newY = b.y + dy;
      newH = potentialH;
    }
  }
}

const u1 = (p1.x - b.x) / oldW;
const v1 = (p1.y - b.y) / oldH;
const u2 = (p2.x - b.x) / oldW;
const v2 = (p2.y - b.y) / oldH;

```

```

const newP1x = newX + u1 * newW;
const newP1y = newY + v1 * newH;
const newP2x = newX + u2 * newW;
const newP2y = newY + v2 * newH;

return {
  x: newP1x,
  y: newP1y,
  x2: newP2x - newP1x,
  y2: newP2y - newP1y,
};
}

// 2.1 Arrow with negative direction (pointing up-left: x2 < 0, y2 < 0) at negative origin
const negArrow: ArrowElement = {
  id: "neg-arrow",
  type: "arrow",
  x: -100,
  y: -100,
  x2: -200,
  y2: -150,
  startArrowhead: "none",
  endArrowhead: "arrow",
  arrowheadSize: 12,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: 1,
  updatedAt: 1,
};
const negBounds = getElementBounds(negArrow);
// minX = min(-100, -300) = -300, minY = min(-100, -250) = -250, maxX = -100, maxY = -100
assert.equal(negBounds.x, -300);
assert.equal(negBounds.y, -250);
assert.equal(negBounds.width, 200);
assert.equal(negBounds.height, 150);

// Resize handle 'w' on negative arrow: dragging dx = -50 (expanding west)
const resW = simulateLineResize(negArrow, "w", -50, 0);
assert.ok(isFinite(resW.x) && isFinite(resW.y) && isFinite(resW.x2) && isFinite(resW.y2), "No NaN or Infinity"
);
// Origin p1 was at (-100, -100) (the east side of bounding box), endpoint p2 at (-300, -250) (west side)
// When west expands by 50, p1 should stay at x = -100, p2 should move to x = -350 -> x2 should become -250!
assert.equal(Math.round(resW.x), -100, "Fixed end stays at -100");
assert.equal(Math.round(resW.x + resW.x2), -350, "Moving end moves to -350");
assert.equal(Math.round(resW.x2), -250, "x2 vector expanded cleanly in negative direction");
pass("Negative coordinate Arrow with negative delta (x2 < 0, y2 < 0) resized without distortion");

// 2.2 Extreme Aspect Ratio: Perfectly horizontal line (y2 = 0)
const horizLine: LineElement = {
  id: "horiz-line",
  type: "line",
  x: 50,
  y: 50,
  x2: 300,
  y2: 0,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: 1,
  updatedAt: 1,
};
const resHorizE = simulateLineResize(horizLine, "e", 100, 0);
assert.equal(resHorizE.x, 50);
assert.equal(resHorizE.y, 50);
assert.equal(resHorizE.x2, 400);
assert.equal(resHorizE.y2, 0);
assert.ok(!Number.isNaN(resHorizE.x2) && !Number.isNaN(resHorizE.y2), "No NaN on flat horizontal line");
pass("Perfect horizontal line (y2 = 0) resized cleanly via handle 'e'");

// 2.3 Extreme Aspect Ratio: Perfectly vertical line (x2 = 0)
const vertLine: LineElement = {
  id: "vert-line",

```

```

    type: "line",
    x: 50,
    y: 50,
    x2: 0,
    y2: 300,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  };
  const resVertS = simulateLineResize(vértLine, "s", 0, 100);
  assert.equal(resVertS.x, 50);
  assert.equal(resVertS.y, 50);
  assert.equal(resVertS.x2, 0);
  assert.equal(resVertS.y2, 400);
  assert.ok(!Number.isNaN(resVertS.x2) && !Number.isNaN(resVertS.y2), "No NaN on vertical line");
  pass("Perfect vertical line (x2 = 0) resized cleanly via handle 's'");

  // 2.4 Zero-length degenerate line (x2 = 0, y2 = 0)
  const zeroLine: LineElement = {
    id: "zero-line",
    type: "line",
    x: 100,
    y: 100,
    x2: 0,
    y2: 0,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  };
  const resZeroSE = simulateLineResize(zeroLine, "se", 50, 50);
  assert.ok(isFinite(resZeroSE.x) && isFinite(resZeroSE.y) && isFinite(resZeroSE.x2) && isFinite(resZeroSE.y2));
  pass("Zero-length line resize does not throw or produce NaN/Infinity");

  // =====
  // AREA 3: ROTATED ELEMENT RESIZE TRANSFORMATIONS WITH ARBITRARY ANGLES
  // =====
  console.log("\n--- [Area 3] Rotated Element Resize Transformations ---");

  // Helper function to simulate rotated rectangle resize
  function simulateRotatedResize(
    el: RectangleElement,
    handle: string,
    worldDx: number,
    worldDy: number
  ): { x: number; y: number; width: number; height: number } {
    const rot = el.rotation || 0;
    const cos = Math.cos(rot);
    const sin = Math.sin(rot);
    const localDx = worldDx * cos + worldDy * sin;
    const localDy = -worldDx * sin + worldDy * cos;

    const origW = el.width || 100;
    const origH = el.height || 100;
    const origCx = el.x + origW / 2;
    const origCy = el.y + origH / 2;

    let newW = origW;
    let newH = origH;
    let localCenterShiftX = 0;
    let localCenterShiftY = 0;

    if (handle.includes("e")) {
      newW = Math.max(origW + localDx, 20);
      localCenterShiftX += (newW - origW) / 2;
    }
    if (handle.includes("w")) {
      newW = Math.max(origW - localDx, 20);
      localCenterShiftX -= (newW - origW) / 2;
    }
  }

```

```

    if (handle.includes("s")) {
      newH = Math.max(origH + localDy, 20);
      localCenterShiftY += (newH - origH) / 2;
    }
    if (handle.includes("n")) {
      newH = Math.max(origH - localDy, 20);
      localCenterShiftY -= (newH - origH) / 2;
    }

    const worldCenterShiftX = localCenterShiftX * cos - localCenterShiftY * sin;
    const worldCenterShiftY = localCenterShiftX * sin + localCenterShiftY * cos;

    const newCx = origCx + worldCenterShiftX;
    const newCy = origCy + worldCenterShiftY;

    const newX = newCx - newW / 2;
    const newY = newCy - newH / 2;

    return { x: newX, y: newY, width: newW, height: newH };
  }

// Test arbitrary rotation angles: 30°, 45°, 90°, 180°, 270°
const testAngles = [
  { name: "30°", rad: Math.PI / 6 },
  { name: "45°", rad: Math.PI / 4 },
  { name: "90°", rad: Math.PI / 2 },
  { name: "180°", rad: Math.PI },
  { name: "270°", rad: (3 * Math.PI) / 2 },
];

for (const { name, rad } of testAngles) {
  const baseRect: RectangleElement = {
    id: `rot-${name}`,
    type: "rectangle",
    x: 200,
    y: 200,
    width: 100,
    height: 60,
    rotation: rad,
    opacity: 1,
    strokeColor: "#000",
    fillColor: "#fff",
    fillStyle: "solid",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  };

  const origCx = baseRect.x + baseRect.width / 2;
  const origCy = baseRect.y + baseRect.height / 2;

  // Invariant: The un-dragged edge must remain fixed in world space!
  // Let's drag handle 'e' along local X direction by +40px:
  // In world coordinates, dragging along local X corresponds to:
  const dragAmount = 40;
  const worldDx = dragAmount * Math.cos(rad);
  const worldDy = dragAmount * Math.sin(rad);

  const resized = simulateRotatedResize(baseRect, "e", worldDx, worldDy);
  assert.equal(Math.round(resized.width), 140, `${name}: width expanded from 100 to 140`);
  assert.equal(Math.round(resized.height), 60, `${name}: height remained 60`);

  // Verify the west edge center in world coordinates did NOT move:
  // West edge unrotated local point: (origCx - origW/2, origCy) = (200, 230)
  const origWestPointWorld = rotatePoint(200, 230, origCx, origCy, rad);

  const newCx = resized.x + resized.width / 2;
  const newCy = resized.y + resized.height / 2;
  const newWestPointLocal = { x: newCx - resized.width / 2, y: newCy };
  const newWestPointWorld = rotatePoint(newWestPointLocal.x, newWestPointLocal.y, newCx, newCy, rad);

  const distShift = Math.hypot(origWestPointWorld.x - newWestPointWorld.x, origWestPointWorld.y - newWestPointWorld.y);
  assert.ok(distShift < 0.01, `${name}: Opposite West edge remained pinned in world space (shift = ${distShift.toFixed(4)})`);
  pass(`Rotated resize at angle ${name} preserved stationary opposite edge and local geometry`);
}

```

```
// 3.2 Anchor Ports under 90° and 180° rotation
const rect90: RectangleElement = {
  id: "rect-90",
  type: "rectangle",
  x: 100,
  y: 100,
  width: 100,
  height: 50,
  rotation: Math.PI / 2, // 90 deg
  opacity: 1,
  strokeColor: "#000",
  fillColor: "#fff",
  fillStyle: "solid",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: 1,
  updatedAt: 1,
};

const ports90 = getNodeAnchorPorts(rect90);
// Center is at (150, 125)
// Unrotated Top port was at (150, 100). Rotated 90 deg clockwise around (150, 125):
// dx = 0, dy = -25 -> rx = 150 - (-25)*1 = 175, ry = 125 + 0 = 125 (points to world RIGHT!)
assert.equal(Math.round(ports90.top.x), 175);
assert.equal(Math.round(ports90.top.y), 125);
// Normal for top port rotated 90 deg: nx = sin(pi/2) = 1, ny = -cos(pi/2) = 0
assert.equal(Math.round(ports90.top.nx), 1);
assert.ok(Math.abs(ports90.top.ny) < 1e-6, "Top port ny normal is ~ 0");
pass("Anchor ports for 90° rotated node correctly rotate coordinates and outward normals");

// =====
// AREA 4: EXPORT SVG SERIALIZATION AGAINST MALFORMED CHARACTERS, XSS, & DEEP NESTING
// =====
console.log("\n--- [Area 4] Export SVG Serialization & Security Stress ---");

const adversarialBoard: Board = {
  id: "adv-board-1",
  folderId: "folder-1",
  name: `Adversarial Board <script>alert('XSS')</script> "test" & 'quotes'`,
  camera: { x: 0, y: 0, zoom: 1 },
  elements: [
    // 4.1 Malicious text with script injection, CDATA break, and XML special chars
    {
      id: "txt-xss",
      type: "text",
      x: 100,
      y: 100,
      width: 200,
      height: 50,
      rotation: 0,
      opacity: 1,
      text: `<script>alert("PWNER")</script>\n&amp; &lt; &gt; "quote" 'single'\n]]> <![CDATA[ evil() ]]>`,
      color: `"><svg onload=alert(1)" fill=`,
      fontSize: 16,
      fontFamily: `" onmouseover=alert('font')`,
      fontWeight: 400,
      align: "left",
      createdAt: 1,
      updatedAt: 1,
    } as TextElement,
    // 4.2 Malicious sticky note
    {
      id: "sticky-xss",
      type: "sticky",
      x: 350,
      y: 100,
      width: 150,
      height: 150,
      rotation: 0,
      opacity: 1,
      text: `Hello <b>World</b>\n\nUnicode: █ ██████ ██████████ ████████ ██████████`,
      color: "yellow",
      fontSize: 14,
      align: "center",
      createdAt: 1,
      updatedAt: 1,
    },
  ],
}
```

```

// 4.3 Malicious connection label & custom hex
{
  id: "conn-xss",
  type: "connection",
  x: 0,
  y: 0,
  rotation: 0,
  opacity: 1,
  source: { boardId: "adv-board-1", elementId: "txt-xss", anchor: "right" },
  target: { boardId: "adv-board-1", elementId: "sticky-xss", anchor: "left" },
  routing: "curved",
  linkType: "relates-to",
  label: `<svg/onload=alert('conn')> & "Test"`,
  dashStyle: "solid",
  startArrowhead: "none",
  endArrowhead: "arrow",
  strokeWidth: 2,
  createdAt: 1,
  updatedAt: 1,
},
// 4.4 Malicious image URL
{
  id: "img-xss",
  type: "image",
  x: 100,
  y: 300,
  width: 100,
  height: 100,
  naturalWidth: 100,
  naturalHeight: 100,
  rotation: 0,
  opacity: 1,
  src: `javascript:alert('img') "><script>alert(3)</script>`,
  createdAt: 1,
  updatedAt: 1,
},
],
createdAt: 1,
updatedAt: 1,
};

const exportedSvg = exportBoardToSvg(adversarialBoard, { background: "white" });

// Verification Assertions for SVG Security and Well-Formedness:
// 1. Must NOT contain raw unescaped `<script>` or `<svg/onload` inside text/attributes
assert.ok(!exportedSvg.includes("<script>alert("), "Raw <script> tag must be escaped to &lt;script&gt;");
assert.ok(!exportedSvg.includes("<svg/onload"), "Raw <svg/onload> must be escaped");
assert.ok(!exportedSvg.includes(" tag must be escaped");
assert.ok(exportedSvg.includes("&lt;script&gt;alert(&quot;PWNER&quot;)&lt;/script&gt;"), "Script tag properly
  XML escaped in text note");
assert.ok(exportedSvg.includes("&amp; &lt; &gt;"), "Ampersands properly escaped");
assert.ok(exportedSvg.includes("■■■■■ ■■■■■"), "Hebrew unicode string preserved");
assert.ok(exportedSvg.includes("■■■■■ ■■■■■"), "Arabic unicode string preserved");
assert.ok(exportedSvg.includes("■"), "Emoji preserved");
assert.ok(exportedSvg.startsWith("<?xml version='1.0' encoding='UTF-8'>"), "Starts with XML header");
assert.ok(exportedSvg.trim().endsWith("</svg>"), "Ends with closing svg tag");
pass("Export SVG security: All XSS vectors and injection payloads escaped; Unicode preserved");

// 4.5 Massive Board Export Stress (1,000 elements)
const massiveElements: WhiteboardElement[] = [];
for (let i = 0; i < 1000; i++) {
  massiveElements.push({
    id: `node-${i}`,
    type: "rectangle",
    x: (i % 50) * 80,
    y: Math.floor(i / 50) * 80,
    width: 60,
    height: 40,
    rotation: (i % 4) * (Math.PI / 4),
    opacity: 0.9,
    strokeColor: "#3b82f6",
    fillColor: "#eff6ff",
    fillStyle: "semi",
    dashStyle: i % 2 === 0 ? "solid" : "dashed",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  });
};

```

```

}
const massiveBoard: Board = {
  id: "massive-board",
  folderId: "folder-1",
  name: "1000-Node Board",
  camera: { x: 0, y: 0, zoom: 1 },
  elements: massiveElements,
  createdAt: 1,
  updatedAt: 1,
};

const massiveStart = performance.now();
const massiveSvg = exportBoardToSvg(massiveBoard);
const massiveElapsed = performance.now() - massiveStart;
assert.ok(massiveSvg.length > 50000, "1000 elements generated complete SVG markup");
assert.ok(massiveElapsed < 100, `1000 element SVG export took ${massiveElapsed.toFixed(2)}ms (< 100ms)`);
pass(`Massive board export stress (1,000 elements) serialized in ${massiveElapsed.toFixed(2)}ms`);

// =====
// AREA 5: SNAP-TO-GRID QUANTIZATION BOUNDARIES
// =====
console.log("\n--- [Area 5] Snap-to-Grid Quantization Boundaries ---");

// Standard GRID_SIZE is 24
assert.equal(snapToGrid(0), 0, "snap(0) = 0");
assert.equal(snapToGrid(11.999), 0, "snap(11.999) = 0");
assert.equal(snapToGrid(12.0), 24, "snap(12.0) = 24 (midpoint rounds up)");
assert.equal(snapToGrid(12.001), 24, "snap(12.001) = 24");
assert.equal(snapToGrid(23.999), 24, "snap(23.999) = 24");
assert.equal(snapToGrid(24.0), 24, "snap(24.0) = 24");
assert.equal(snapToGrid(24.001), 24, "snap(24.001) = 24");
assert.equal(snapToGrid(35.999), 24, "snap(35.999) = 24");
assert.equal(snapToGrid(36.0), 48, "snap(36.0) = 48 (next midpoint rounds up)");

// Negative numbers & IEEE -0 normalization
assert.equal(Object.is(snapToGrid(-0.0001), 0), true, "snap(-0.0001) returns +0, not -0");
assert.equal(snapToGrid(-11.999), 0, "snap(-11.999) = 0");
assert.equal(snapToGrid(-12.0), 0, "snap(-12.0) rounds to 0 (Math.round(-0.5) is -0, normalized to +0)");
assert.equal(snapToGrid(-12.001), -24, "snap(-12.001) = -24");
assert.equal(snapToGrid(-24.0), -24, "snap(-24.0) = -24");
assert.equal(snapToGrid(-35.999), -24, "snap(-35.999) = -24");
assert.equal(snapToGrid(-36.0), -24, "snap(-36.0) = -24 (-1.5 rounds to -1 -> -24)");
assert.equal(snapToGrid(-36.001), -48, "snap(-36.001) = -48");

// Non-standard grid sizes
assert.equal(snapToGrid(15, 10), 20, "snap(15, gridSize=10) = 20");
assert.equal(snapToGrid(14, 10), 10, "snap(14, gridSize=10) = 10");
assert.equal(snapToGrid(47, 0), 47, "snap(47, gridSize=0) safely returns original val without division by zero");
assert.equal(snapToGrid(47, -10), 47, "snap(47, gridSize=-10) safely returns original val");

// 2D Point snapping
const pt1 = snapPointToGrid(12.5, -36.5);
assert.deepEqual(pt1, { x: 24, y: -48 });
pass("Snap-to-grid quantization boundaries, midpoints, negative float normalization and custom sizes verified");

console.log("\n=====");
console.log(`=== ALL CHALLENGER 1 ADVERSARIAL TESTS PASSED SUCCESSFULLY! (${totalAssertions} assertions) ===`);
console.log("=====");

// === END FILE: tests/adversarial_challenger1.test.ts ===

// =====
// === START FILE: tests/adversarial_zoom_gestures.test.ts ===
// File 136 of 194 | Lines: 493 | Size: 16375 bytes
// =====

import assert from "node:assert/strict";
import {
  screenToWorld,
  worldToScreen,
  worldToScreenDelta,

```



```

    screenToWorldDelta,
    clamp,
    getElementBounds,
    hitTestElement,
    type Viewport,
  } from "../src/lib/whiteboard/geometry";
import {
  MIN_ZOOM,
  MAX_ZOOM,
  type Camera,
  type WhiteboardElement,
  type StrokeElement,
  type RectangleElement,
  type TextElement,
} from "../src/lib/whiteboard/types";

console.log("=====");
console.log("CHALLENGER STRESS TEST: Camera Zoom Geometry, Viewport Math & Touch Gestures");
console.log("=====\\n");

let passedCount = 0;
let failedCount = 0;

function runTest(name: string, fn: () => void) {
  try {
    fn();
    console.log(` ✓ PASS: ${name}`);
    passedCount++;
  } catch (err: any) {
    console.error(` ✗ FAIL: ${name}`);
    console.error(`   ${err.message}`);
    failedCount++;
  }
}

// =====
// SUITE 1: Coordinate Roundtrip & Invertibility across Extreme Conditions
// =====
console.log("--- Suite 1: Screen <-> World Coordinate Roundtrips ---");

const testViewports: Viewport[] = [
  { width: 1280, height: 800 }, // Standard laptop
  { width: 1920, height: 1080 }, // 1080p Desktop
  { width: 800, height: 600 }, // Small window
  { width: 3840, height: 2160 }, // 4K Ultra HD
  { width: 375, height: 667 }, // iPhone SE mobile
  { width: 3440, height: 1440 }, // Ultrawide
  { width: 1024, height: 768 }, // iPad
  { width: 1, height: 1 }, // 1px micro-viewport
];

const testZooms = [
  MIN_ZOOM, // 0.05
  0.1,
  0.25,
  0.5,
  1.0,
  1.5,
  2.0,
  3.14159,
  4.0,
  MAX_ZOOM, // 8.0
];

const testPoints = [
  { x: 0, y: 0 },
  { x: 100, y: 200 },
  { x: -500, y: -800 },
  { x: 123.4567, y: 987.6543 },
  { x: 1000000, y: -1000000 },
  { x: -999999.99, y: 999999.99 },
];

runTest("Screen -> World -> Screen roundtrip exactness across all viewports, zooms, and coordinates", () => {
  for (const vp of testViewports) {
    for (const zoom of testZooms) {
      const camera: Camera = { x: 150.5, y: -275.25, zoom };
      for (const pt of testPoints) {

```

```

    const world = screenToWorld(pt.x, pt.y, camera, vp);
    const screen = worldToScreen(world.x, world.y, camera, vp);
    const errX = Math.abs(screen.x - pt.x);
    const errY = Math.abs(screen.y - pt.y);
    assert.ok(
      errX < 1e-7 && errY < 1e-7,
      `Roundtrip failed for vp=${vp.width}x${vp.height}, zoom=${zoom}, pt=(${pt.x},${pt.y}). errX=${errX},
        errY=${errY}`
    );
  }
}
});

runTest("World -> Screen -> World roundtrip exactness across all viewports, zooms, and coordinates", () => {
  for (const vp of testViewports) {
    for (const zoom of testZooms) {
      const camera: Camera = { x: -500, y: 300, zoom };
      for (const pt of testPoints) {
        const screen = worldToScreen(pt.x, pt.y, camera, vp);
        const world = screenToWorld(screen.x, screen.y, camera, vp);
        const errX = Math.abs(world.x - pt.x);
        const errY = Math.abs(world.y - pt.y);
        assert.ok(
          errX < 1e-7 && errY < 1e-7,
          `World roundtrip failed for vp=${vp.width}x${vp.height}, zoom=${zoom}, pt=(${pt.x},${pt.y}). errX=${errX}, errY=${errY}`
        );
      }
    }
  }
});

runTest("Delta conversions worldToScreenDelta and screenToWorldDelta are exact inverses", () => {
  for (const zoom of testZooms) {
    const d = 42.195;
    const sDelta = worldToScreenDelta(d, zoom);
    const wDelta = screenToWorldDelta(sDelta, zoom);
    assert.ok(Math.abs(wDelta - d) < 1e-10, `Delta roundtrip failed for zoom=${zoom}`);
  }
});

// =====
// SUITE 2: zoomTo Cursor Fixed-Point Invariance & Sidebar Offset Invariance
// =====
console.log("\n--- Suite 2: zoomTo Cursor Invariance & Sidebar Offset ---");

function simulateZoomTo(
  currentCamera: Camera,
  targetZoom: number,
  cursorScreenX: number,
  cursorScreenY: number,
  vp: Viewport
): Camera {
  const newZoom = clamp(targetZoom, MIN_ZOOM, MAX_ZOOM);
  const worldX = (cursorScreenX - vp.width / 2) / currentCamera.zoom + currentCamera.x;
  const worldY = (cursorScreenY - vp.height / 2) / currentCamera.zoom + currentCamera.y;
  const newCameraX = worldX - (cursorScreenX - vp.width / 2) / newZoom;
  const newCameraY = worldY - (cursorScreenY - vp.height / 2) / newZoom;
  return { x: newCameraX, y: newCameraY, zoom: newZoom };
}

runTest("Cursor fixed point invariance: Point under cursor MUST NOT MOVE on screen during zoomTo", () => {
  const testCursors = [
    { x: 0, y: 0 }, // Top-left corner of viewport
    { x: 640, y: 400 }, // Exact center of 1280x800
    { x: 1280, y: 800 }, // Bottom-right corner of 1280x800
    { x: 123, y: 789 }, // Arbitrary screen point
    { x: -100, y: 900 }, // Off-screen cursor
  ];

  for (const vp of testViewports) {
    for (const startZoom of [0.1, 0.5, 1.0, 2.0, 4.0]) {
      for (const targetZoom of [0.05, 0.2, 0.8, 1.5, 3.0, 6.0, 8.0]) {
        for (const cursor of testCursors) {
          const cam: Camera = { x: 250, y: -180, zoom: startZoom };
          const worldBefore = screenToWorld(cursor.x, cursor.y, cam, vp);
          const newCam = simulateZoomTo(cam, targetZoom, cursor.x, cursor.y, vp);

```

```

    const screenAfter = worldToScreen(worldBefore.x, worldBefore.y, newCam, vp);

    const errX = Math.abs(screenAfter.x - cursor.x);
    const errY = Math.abs(screenAfter.y - cursor.y);
    assert.ok(
      errX < 1e-7 && errY < 1e-7,
      `Zoom cursor fixed point broken: moved by (${errX}, ${errY})`
    );
  }
}
}
});

runTest("Sidebar offset invariance: Opening/closing sidebar changes container width, container viewport preserves center", () => {
  const vpOpen: Viewport = { width: 1280, height: 864 };
  const vpClosed: Viewport = { width: 1536, height: 864 };

  const cam: Camera = { x: 0, y: 0, zoom: 1.0 };

  const worldPointOpen = screenToWorld(640, 432, cam, vpOpen);
  assert.equal(worldPointOpen.x, 0, "Container center maps to world (0,0) when sidebar is open");
  assert.equal(worldPointOpen.y, 0, "Container center maps to world (0,0) when sidebar is open");

  const zoomedCam = simulateZoomTo(cam, 2.0, 640, 432, vpOpen);
  assert.equal(zoomedCam.x, 0, "Camera center does not drift");
  assert.equal(zoomedCam.y, 0, "Camera center does not drift");
});

// =====
// SUITE 3: zoomToFit Math, Bounding Boxes & Viewport Extremes
// =====
console.log("\n--- Suite 3: zoomToFit Viewport Math & Edge Cases ---");

function simulateZoomToFit(
  elements: WhiteboardElement[],
  vp: Viewport
): Camera {
  if (elements.length === 0) {
    return { x: 0, y: 0, zoom: 1 };
  }
  let minX = Infinity, minY = Infinity, maxX = -Infinity, maxY = -Infinity;
  for (const el of elements) {
    if (el.type === "connection") continue;
    const b = getElementBounds(el);
    if (b.x < minX) minX = b.x;
    if (b.y < minY) minY = b.y;
    if (b.x + b.width > maxX) maxX = b.x + b.width;
    if (b.y + b.height > maxY) maxY = b.y + b.height;
  }
  if (!isFinite(minX)) {
    return { x: 0, y: 0, zoom: 1 };
  }
  const padding = 120;
  const contentW = maxX - minX + padding * 2;
  const contentH = maxY - minY + padding * 2;
  const cx = (minX + maxX) / 2;
  const cy = (minY + maxY) / 2;
  const zoom = clamp(Math.min(vp.width / contentW, vp.height / contentH), MIN_ZOOM, 2.0);
  return { x: cx, y: cy, zoom };
}

runTest("zoomToFit on empty board returns default camera (0,0,1)", () => {
  const cam = simulateZoomToFit([], { width: 1280, height: 800 });
  assert.deepStrictEqual(cam, { x: 0, y: 0, zoom: 1 });
});

runTest("zoomToFit with single small element centers content and caps zoom at 2.0", () => {
  const el: RectangleElement = {
    id: "r1",
    type: "rectangle",
    x: 500,
    y: 300,
    width: 100,
    height: 100,
    rotation: 0,
    opacity: 1,
  };

```

```
    strokeColor: "#000",
    fillColor: "#fff",
    fillStyle: "solid",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 0,
    updatedAt: 0,
  });
const vp: Viewport = { width: 1280, height: 800 };
const cam = simulateZoomToFit([el], vp);

assert.equal(cam.x, 550, "Camera x is center of element");
assert.equal(cam.y, 350, "Camera y is center of element");
assert.equal(cam.zoom, 2.0, "Zoom is capped at 2.0 for small content");

const centerScreen = worldToScreen(550, 350, cam, vp);
assert.equal(centerScreen.x, 640);
assert.equal(centerScreen.y, 400);
});

runTest("zoomToFit with huge spread scales down and never goes below MIN_ZOOM", () => {
  const el1: RectangleElement = {
    id: "r1",
    type: "rectangle",
    x: -50000,
    y: -50000,
    width: 100,
    height: 100,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    fillColor: "#fff",
    fillStyle: "solid",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 0,
    updatedAt: 0,
  };
  const el2: RectangleElement = {
    id: "r2",
    type: "rectangle",
    x: 50000,
    y: 50000,
    width: 100,
    height: 100,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    fillColor: "#fff",
    fillStyle: "solid",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 0,
    updatedAt: 0,
  };
  const vp: Viewport = { width: 1280, height: 800 };
  const cam = simulateZoomToFit([el1, el2], vp);

  assert.equal(cam.zoom, MIN_ZOOM, "Zoom clamps to MIN_ZOOM (0.05)");
  assert.equal(cam.x, 50, "Camera X is exact centroid");
  assert.equal(cam.y, 50, "Camera Y is exact centroid");
});

runTest("zoomToFit with 0x0 or degenerated viewport produces safe numbers (no NaN or Infinity)", () => {
  const el: RectangleElement = {
    id: "r1",
    type: "rectangle",
    x: 100,
    y: 100,
    width: 200,
    height: 200,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    fillColor: "#fff",
    fillStyle: "solid",
    dashStyle: "solid",
    strokeWidth: 2,
  };
  const vp: Viewport = { width: 0, height: 0 };
  const cam = simulateZoomToFit([el], vp);

  assert.equal(cam.zoom, MIN_ZOOM, "Zoom clamps to MIN_ZOOM (0.05)");
  assert.equal(cam.x, 100, "Camera X is exact centroid");
  assert.equal(cam.y, 100, "Camera Y is exact centroid");
});
```

```

    createdAt: 0,
    updatedAt: 0,
  };
  const cam0 = simulateZoomToFit([el], { width: 0, height: 0 });
  assert.ok(Number.isFinite(cam0.x), "cam.x must be finite");
  assert.ok(Number.isFinite(cam0.y), "cam.y must be finite");
  assert.ok(Number.isFinite(cam0.zoom), "cam.zoom must be finite");
  assert.equal(cam0.zoom, MIN_ZOOM, "0x0 viewport clamps to MIN_ZOOM");
});

// =====
// SUITE 4: Touch Gestures Math & Directional Pan Analysis
// =====
console.log("\n--- Suite 4: Touch Gestures, Pan Direction & Pinch Zoom ---");

function simulatePanCamera(camera: Camera, dx: number, dy: number): Camera {
  return {
    ...camera,
    x: camera.x - dx / camera.zoom,
    y: camera.y - dy / camera.zoom,
  };
}

runTest("Verified panCamera forward delta direction (1-Finger Pointer & 2-Finger Touch)", () => {
  const startCam: Camera = { x: 0, y: 0, zoom: 1.0 };
  const vp: Viewport = { width: 1000, height: 1000 };
  const worldPoint = { x: 0, y: 0 };

  const initialScreen = worldToScreen(worldPoint.x, worldPoint.y, startCam, vp);
  const lastX = 500;
  const currX = 600;

  // In WhiteboardCanvas.tsx:
  // Forward displacement: dx = cp.x - interaction.lastX = 600 - 500 = +100
  const dx = currX - lastX;

  const newCam = simulatePanCamera(startCam, dx, 0);
  const screenAfter = worldToScreen(worldPoint.x, worldPoint.y, newCam, vp);

  const screenDelta = screenAfter.x - initialScreen.x;
  const pointerDelta = currX - lastX;

  console.log(`[Pan Implementation] Pointer dragged: +${pointerDelta}px | Canvas moved on screen: +${screenDelta}px`);
  assert.equal(screenDelta, pointerDelta, "Canvas on screen must move in the exact same direction and magnitude as pointer drag (+100px)");

  // Verify at multiple zoom levels (0.25x, 2.0x, 4.0x)
  for (const zoom of [0.25, 0.5, 2.0, 4.0]) {
    const camZ: Camera = { x: 50, y: 50, zoom };
    const pDeltaX = 80;
    const pDeltaY = -40;
    const pBefore = worldToScreen(worldPoint.x, worldPoint.y, camZ, vp);
    const camAfter = simulatePanCamera(camZ, pDeltaX, pDeltaY);
    const pAfter = worldToScreen(worldPoint.x, worldPoint.y, camAfter, vp);

    assert.ok(
      Math.abs(pAfter.x - pBefore.x - pDeltaX) < 1e-7,
      `Screen X translation must equal pointer delta at zoom=${zoom}`
    );
    assert.ok(
      Math.abs(pAfter.y - pBefore.y - pDeltaY) < 1e-7,
      `Screen Y translation must equal pointer delta at zoom=${zoom}`
    );
  }
});

runTest("Pinch-to-zoom math preserves midpoint world position on screen", () => {
  const startCam: Camera = { x: 100, y: 100, zoom: 1.0 };
  const vp: Viewport = { width: 1280, height: 800 };

  const touch1 = { x: 500, y: 400 };
  const touch2 = { x: 700, y: 400 };
  const startDist = Math.hypot(touch1.x - touch2.x, touch1.y - touch2.y);
  const midScreen = { x: (touch1.x + touch2.x) / 2, y: (touch1.y + touch2.y) / 2 };

  const worldMidBefore = screenToWorld(midScreen.x, midScreen.y, startCam, vp);

```

```

const currDist = 400;
const scale = currDist / startDist;
const targetZoom = startCam.zoom * scale; // 2.0

const newCam = simulateZoomTo(startCam, targetZoom, midScreen.x, midScreen.y, vp);
const screenMidAfter = worldToScreen(worldMidBefore.x, worldMidBefore.y, newCam, vp);

assert.ok(
  Math.abs(screenMidAfter.x - midScreen.x) < 1e-7 &&
  Math.abs(screenMidAfter.y - midScreen.y) < 1e-7,
  "Pinch zoom did not preserve midpoint screen position"
);
});

// =====
// SUITE 5: Drag Erasing Across Multiple Element Types
// =====
console.log("\n--- Suite 5: Continuous Drag Erasing Hit Detection ---");

runTest("Continuous drag path intersects multiple elements and hit tests all of them", () => {
  const elDot: StrokeElement = {
    id: "s_dot",
    type: "stroke",
    x: 100,
    y: 100,
    rotation: 0,
    opacity: 1,
    points: [[0, 0, 0.5]],
    color: "#000",
    strokeWidth: 4,
    strokeStyle: "pen",
    createdAt: 0,
    updatedAt: 0,
  };

  const elRect: RectangleElement = {
    id: "s_rect",
    type: "rectangle",
    x: 200,
    y: 80,
    width: 60,
    height: 60,
    rotation: 0,
    opacity: 1,
    strokeColor: "#000",
    fillColor: "#fff",
    fillStyle: "solid",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 0,
    updatedAt: 0,
  };

  const elText: TextElement = {
    id: "s_text",
    type: "text",
    x: 350,
    y: 90,
    width: 100,
    height: 40,
    rotation: 0,
    opacity: 1,
    text: "Hello",
    color: "#000",
    fontSize: 16,
    fontFamily: "sans-serif",
    fontWeight: 400,
    align: "left",
    createdAt: 0,
    updatedAt: 0,
  };

  const elements: WhiteboardElement[] = [elDot, elRect, elText];

  const dragSteps = [
    { x: 50, y: 100 },
    { x: 100, y: 100 }, // hits dot
    { x: 150, y: 100 },
  ];

```

```

    { x: 230, y: 100 }, // hits rect
    { x: 300, y: 100 },
    { x: 380, y: 100 }, // hits text
    { x: 450, y: 100 },
  ];

  const erasedIds = new Set<string>();

  for (const pt of dragSteps) {
    for (const el of elements) {
      if (hitTestElement(el, pt.x, pt.y)) {
        erasedIds.add(el.id);
      }
    }
  }

  assert.ok(erasedIds.has("s_dot"), "Single dot must be hit during drag-erase");
  assert.ok(erasedIds.has("s_rect"), "Rectangle must be hit during drag-erase");
  assert.ok(erasedIds.has("s_text"), "Text must be hit during drag-erase");
  assert.equal(erasedIds.size, 3, "All 3 elements along drag path must be erased");
});

console.log("\n=====");
console.log(`CHALLENGER STRESS TEST SUMMARY: Passed: ${passedCount}, Failed: ${failedCount}`);
console.log("=====\\n");

=====
// === END FILE: tests/adversarial_zoom_gestures.test.ts ===
=====

=====
// === START FILE: tests/capture.test.ts ===
// File 137 of 194 | Lines: 361 | Size: 15960 bytes
=====

/**
 * Capture (screenshot of an area).
 * - The area is measured in the canvas' world frame, trimmed to the paper in
 *   a notebook, and may span two sheets.
 * - The picture is built by the export engine: paper, ruling, PDF page and
 *   elements per the chosen background; a freeform loop masks it.
 * - Adding it to a document is one undoable step, taken from where you are.
 * - Screenshot mode only borrows the SELECT gestures.
 */

import assert from "node:assert/strict";
import {
  boundsOfPoints,
  buildCaptureSvg,
  captureFilename,
  captureKeep,
  capturePixelSize,
  clampCaptureRegion,
  defaultCaptureScale,
  intersectRect,
  polygonArea,
  rectFromCorners,
  selectionCaptureRegion,
  sheetsInRegion,
  type CaptureSource,
} from "../src/lib/whiteboard/capture";
import { placeCaptureBeside, placeCaptureOnPage } from "../src/lib/whiteboard/placement";
import { PAGE_GAP, getActivePage, getPages, getBoardElements, pageSizeOf } from "../src/lib/whiteboard/notebook";
import { appClipForPaste, clearAppClip, putAppClipImage } from "../src/lib/whiteboard/appClipboard";
import { screenshotShapeFor } from "../src/store/capture";
import { useWhiteboard } from "../src/store/whiteboard";
import type { Board, NotebookPage, WhiteboardElement } from "../src/lib/whiteboard/types";

const NL = String.fromCharCode(10);
let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  }

```

```

    } catch (err) {
      failed++;
      console.error(` ✖ ${name}`);
      console.error(`${(err as Error).message}`);
    }
  }

function rect(id: string, x: number, y: number, w = 20, h = 20): WhiteboardElement {
  return {
    id,
    type: "rectangle",
    x,
    y,
    width: w,
    height: h,
    rotation: 0,
    opacity: 1,
    strokeColor: "#112233",
    fillColor: "transparent",
    fillStyle: "transparent",
    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: 0,
    updatedAt: 0,
  } as unknown as WhiteboardElement;
}

function page(id: string, elements: WhiteboardElement[] = [], extra: Partial<NotebookPage> = {}): NotebookPage {
  return { id, elements, createdAt: 0, updatedAt: 0, ...extra };
}

function notebook(pages: NotebookPage[], activeIndex = 0): Board {
  return {
    id: "nb",
    name: "Physics notes",
    kind: "notebook",
    pageSize: "a4",
    pages,
    activePageId: pages[activeIndex].id,
    elements: [],
    background: { pattern: "lines", size: 24 },
    createdAt: 0,
    updatedAt: 0,
  } as unknown as Board;
}

function canvas(elements: WhiteboardElement[]): Board {
  return {
    id: "wb",
    name: "Board",
    elements,
    background: { pattern: "dots", size: 24 },
    createdAt: 0,
    updatedAt: 0,
  } as unknown as Board;
}

const A4 = pageSizeOf(notebook([page("p")]));
const STRIDE = A4.height + PAGE_GAP;

console.log(NL + "■■ Capture: region maths ■■");

test("a box is the same whichever corner you start from", () => {
  assert.deepEqual(rectFromCorners({ x: 50, y: 80 }, { x: 10, y: 20 }), { x: 10, y: 20, width: 40, height: 60 });
});

test("a loop's bounds ignore non-finite samples, and a scribble has no area", () => {
  assert.deepEqual(boundsOfPoints([
    { x: 1, y: 2 }, { x: NaN, y: 0 }, { x: 5, y: 9 }
  ]), { x: 1, y: 2, width: 4, height: 7 });
  assert.equal(boundsOfPoints([]), null);
  assert.equal(polygonArea([
    { x: 0, y: 0 }, { x: 10, y: 0 }, { x: 10, y: 10 }, { x: 0, y: 10 }
  ]), 100);
  assert.equal(polygonArea([
    { x: 0, y: 0 }, { x: 5, y: 5 }, { x: 10, y: 10 }
  ]), 0);
});

test("rectangles that only touch do not intersect", () => {
  assert.equal(intersectRect({ x: 0, y: 0, width: 10, height: 10 }, { x: 10, y: 0, width: 5, height: 5 }), nul

```